

HỆ THỐNG SLAM ĐẦY ĐỦ



TÓM TẮT

Phần B cho các mảnh rời. Phần C là chỗ chúng trở thành một *hệ thống* chạy được lâu dài. Sáu chương xử lý sáu câu hỏi mà không chương nào ở Phần B trả lời được: bản đồ được quản lý thế nào khi nó lớn lên vô hạn và thế giới thì thay đổi (chương 13); hiệu chuẩn — vốn được coi là hằng số suốt Phần A và B — thực ra là một nhánh của chính SLAM (chương 14); thêm cảm biến vào thế nào cho đúng (chương 15); làm sao có một bản đồ dày chứ không chỉ một đám mây điểm thưa (chương 16); học máy đóng góp ở đâu và ở đâu thì không (chương 17); và làm sao biết hệ thống của mình có thật sự tốt hơn không (chương 18). Nếu chỉ nhớ một điều: *chương 14 là chỗ luận đề trung tâm của cây được chứng minh bằng tay* — ta đi từ ô “hiệu chuẩn” sang ô “SLAM” trong bảng ở 00-map bằng cách thả dần các biến ra, và không có gì trong bộ giải phải thay đổi.

Phần này khác Phần B ở một điểm về bản chất. Phần B trả lời các câu hỏi có lời giải *đúng hoặc sai*: thuật toán này khớp điểm tốt hơn thuật toán kia, kiến trúc này chi phí thấp hơn kiến trúc kia. Phần C trả lời các câu hỏi có lời giải *phù hợp hoặc không phù hợp*: giữ bao nhiêu keyframe là đủ, cập nhật bản đồ theo chu kỳ nào, thêm cảm biến nào cho ứng dụng nào. Đó là các câu hỏi kỹ thuật hệ thống, và chúng phụ thuộc vào ràng buộc sản phẩm chứ không chỉ vào toán học.

Đây cũng là lý do phần này ít tài liệu học thuật hơn hẳn. Một bài báo có thể chứng minh thuật toán khớp điểm mới tốt hơn; rất khó chứng minh một chính sách quản lý bản đồ tốt hơn, vì “tốt hơn” phụ thuộc vào việc robot chạy bao lâu, trong không gian nào, và ai trả tiền cho bộ nhớ. Hệ quả thực tế là kiến thức trong phần này chủ yếu nằm trong mã nguồn của vài hệ thống trưởng thành chứ không nằm trong bài báo — và đó là lý do E/23 khuyến nghị đọc mã ORB-SLAM3 và maplab chứ không chỉ đọc bài của họ.

Sáu chương có vai trò rất khác nhau. Chương 13 là chương bị coi nhẹ nhất trong toàn lĩnh vực và có ảnh hưởng lớn nhất tới sản phẩm; nó nên được đọc kỹ ngay cả khi bạn chưa thấy vì sao. Chương 14 là chương hợp nhất của cả cây — nếu chỉ đọc một chương của Phần C thì đọc nó. Chương 15 là chương ngắn nhất vì phần lớn nội dung nằm ở các cây khác, nhưng nó chứa một nguyên tắc quan trọng về cách thêm nguồn thông tin. Chương 16 và 17 là hai chương nóng nhất về mặt nghiên cứu và cũng là hai chương hết hạn nhanh nhất. Chương 18 khép phần lại bằng câu hỏi “làm sao biết mình đúng”, và nó là chương mà mọi chương trước đều nợ.

Bảng 1. Bản đồ Phần C: các chương, nội dung, và mức đầu tư đề xuất.

Ch.	Thư mục	Nội dung	Mức
13	13-quan-ly-ban-do	Chọn và loại keyframe; loại landmark; đồ thị đồng quan sát và essential graph; cửa sổ bản đồ cục bộ; làm thưa bản đồ; ghép nhiều phiên và Atlas; cập nhật dài hạn; nén và tuần tự hoá; quyền riêng tư	● ● ●
14	14-hieu-chuan-nhu-mc	Hiệu chuẩn nội từ target; tự hiệu chuẩn; ngoại stereo và camera-IMU; hiệu chuẩn trực tuyến có cổng observability; tự đánh giá chất lượng; suy biến trong hiệu chuẩn	● ● ●
15	15-hop-nhat-da-cam-t	Nguyên tắc chung để thêm một nguồn; IMU, odometry bánh xe, LiDAR, GNSS, marker, UWB; ràng buộc nonholonomic; căn chỉnh hệ toàn cục	● ●
16	16-dense-mapping-va-	Stereo và MVS; TSDF và voxel hashing; surfel và mesh; octree và ESDF cho planning; NeRF-SLAM; 3D Gaussian Splatting; đánh đổi dày so với thưa	● ●
17	17-hoc-may-trong-sla	Front-end học được; prior độ sâu; DROID-SLAM và luồng học được; bất định học được; foundation model 3D; SLAM khả vi; semantic SLAM; vì sao hybrid là hướng đúng	● ●
18	18-danh-gia-va-phuor	ATE và RPE, căn chỉnh đúng số bậc; drift theo đoạn; NEES; đánh giá reloc và loop closure; ablation có kỷ luật; mô phỏng; ý nghĩa thống kê; đánh giá ở đuôi phân bố	● ● ●

Chương 14 là chương nên đọc trước nếu chỉ có thời gian cho một chương — nó là chỗ luận đề trung tâm của cây được chứng minh, và nó thay đổi cách bạn nhìn cả Phần A. Chương 13 và 18 là hai chương có tỉ lệ giá trị trên độ nổi tiếng cao nhất: cả hai đều bị coi nhẹ trong tài liệu và cả hai đều quyết định hệ thống có dùng được ngoài phòng thí nghiệm hay không. Chương 15 ngắn và chủ yếu trở sang các cây khác; đọc nó để lấy nguyên tắc, rồi sang `../IMU_Fusion/` và `../marker/` khi cần chi tiết. Chương 16 và 17 nên đọc lướt để biết bản đồ lĩnh vực, rồi quay lại khi thật sự cần — chúng là hai chương hết hạn nhanh nhất và được chốt tại tháng 9/2026.

Quản lý bản đồ (Map Management)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (phần bù Schur — đồ thị đồng quan sát có gốc rễ toán học ở đó, và chi phí bất đối xứng giữa tư thế và điểm là gốc của mọi chính sách trong chương này); B/09-paradigm-uoc-luong-back-end (kiến trúc nhiều luồng); B/11-loop-closure-va-nhat-quan-toan-cuc (vấn đề hoà nhập); B/12-relocalization (bản đồ lỗi thời).

TÓM TẮT

Đây là chương bị coi nhẹ nhất trong toàn lĩnh vực và có ảnh hưởng lớn nhất tới việc một hệ SLAM có dùng được trong sản phẩm hay không. Lý do bị coi nhẹ thì dễ hiểu: không ai viết được một bài báo chứng minh chính sách chọn keyframe của mình tốt hơn, vì “tốt hơn” phụ thuộc vào robot chạy bao lâu, ở đâu, và ai trả tiền cho bộ nhớ. Lý do quan trọng thì cũng dễ hiểu: mọi thứ ở Phần B giả định có một bản đồ, và không chương nào ở đó nói bản đồ ấy được tạo ra, cắt tỉa, lưu trữ, cập nhật và chia sẻ thế nào. Nội dung có bốn phần. Một: chính sách chọn và loại — keyframe và landmark — với lập luận rằng cả hai chính sách đều bắt nguồn từ một sự kiện toán học duy nhất ở A/04. Hai: cấu trúc đồ thị của bản đồ, và vì sao đồ thị đồng quan sát không phải một heuristic mà là cấu trúc khác không của một ma trận. Ba: bản đồ sống lâu — nhiều phiên, ghép bản đồ, và bài toán chưa giải là thế giới thay đổi. Bốn: các ràng buộc kỹ thuật thật — nén, tuần tự hoá, phiên bản, và quyền riêng tư. Nếu chỉ nhớ một điều: *tư thế đắt, điểm rẻ* — và gần như mọi chính sách trong chương này là hệ quả của câu đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **keyframe** — một khung hình được giữ lại trong bản đồ như một biến tư thế, khác với các khung chỉ dùng để tracking rồi bỏ. Việc chọn khung nào thành keyframe là quyết định quan trọng nhất trong chương này.
- **đồ thị đồng quan sát (covisibility graph)** — đồ thị có đỉnh là keyframe và cạnh nối hai keyframe cùng nhìn thấy đủ nhiều landmark chung. Nó xác định “vùng lân cận” của một keyframe trong bản đồ.
- **essential graph** — một đồ thị con thừa hơn của đồ thị đồng quan sát, giữ lại cây bao trùm cộng các cạnh mạnh nhất và các cạnh loop closure. Dùng cho tối ưu pose graph, nơi ta cần liên thông chứ không cần mọi cạnh.
- **bản đồ cục bộ (local map)** — tập con của bản đồ mà tracking đang dùng ở thời điểm hiện tại — thường là các keyframe lân cận trong đồ thị đồng quan sát cùng landmark của chúng.
- **làm thưa bản đồ (map sparsification)** — chọn một tập con landmark để giữ sao cho khả năng định vị không giảm đáng kể trong khi kích thước giảm nhiều lần. Bài toán tối ưu tổ hợp, giải bằng heuristic.
- **Atlas** — kiến trúc giữ *nhiều* bản đồ độc lập cùng lúc thay vì một, với cơ chế ghép khi phát hiện chồng lấn. Cách xử lý mất bám tinh tế hơn so với vứt bản đồ và làm lại.
- **map inversion** — tấn công tái tạo lại ảnh gốc từ một bản đồ gồm điểm 3D và descriptor. Đây là một vấn đề quyền riêng tư thật, không phải lo xa.

CÂU HỎI MỞ ĐẦU

- Q1. Giữ nhiều keyframe hơn cho bản đồ chi tiết hơn và tracking bền hơn. Vì sao mọi hệ thống lại tiết kiệm keyframe một cách gắt gao trong khi hào phóng với landmark?
- Q2. Đồ thị đồng quan sát nghe như một cấu trúc dữ liệu tiện lợi do ai đó nghĩ ra. Nêu nguồn gốc toán học của nó, và nói vì sao điều đó có nghĩa là nó không tùy tiện.
- Q3. Robot chạy tám tiếng mỗi ngày trong cùng một toà nhà, ba trăm ngày một năm. Nêu ba thứ sẽ hỏng nếu bạn chỉ đơn giản tiếp tục thêm vào cùng một bản đồ, và nói cái nào khó xử lý nhất.
- Q4. Hệ thống mất bám. Lựa chọn hiển nhiên là vứt bản đồ và khởi tạo lại. Nêu một lựa chọn tốt hơn và nói nó đòi hỏi gì về kiến trúc.
- Q5. Bản đồ của bạn chỉ gồm tọa độ 3D và descriptor nhị phân — không có ảnh nào. Vì sao đó *không* đủ để nói nó không chứa thông tin riêng tư?

1.1 Đặt vấn đề

Mọi chương ở Phần B giả định có một bản đồ. Chúng nói làm sao tạo ra nó (B/10), làm sao dùng nó (B/12), làm sao sửa nó khi đóng vòng (B/11). Không chương nào nói nó *lớn lên thế nào và ta làm gì với việc đó*.

Bài toán gốc phát biểu đơn giản: một hệ SLAM chạy liên tục thêm keyframe và landmark vào bản đồ. Nếu không làm gì, bản đồ tăng tuyến tính theo thời gian, và ba thứ hỏng theo. *Bộ nhớ cạn. Chi phí tính toán tăng* — bundle adjustment cục bộ chậm dần, truy hồi loop closure chậm dần. Và, ít rõ hơn nhưng nghiêm trọng hơn, *chất lượng giảm*: một bản đồ đầy các keyframe gần trùng nhau và các landmark chất lượng kém không tốt hơn một bản đồ gọn gàng — nó tệ hơn, vì tỉ lệ khớp sai tăng theo số ứng viên (B/12 mục 2B).

Ai gặp bài toán này? Chỉ những người chạy hệ thống lâu hơn một chuỗi benchmark. Đây là lý do cấu trúc khiến chương này thiếu tài liệu: EuRoC dài hai phút, KITTI dài vài phút. Không bộ dữ liệu chuẩn nào dài tám tiếng, nên không ai cần giải bài toán này để đăng bài — và mọi người cần giải nó để bán sản phẩm.

Trước khi có các chính sách có nguyên tắc, cách làm phổ biến là ngưỡng thủ công: “tạo keyframe nếu đi quá x mét hoặc xoay quá y độ”. Cách đó hoạt động và nó là cách đúng để bắt đầu, nhưng nó hỏng ở hai chỗ. Nó không thích ứng với cảnh — x mét trong một hành lang hẹp khác hẳn x mét ngoài bãi đỗ xe. Và nó không có cơ chế *loại bỏ*, nên bản đồ vẫn tăng vô hạn, chỉ chậm hơn.

Chương này lập luận rằng phần lớn các chính sách đúng đều suy ra từ một sự kiện toán học duy nhất, đã được thiết lập ở A/04 mục 4C: *chi phí bundle adjustment tăng theo lũy thừa ba của số tư thế nhưng gần tuyến tính theo số điểm*. Tư thế đắt, điểm rẻ. Một khi nắm được điều đó, các chính sách không còn là heuristic tùy tiện mà là hệ quả kinh tế.

TRỰC GIÁC

Hình dung bản đồ như một cuốn album ảnh kèm chú thích, và bạn phải mang nó theo suốt đời.

Mỗi trang album là một keyframe — một chỗ bạn từng đứng, kèm danh sách những gì nhìn thấy từ đó. Mỗi chú thích là một landmark.

Bây giờ một sự kiện quyết định: *thêm một trang thì đắt, thêm một chú thích vào trang có sẵn thì rẻ*. Vì mỗi khi bạn muốn chỉnh lại album cho nhất quán, bạn phải so từng trang với từng trang khác — chi phí tăng theo bình phương số trang. Còn chú thích thì chỉ thuộc về một trang, nên thêm bao nhiêu cũng không làm việc so trang khó hơn.

Từ sự kiện đó suy ra gần hết chính sách. Thêm trang mới chỉ khi nó cho thấy thứ mà các trang cũ không thấy — nếu bạn chỉ nhích sang một bước thì trang mới gần như trùng trang cũ, không đáng. Ngược lại, ghi thật nhiều chú thích vào mỗi trang, vì chúng gần như miễn phí và chúng làm trang đó dễ nhận ra lại hơn.

Rồi có ba chuyện xảy ra theo thời gian, và chúng khó dần.

Album dày lên. Sau vài năm nó nặng quá không mang nổi. Bạn phải xé bớt trang — nhưng xé trang nào? Trang ở chỗ bạn ít khi quay lại, hoặc trang mà các trang lân cận đã ghi hết những gì nó ghi.

Bạn có nhiều album. Mỗi chuyến đi một cuốn. Có lúc bạn nhận ra hai cuốn ghi cùng một nơi, và bạn muốn ghép chúng.

Thế giới thay đổi. Cái nhà trong ảnh đã bị phá. Chú thích của bạn vẫn nói nó ở đó. Bạn không có cách nào biết cho tới khi quay lại — và khi quay lại, bạn không phân biệt được “mình nhớ nhầm chỗ” với “chỗ này đã đổi”.

Chuyện thứ ba là chuyện khó nhất, và nó chưa có lời giải tốt.

1.2 Chính sách chọn và loại

A. Vì sao tư thế đắt còn điểm rẻ

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là nền của cả chương.

Từ A/04 mục 4C: sau phần bù Schur, hệ còn lại có kích thước $6m \times 6m$ với m là số tư thế; chi phí giải nó là $O(m^3)$ trong trường hợp xấu (thực tế tốt hơn nhờ độ thưa, nhưng vẫn siêu tuyến tính). Việc khử n landmark tốn $O(n)$ vì khối V là khối chéo.

Hệ quả kinh tế trực tiếp: *gấp đôi số landmark làm chi phí tăng gấp đôi; gấp đôi số keyframe làm chi phí tăng nhiều lần hơn thế.* Với một ngân sách tính toán cố định, tối ưu là giữ ít keyframe và nhiều landmark trên mỗi keyframe.

Có một lý do thứ hai, độc lập và cũng quan trọng: *thông tin*. Hai keyframe cách nhau một bước chân nhìn thấy gần như cùng một thứ từ gần như cùng một chỗ, nên keyframe thứ hai thêm rất ít thông tin — các hàng Jacobian của nó gần tuyến tính phụ thuộc với hàng của keyframe thứ nhất. Nó tốn chi phí đầy đủ mà cho lợi ích gần bằng không. Ngược lại, thêm một landmark mới nhìn từ một hướng mới thật sự thêm ràng buộc.

Hai lý do ấy hội tụ về cùng một chính sách, và đó là lý do nó vững chắc.

B. Tiêu chí tạo keyframe

Ba tiêu chí, và điều đáng chú ý là cả ba đều là *tiêu chí thông tin*, không phải tiêu chí khoảng cách.

Một — đủ parallax so với keyframe gần nhất. Đây là tiêu chí đúng thay cho “đi quá x mét”, vì nó tự thích ứng với độ sâu cảnh: trong hành lang hẹp, một mét cho nhiều parallax; ngoài bãi đỗ xe, một mét cho rất ít. Cách đo: parallax trung vị của các landmark đang theo dõi, đúng đại lượng đã dùng ở B/10 mục 4B.

Hai — tỉ lệ theo dõi giảm. Nếu số landmark cũ còn nhìn thấy được đã giảm dưới một tỉ lệ nào đó, ta đang rời khỏi vùng mà bản đồ hiện tại mô tả, và cần một keyframe để neo vùng mới.

Ba — đã đủ xa về thời gian. Một điều kiện an toàn: dù hai tiêu chí trên chưa kích hoạt, vẫn tạo keyframe sau một số khung nhất định, để tránh trường hợp hệ thống trôi mà không có neo nào.

Tôi không đưa ra các giá trị ngưỡng cụ thể, và đó là có chủ ý. Chúng phụ thuộc vào σ của front-end (B/07), vào độ sâu điển hình của cảnh, vào tốc độ di chuyển, và vào ngân sách tính toán. Cách

đúng là *tính* chúng từ những đại lượng đó — ngưỡng parallax tính theo $B/10$ mục 2C — rồi hiệu chỉnh bằng đo đạc trên dữ liệu của chính mình. Chép một hằng số từ mã nguồn của hệ khác là cách nhanh nhất để có một con số kỳ bí mà ba năm sau không ai biết vì sao.

C. Loại bỏ: phần bị bỏ quên

Tạo thì ai cũng làm; loại thì ít hơn nhiều. Nhưng không có cơ chế loại thì bản đồ vẫn tăng vô hạn.

Loại keyframe dư thừa. Tiêu chí của ORB-SLAM [1] đáng học vì nó đúng nguyên tắc: loại một keyframe nếu phần lớn landmark của nó *cũng* được nhìn thấy bởi ít nhất ba keyframe khác. Lập luận: keyframe ấy không neo landmark nào một cách độc nhất, nên loại nó không mất thông tin về cấu trúc. Chú ý điều này khác hẳn với “loại keyframe cũ” — nó loại theo *mức dư thừa* chứ không theo tuổi, nên một keyframe cũ ở một góc ít ai qua sẽ được giữ, còn mười keyframe mới chồng lên nhau ở một chỗ sẽ bị tía.

Loại landmark chất lượng kém. Một landmark bị loại nếu nó được quan sát quá ít lần so với số lần *lẽ ra* được quan sát (tức nó nằm trong tầm nhìn nhưng không khớp được — dấu hiệu nó là một khớp sai hoặc một vật động), hoặc nếu phần dư của nó liên tục lớn, hoặc nếu bất định của nó không giảm dù đã quan sát nhiều lần (dấu hiệu suy biến hình học).

Điều đáng nhấn mạnh về chính sách loại landmark: *ngưỡng nên gặt lúc mới tạo và nới dần theo tuổi*. Một landmark vừa được triangulate chưa được kiểm chứng và nên bị loại dễ dàng; một landmark đã sống sót qua hàng trăm quan sát là tài sản quý và nên được bảo vệ. Chính sách này rẻ và hiệu quả bất ngờ, vì phần lớn landmark sai bị loại trong vài khung đầu.

1.3 Cấu trúc đồ thị của bản đồ

A. Đồ thị đồng quan sát và gốc rễ toán học của nó

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những kết nối đẹp nhất trong cây.

Đồ thị đồng quan sát nối hai keyframe khi chúng cùng nhìn thấy đủ nhiều landmark chung. Nó trông như một cấu trúc dữ liệu tiện lợi do ai đó nghĩ ra vì nó hữu ích.

Nó không phải vậy. Quay lại A/04 mục 4C: sau phần bù Schur, ma trận còn lại là

$$S = U - WV^{-1}W^T,$$

và khối (i, i') của S khác không *đúng khi* tồn tại một landmark j mà cả tư thế i lẫn tư thế i' đều quan sát — vì chỉ lúc đó cả W_{ij} lẫn $W_{i'j}$ mới khác không.

Nói cách khác: *đồ thị đồng quan sát chính xác là cấu trúc khác không của S* . Nó không phải một heuristic mà là hình ảnh của một ma trận. (Cửa kiểm chứng a: suy trực tiếp từ công thức Schur.)

Điều này có ba hệ quả thực dụng. *Một:* hai keyframe nối nhau trong đồ thị chính là hai keyframe mà bộ giải *phải* xử lý cùng nhau; tách chúng ra hai bài toán con là làm mất một khối khác không. *Hai:* chọn bản đồ cục bộ cho tracking bằng cách lấy lân cận trong đồ thị đồng quan sát là đúng theo nghĩa toán học — đó chính là tập biến có tương tác trực tiếp. *Ba:* mật độ của đồ thị quyết định chi phí, nên chính sách keyframe ở mục 2 cũng chính là chính sách kiểm soát chi phí.

B. Essential graph và vì sao cần một đồ thị thưa hơn

Đồ thị đồng quan sát đầy đủ có thể rất đặc: trong một căn phòng nhỏ, mọi keyframe nhìn thấy mọi landmark, nên mọi cặp đều nối nhau. Với tối ưu pose graph ở B/11, điều đó làm bài toán đắt không cần thiết.

Essential graph là một đồ thị con giữ lại: cây bao trùm (đảm bảo liên thông), các cạnh có số landmark chung cao nhất, và *mọi* cạnh loop closure. Lập luận: với pose graph, mục tiêu là phân phối sai số trên toàn quỹ đạo, và điều đó cần *liên thông* chứ không cần mọi cạnh. Các cạnh yếu đóng góp ít và có thể bỏ.

Đây là một ví dụ của một khuôn mẫu chung: *dùng biểu diễn đầy đủ cho tối ưu cục bộ, biểu diễn thưa cho tối ưu toàn cục*. Cùng khuôn mẫu xuất hiện ở việc chia luồng ở mục 4A.

C. Làm thưa bản đồ

Với bản đồ dùng cho localization lâu dài, ta không cần *mọi* landmark — ta cần một tập con đủ để định vị ở mọi chỗ với độ chính xác yêu cầu. Bài toán chọn tập con ấy là một bài toán tối ưu tổ hợp, và nó có một cấu trúc đẹp: ta muốn tối đa hoá *độ phủ* (mọi vùng đều có đủ landmark nhìn thấy được) với ràng buộc về kích thước.

Dymczyk và cộng sự [2] đề nghị chọn theo điểm số kết hợp số lần quan sát, độ bền qua các phiên, và độ phủ không gian. Kết quả là một bản đồ nhỏ hơn nhiều lần mà khả năng định vị giảm ít.

Điều đáng nhớ về mặt nguyên tắc: tiêu chí chọn nên là *độ phủ*, không phải *chất lượng trung bình*. Một trăm landmark xuất sắc tụ ở một góc phòng kém hữu ích hơn ba mươi landmark trung bình trải khắp phòng — cùng lập luận với phân bố đặc trưng ở B/07 mục 4A, ở một quy mô khác.

1.4 Bản đồ sống lâu

A. Kiến trúc nhiều luồng

Một hệ SLAM thật có ít nhất ba nhịp độ khác nhau, và cố ép chúng vào một luồng là một lỗi thiết kế.

Tracking chạy ở tần số khung hình — 30 Hz nghĩa là 33 ms cho mỗi khung, và nó *không bao giờ được chặn*, vì camera vẫn gửi khung và robot vẫn di chuyển. *Local mapping* chạy khi có keyframe mới — vài lần mỗi giây, và có thể mất vài trăm mili giây. *Loop closing* chạy hiếm nhưng lâu — có thể vài giây.

Ba nhịp độ ấy chia nhau một bản đồ, và đó là nguồn của phần lớn độ phức tạp kỹ thuật trong một hệ SLAM thật. Ba vấn đề cụ thể: tracking đọc bản đồ trong khi local mapping ghi vào nó; loop closing sắp sửa dịch chuyển các keyframe mà tracking đang dùng; và khi loop closing hoàn thành, các khung đã xử lý trong lúc đó cần được hoà nhập.

CẠM BẮY

Các vấn đề ở mục 4A không xuất hiện trong bài báo nhưng chiếm phần lớn thời gian kỹ thuật của một hệ SLAM thật.

Một bài báo mô tả thuật toán như thế nó chạy tuần tự: tracking, rồi mapping, rồi loop closing. Mã nguồn thật thì đầy hàng đợi, khoá, bản sao, và cờ trạng thái — và phần lớn lỗi khó tái hiện nhất nằm ở đó chứ không nằm trong toán.

Ba hệ quả thực hành. *Một*: đọc mã ORB-SLAM3 hay VINS-Fusion để học *kiến trúc* có giá trị cao hơn đọc bài báo của chúng, và E/23 khuyến nghị điều này. *Hai*: khi thiết kế, hãy quyết định sớm ai sở hữu bản đồ và ai được ghi vào nó — sửa điều này về sau rất đắt. *Ba*: **tính tất định** là một yêu cầu thật chứ không phải một điều xa xỉ; nếu hệ thống cho kết quả khác nhau giữa hai lần chạy trên cùng dữ liệu vì thứ tự luồng khác nhau, bạn không debug được một lỗi từ hiện trường. Đây là chủ đề của D/20 và là một trong những lý do chính đáng nhất để chấp nhận chạy chậm hơn một chút.

B. Nhiều phiên và Atlas

Đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Khi hệ thống mất bám, lựa chọn hiển nhiên là vứt bản đồ và khởi tạo lại (B/10 mục 4C). Nó lãng phí: bản đồ cũ có thể hoàn toàn tốt, chỉ là ta tạm thời không biết mình đang ở đâu trong nó.

Atlas [3] làm khác: khi mất bám, bắt đầu một bản đồ *mới* và giữ bản đồ cũ ở trạng thái không hoạt động. Hệ thống tiếp tục chạy trên bản đồ mới. Song song, nó liên tục thử relocalize vào các bản đồ cũ; nếu thành công, nó *ghép* hai bản đồ bằng phép biến đổi tìm được và tiếp tục với bản đồ hợp nhất.

Ba lợi ích. *Không mất dữ liệu*: công sức lập bản đồ trước đó được giữ. *Phục hồi mềm*: hệ thống vẫn hoạt động ngay cả khi chưa tìm lại được. *Nhiều phiên tự nhiên*: bản đồ từ hôm qua chỉ là một bản đồ không hoạt động trong atlas, và cùng cơ chế ghép hoạt động.

Yêu cầu về kiến trúc: bản đồ phải là một đối tượng *tách rời và ghép được* — có hệ quy chiếu riêng, có thể serialize, và có phép toán ghép. Nhiều hệ thống có bản đồ gắn chặt với trạng thái toàn cục và không làm được điều này. Đây là quyết định thiết kế nên đưa ra sớm.

C. Thế giới thay đổi: bài toán chưa giải

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và là chỗ tôi phải thành thật rằng lĩnh vực chưa có lời giải tốt.

Ba thứ hòng khi chạy lâu trong cùng một môi trường, xếp theo mức khó.

Dễ — *kích thước*. Giải được bằng các chính sách ở mục 2 và mục 3C. Không tầm thường nhưng có lời giải.

Khó vừa — *thay đổi về ngoài theo chu kỳ*. Ánh sáng ban ngày và ban đêm, rèm mở và đóng. Lời giải một phần: lưu *nhiều biểu diễn* cho cùng một địa điểm — mỗi điều kiện một tập descriptor — và chọn theo điều kiện hiện tại. Tồn bộ nhớ và cần một cách phân loại điều kiện, nhưng khả thi. OpenLORIS [4] là một trong ít bộ dữ liệu đo được điều này.

Khó nhất — *thay đổi cấu trúc*. Đồ đạc được sắp lại, một bức tường được dựng, một phòng được cải tạo. Đây là bài toán chưa giải, và lý do nó khó nằm ở một chỗ cụ thể: *không có cách nào phân biệt “bản đồ sai” với “tôi ở sai chỗ” từ bên trong hệ thống*. Cả hai cho cùng triệu chứng — landmark không khớp — và hai kết luận đòi hai hành động ngược nhau: cập nhật bản đồ, hay relocalize.

Các cách tiếp cận hiện có đều là thỏa hiệp. *Quên theo thời gian*: hạ trọng số landmark cũ dần — đơn giản, nhưng nó quên cả những thứ không đổi. *Đếm phiếu*: landmark nào liên tục không khớp qua nhiều lần đi qua thì loại — tốt hơn, nhưng cần nhiều lần đi qua và cần chắc rằng ta thật sự đi qua đúng chỗ, tức cần relocalization đáng tin, tức vòng tròn. *Lưu nhiều phiên bản*: giữ cả bản đồ cũ và mới — tồn bộ nhớ và chuyển bài toán thành việc chọn phiên bản.

Không cách nào giải quyết trường hợp khó nhất, và D/22 xếp đây là một trong tám khoảng trống lớn nhất. Tôi cho rằng nó là khoảng trống *đau nhất* trong triển khai thực tế, vì nó là điểm khác biệt lớn nhất giữa một hệ chạy được một ngày và một hệ chạy được một năm.

1.5 Ràng buộc kỹ thuật thật

A. Nén và tuần tự hoá

Một bản đồ phải lưu được xuống đĩa và tải lại — cho cold start (B/12), cho nhiều phiên, cho cập nhật OTA. Ba yêu cầu thường xung đột.

Kích thước: descriptor chiếm phần lớn. Với ORB, mỗi descriptor là 32 byte; với một bản đồ triệu landmark có vài quan sát mỗi cái, đó là hàng trăm megabyte. Các cách giảm: chỉ lưu một descriptor đại diện cho mỗi landmark thay vì mọi quan sát; lượng tử hoá; hoặc lưu chỉ số từ điển BoW thay vì descriptor đầy đủ — cách cuối làm mất khả năng khớp chính xác nhưng giữ được khả năng truy hồi.

Tốc độ tải: quan trọng cho cold start, nơi người dùng đang chờ. Định dạng nhị phân có cấu trúc phẳng tải nhanh hơn nhiều so với định dạng có nhiều con trỏ phải dựng lại.

Tính tương thích: bản đồ lưu bằng phiên bản phần mềm này phải đọc được bằng phiên bản sau. Đây là yêu cầu thật cho sản phẩm có cập nhật OTA, và nó đòi hỏi *phiên bản hoá định dạng* ngay từ đầu — thêm vào sau rất đau.

B. Quyền riêng tư và map inversion

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó là một vấn đề thật chứ không phải lo xa.

Trực giác thông thường: một bản đồ chỉ gồm toạ độ 3D và descriptor thì không chứa ảnh, nên nó không riêng tư. Trực giác đó *sai*.

Pittaluga và cộng sự [5] cho thấy có thể tái tạo lại ảnh nhận ra được từ một bản đồ điểm kèm descriptor. Lý do: descriptor mã hoá về ngoài cục bộ, và với đủ điểm, một mô hình học được có thể ghép chúng lại thành một ảnh. Nói cách khác, *bản đồ SLAM của một căn nhà là một dạng ảnh chụp căn nhà đó*, chỉ ở một mã hoá khác.

Hệ quả cho sản phẩm là thật và đáng nghĩ sớm. Nếu bản đồ được tải lên đám mây — cho xử lý, cho chia sẻ giữa các thiết bị, cho cập nhật — thì đó là truyền dữ liệu riêng tư, với mọi nghĩa vụ pháp lý kèm theo. Nếu bản đồ được lưu trên thiết bị, nó là một mục tiêu.

Các hướng giảm thiểu đang được nghiên cứu: thay điểm bằng *đường* (làm mất thông tin vị trí chính xác trong khi vẫn định vị được), làm nhiều descriptor, hoặc lưu bản đồ ở dạng chỉ dùng được để định vị chứ không dùng được để tái tạo. Không hướng nào hoàn thiện. Điều tối thiểu nên làm: *biết rằng bản đồ là dữ liệu riêng tư và đối xử với nó như vậy*.

CẦN NHỚ

Mọi chính sách trong chương này suy ra từ một câu: tư thế đắt, điểm rẻ.

Chi phí BA là $O(m^3)$ theo tư thế và $O(n)$ theo điểm ($A/04$ mục 4C). Từ đó:

Tạo keyframe **tiết kiệm**, theo tiêu chí thông tin (parallax, tỉ lệ theo dõi) chứ không theo khoảng cách; tạo landmark **hào phóng**.

Loại keyframe theo **mức dư thừa**, không theo tuổi — một keyframe cũ ở góc hiểm người qua đáng giá hơn mười keyframe mới chồng lên nhau.

Chọn bản đồ cục bộ theo **lân cận trong đồ thị đồng quan sát**, vì đó chính là tập biến tương tác trực tiếp trong ma trận S .

Làm thưa bản đồ theo **độ phủ**, không theo chất lượng trung bình.

Và một điều không suy ra từ toán mà từ thực tế: **ngưỡng loại landmark nên gắt lúc mới tạo và nới dần theo tuổi**, vì phần lớn landmark sai lộ ra trong vài khung đầu, còn một landmark sống sót qua hàng trăm quan sát là tài sản đã được kiểm chứng.

❗ Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [3]	Covisibility + essential graph + spanning tree; culling keyframe theo dư thừa; Atlas nhiều bản đồ	Cài đặt đầy đủ nhất của cả chương; đọc mã để học kiến trúc ba luồng
maplab [6]	Nhiều phiên, ghép bản đồ, làm thưa, quản lý dài hạn	Hiếm framework coi quản lý bản đồ là chủ đề chính chứ không phải phụ
[2]	Làm thưa bản đồ theo độ phủ và độ bền qua các phiên	Mục 3C — nguyên tắc chọn theo độ phủ
RTAB-Map	Quản lý bộ nhớ có cấu trúc: working memory và long-term memory	Một cách tiếp cận khác cho bài toán kích thước, lấy cảm hứng từ mô hình trí nhớ người
[5]	Tái tạo ảnh từ point cloud kèm descriptor	Mục 5B — đọc để hết ảo tưởng rằng bản đồ không riêng tư
[4]	Bộ dữ liệu đo thay đổi môi trường theo thời gian	Một trong ít nguồn dữ liệu để nghiên cứu mục 4C

MINI PROJECT SLAM · Chạy một hệ SLAM trong tám tiếng, rồi xem cái gì hỏng · 4 ngày

Mục tiêu	thấy tận mắt ba chế độ hỏng ở mục 4C, vốn không xuất hiện trên bất kỳ benchmark nào vì mọi benchmark đều quá ngắn.
Dữ liệu	tự thu: đặt camera trên một xe đẩy hoặc robot, đi lặp lại một tuyến trong văn phòng hoặc nhà bạn, <i>nhiều lần trong nhiều ngày</i> — ít nhất năm phiên trải trên một tuần, và cố ý dịch chuyển vài đồ vật giữa các phiên.
Các bước	chạy một hệ có sẵn (ORB-SLAM3) trên toàn bộ dữ liệu theo trình tự thời gian; ghi lại theo thời gian: số keyframe, số landmark, bộ nhớ, thời gian mỗi khung của cả ba luồng, tỉ lệ tracking, tỉ lệ relocalization thành công ở mỗi phiên; rồi so bản đồ cuối với bản đồ sau phiên đầu và đếm bao nhiêu landmark của phiên đầu còn sống.
Xong khi	bạn có ba đường cong theo thời gian và một con số. Đường cong: kích thước bản đồ, thời gian mỗi khung, và tỉ lệ tracking. Con số: tỉ lệ landmark từ phiên đầu còn sống sau phiên cuối — đó là <i>tốc độ lão hoá</i> của bản đồ trong môi trường của bạn, và tôi không biết ai đã công bố con số này cho một môi trường thật. Nếu tỉ lệ tracking giảm dần qua các phiên, bạn vừa đo được bài toán ở mục 4C bằng dữ liệu của chính mình.
Nối với	D/19-ben-vung-trong-the-gioi-thuc và D/21-tu-prototype-len-mass-production xây trực tiếp trên kết quả này; D/22-huong-nghien-cuu-con-trong lấy lifelong mapping làm một khoảng trống, và số liệu của bạn là bằng chứng cho nó.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chi phí bất đối xứng: sau phần bù Schur, hệ còn lại có kích thước $6m \times 6m$ và chi phí giải là siêu tuyến tính theo m , trong khi việc khử n landmark chỉ tốn $O(n)$ vì khối V là khối chéo (A/04 mục 4C). Gấp đôi landmark làm chi phí tăng gấp đôi; gấp đôi keyframe làm nó tăng nhiều lần hơn. Có lý do thứ hai độc lập và cũng quan trọng: hai keyframe cách nhau một bước chân cho các hàng Jacobian gần tuyến tính phụ thuộc nên thêm rất ít *thông tin* trong khi tốn chi phí đầy đủ. Hai lý do hội tụ về cùng chính sách, và đó là lý do nó vững.
- A2. Nó *chính xác* là cấu trúc khác không của ma trận $S = U - WV^{-1}W^T$ sau phần bù Schur: khối (i, i') của S khác không đúng khi tồn tại một landmark mà cả hai tư thế cùng quan sát. Nghĩa là nó không tùy tiện — nó là hình ảnh của một ma trận. Ba hệ quả: hai keyframe nối nhau là hai keyframe bộ giải *phải* xử lý cùng nhau; chọn bản đồ cục bộ bằng lân cận trong đồ thị là đúng theo nghĩa toán học vì đó là tập biến tương tác trực tiếp; và mật độ đồ thị quyết định chi phí, nên chính sách keyframe cũng là chính sách kiểm soát chi phí.
- A3. Ba thứ, theo mức khó tăng dần. (a) *Kích thước* — bộ nhớ cạn, tính toán chậm dần, và chất lượng giảm vì tỉ lệ khớp sai tăng theo số ứng viên; giải được bằng chính sách loại và làm thưa. (b) *Thay đổi về ngoài theo chu kỳ* — ánh sáng ngày đêm; giải một phần được bằng cách lưu nhiều

biểu diễn cho cùng địa điểm. (c) *Thay đổi cấu trúc* — đồ đạc sắp lại, tường dựng thêm; **khó nhất và chưa có lời giải**, vì không cách nào phân biệt “bản đồ sai” với “tôi ở sai chỗ” từ bên trong hệ thống — cả hai cho cùng triệu chứng nhưng đòi hai hành động ngược nhau.

- A4. Lựa chọn tốt hơn là *Atlas*: bắt đầu một bản đồ mới và giữ bản đồ cũ ở trạng thái không hoạt động, tiếp tục chạy trên bản đồ mới, đồng thời liên tục thử relocalize vào các bản đồ cũ — nếu thành công thì ghép. Ba lợi ích: không mất công sức đã bỏ ra, hệ vẫn hoạt động ngay cả khi chưa tìm lại được, và nhiều phiên trở thành trường hợp tự nhiên của cùng cơ chế. Yêu cầu kiến trúc: bản đồ phải là một *đối tượng tách rời và ghép được* — có hệ quy chiếu riêng, serialize được, và có phép toán ghép. Nhiều hệ có bản đồ gắn chặt với trạng thái toàn cục và không làm được; đây là quyết định nên đưa ra sớm.
- A5. Vì descriptor mã hoá về ngoài cục bộ, và với đủ điểm, một mô hình học được có thể ghép chúng lại thành một ảnh nhận ra được [5]. Nói cách khác, bản đồ SLAM của một căn nhà là một dạng ảnh chụp căn nhà đó, chỉ ở mã hoá khác. Hệ quả cho sản phẩm: tải bản đồ lên đám mây là truyền dữ liệu riêng tư với mọi nghĩa vụ pháp lý kèm theo; lưu trên thiết bị thì nó là một mục tiêu tấn công. Các hướng giảm thiểu — thay điểm bằng đường, làm nhiều descriptor — đều chưa hoàn thiện. Điều tối thiểu: biết rằng bản đồ là dữ liệu riêng tư và đối xử với nó như vậy.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về bản đồ như một cuốn album ảnh có chú thích mà bạn phải mang theo suốt đời. Mỗi trang là một chỗ bạn từng đứng; mỗi chú thích là một thứ bạn nhìn thấy từ đó. Và có một sự thật quyết định mọi chuyện: *thêm một trang thì đắt, thêm một chú thích thì rẻ*. Vì mỗi lần muốn chỉnh album cho nhất quán, bạn phải so từng trang với từng trang khác — càng nhiều trang càng tốn công theo cấp số. Chú thích thì chỉ thuộc về một trang nên không làm việc so trang khó hơn. Từ đó suy ra: keo kiệt với trang, hào phóng với chú thích. Chỉ thêm trang mới khi nó cho thấy thứ mà các trang cũ không thấy — nhích sang một bước thì không đáng. Và khi phải xé bớt trang, đừng xé theo tuổi mà xé theo mức *thừa*: một trang cũ chụp cái góc mà không trang nào khác có thì quý, còn mười trang mới chụp cùng một chỗ thì tĩa chín.

Rồi có ba chuyện xảy ra theo năm tháng, và chúng khó dần.

Album dày lên cho tới lúc không mang nổi — chuyện này giải được, chỉ cần kỷ luật xé bớt.

Bạn có nhiều album từ nhiều chuyến đi, và có lúc nhận ra hai cuốn ghi cùng một nơi — chuyện này cũng giải được, chỉ cần biết cách ghép.

Và chuyện thứ ba: *thế giới đổi*. Cái nhà trong ảnh đã bị phá, cái ghế đã dời chỗ. Chú thích của bạn vẫn nói chúng ở đó. Bạn chỉ phát hiện khi quay lại — và khi quay lại, bạn không phân biệt được “mình nhớ nhầm chỗ” với “chỗ này đã đổi”. Hai kết luận ấy đòi hai việc hoàn toàn trái ngược: một bên là đi tìm lại, một bên là sửa album.

Chuyện thứ ba chưa ai giải được, và nó là lý do chính khiến một hệ chạy tốt một ngày lại không chạy tốt được một năm.

Chỗ không rút gọn được: chỗ không rút gọn được là cái vòng luẩn quẩn của chuyện thứ ba: để biết bản đồ sai, bạn phải chắc rằng mình đang ở đúng chỗ; để chắc mình ở đúng chỗ, bạn phải tin bản đồ. Mọi cách tiếp cận hiện có đều là cách phá vòng ấy một cách thô — đếm phiếu qua nhiều lần đi qua, quên dần theo thời gian — và không cách nào thoả đáng.

Một câu: Tư thế thì đắt, điểm thì rẻ — và gần như mọi quyết định về bản đồ là hệ quả của câu đó.

1.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *đồ thị đồng quan sát là cấu trúc khác không của S* — là một sự kiện đại số suy trực tiếp từ công thức Schur (cửa a), kiểm được bằng cách dựng S và so mẫu khác không với đồ thị. Nó bị bác bỏ nếu có nhân tử nào nổi hai tư thế trực tiếp — chẳng hạn một ràng buộc odometry hay tiền tích phân IMU — vì lúc đó S có thêm khối khác không không đến từ landmark chung. Đây là một điều kiện thật và đáng nêu rõ: *với hệ có odometry, đồ thị đồng quan sát là một tập con của cấu trúc S , không phải toàn bộ*.

Mệnh đề thứ hai — *tư thế đất, điểm rẽ* — suy từ A/04 và kiểm được bằng đo đạc (mini project ở chương 4). Nó mất hiệu lực khi đồ thị đồng quan sát rất đặc, vì lúc đó S đặc và chi phí theo m còn tệ hơn dự đoán — tức mệnh đề đúng theo hướng *mạnh hơn*, không phải yếu hơn.

Mệnh đề thứ ba — *thay đổi cấu trúc chưa có lời giải tốt* — là một phát biểu về trạng thái của lĩnh vực, và nó là mệnh đề tôi ít chắc nhất vì tôi *chưa khảo sát có hệ thống* tài liệu về lifelong SLAM theo chuẩn `research-rules.md`. Nó bị bác bỏ nếu một hệ duy trì được tỉ lệ relocalization ổn định qua nhiều tháng trong một môi trường thay đổi thật, được kiểm định độc lập. Đây là một trong những mục quan trọng nhất trong `99-chua-biet.md`, và nó nên được nâng lên thành một dự án khảo sát riêng.

Một mệnh đề mềm: chương này khẳng định *quản lý bản đồ bị coi nhẹ trong tài liệu*. Đó là một quan sát về văn hoá nghiên cứu chứ không phải một sự kiện đo được. Nó bị bác bỏ nếu một khảo sát cho thấy tỉ lệ đáng kể các bài SLAM có phần thực nghiệm về hoạt động dài hạn. Tôi cho là không, dựa trên việc các bộ dữ liệu chuẩn đều ngắn, nhưng tôi không đếm (*tôi suy ra, chưa đổi chiều nguồn*).

1.7 Kết nối

Nối lên trên. A/04-uoc-luong-map-va-bundle-adjustment là nguồn của toàn bộ chương: phần bù Schur cho cả đồ thị đồng quan sát lẫn chi phí bất đối xứng. B/09-paradigm-uoc-luong-back-end cho kiến trúc nhiều luồng và pose graph. B/11-loop-closure-va-nhat-quan-toan-cuc nêu vấn đề hoà nhập mà mục 4A ở đây xử lý. B/12-relocalization nêu vấn đề bản đồ lỗi thời mà mục 4C ở đây mở rộng — và vẫn không giải được.

Nối xuống dưới. C/14-hieu-chuan-nhu-mot-nhanh-cua-slam chia sẻ một vấn đề với chương này: cái gì được coi là cố định và cái gì được phép thay đổi theo thời gian. C/16-dense-mapping-va-bieu-dien là chỗ bài toán kích thước trở nên gay gắt hơn nhiều bậc, vì biểu diễn dày tốn bộ nhớ hơn biểu diễn thưa hàng trăm lần. C/18-danh-gia-va-phuong-phap-luan là chỗ ta cần các chỉ tiêu để so sánh hai chính sách quản lý bản đồ — và chương đó phải thừa nhận rằng các chỉ tiêu hiện có không đo được điều này.

Nối xa hơn. D/20-toi-uu-hieu-nang-va-he-thong lấy vấn đề nhiều luồng ở mục 4A làm chủ đề chính, đặc biệt là tính tắt định và ngân sách bộ nhớ. D/21-tu-prototype-len-mass-production là chỗ mục 5 trở thành yêu cầu thật: nén bản đồ cho thiết bị nhúng, phiên bản hoá cho cập nhật OTA, và quyền riêng tư cho nghĩa vụ pháp lý. D/22-huong-nghien-cuu-con-trong lấy lifelong mapping ở mục 4C làm khoảng trống số ba, và tôi cho rằng đó là khoảng trống đau nhất trong triển khai thực tế.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao mọi hệ thống tiết kiệm keyframe mà hào phóng landmark? Nêu hai lý do độc lập.
2. Nêu gốc rễ toán học của đồ thị đồng quan sát, và nêu một điều kiện khiến phát biểu đó không còn đúng hoàn toàn.
3. Tiêu chí loại keyframe của ORB-SLAM loại theo mức dư thừa chứ không theo tuổi. Vì sao điều đó quan trọng, và nêu một trường hợp mà loại theo tuổi sẽ sai.
4. Ba nhịp độ của một hệ SLAM là gì, và nêu ba vấn đề kỹ thuật sinh ra từ việc chúng chia nhau một bản đồ.
5. Atlas giải quyết vấn đề gì mà “vứt bản đồ và làm lại” không giải quyết được, và nó đòi hỏi gì về kiến trúc?

6. Vì sao “bản đồ sai” và “tôi ở sai chỗ” không phân biệt được từ bên trong hệ thống, và vì sao điều đó làm lifelong mapping khó?
7. Bản đồ chỉ gồm điểm và descriptor. Vì sao nó vẫn là dữ liệu riêng tư, và hệ quả gì cho thiết kế sản phẩm?

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.

Hiệu chuẩn như một nhánh của SLAM (Calibration as a Branch of SLAM)

PHỤ THUỘC A/02-mô-hình-camera (mô hình nội và méo, và kỹ năng phân biệt mô hình sai với tham số sai); A/04-uoc-luong-map-va-bundle-adjustment (khung MAP — chương này chỉ đổi danh sách biến); A/05-observability-gauge-va-suy-bien (điều kiện kích thích, Bảng điều kiện — không có nó thì hiệu chuẩn trực tuyến là mở rộng trạng thái mù quáng); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai tham số là thứ nên in ra thay cho RMS).

TÓM TẮT

Đây là chương hợp nhất của cả cây. Suốt Phần A và Phần B, tham số hiệu chuẩn được coi là hằng số nạp từ một tệp — một giả định tiện lợi mà không chương nào biện minh. Chương này bỏ giả định ấy và cho thấy điều gì xảy ra: *không có gì trong bộ giải phải thay đổi*. Hiệu chuẩn và SLAM là cùng một bài toán MAP; chúng khác nhau đúng ở danh sách biến tự do. Nội dung đi theo một mạch duy nhất: bắt đầu từ hiệu chuẩn nội với target đã biết — bài toán dễ nhất vì cấu trúc bị fix — rồi thả dần các biến ra, qua tự hiệu chuẩn, qua hiệu chuẩn ngoại, tới hiệu chuẩn trực tuyến trong một hệ VIO đang chạy, tức tới chính bài toán SLAM đầy đủ. Mỗi bước thả một biến, và mỗi bước đòi một điều kiện kích thích cụ thể — đó là chỗ A/05 trả công toàn bộ. Hai mục cuối bàn về hai thứ mà sản phẩm cần mà nghiên cứu ít nói: cách tự đánh giá chất lượng hiệu chuẩn, và cách phát hiện khi nó đã trôi. Nếu chỉ nhớ một điều: *calibration target chỉ là một cách giảm phương sai bằng cách fix cấu trúc; khi không fix được cấu trúc, bạn phải dùng chuyển động để thay thế* — và đó là toàn bộ khác biệt giữa phòng lab và robot đang chạy.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hiệu chuẩn nội (intrinsic calibration)** — xác định các tham số mô tả cách camera biến tia thành pixel — tiêu cự, điểm chính, hệ số méo (A/02).
- **hiệu chuẩn ngoại (extrinsic calibration)** — xác định tư thế tương đối giữa hai cảm biến — hai camera trong cặp stereo, camera và IMU, camera và LiDAR.
- **sai lệch thời gian (time offset, t_d)** — độ lệch giữa mốc thời gian mà hai cảm biến gán cho cùng một sự kiện vật lý. Nguyên nhân: độ trễ truyền, độ trễ phơi sáng, đồng hồ khác nhau.
- **target hiệu chuẩn** — một vật có hình học đã biết chính xác — bảng cờ vua, ChArUco, lưới vòng tròn. Vai trò thật của nó là *fix cấu trúc*, tức loại bỏ một nhóm biến khỏi bài toán.
- **tự hiệu chuẩn (self-calibration)** — ước lượng tham số hiệu chuẩn mà không có target, bằng cách coi chúng là biến trong chính bài toán SLAM hoặc SfM.
- **hiệu chuẩn trực tuyến (online calibration)** — tự hiệu chuẩn chạy liên tục trong lúc hệ thống hoạt động, để bù trôi theo nhiệt độ, rung, lão hoá.
- **cổng observability (observability gating)** — cơ chế chỉ cập nhật một tham số khi chuyển động hiện tại kích thích nó đủ, và đóng băng nó khi không. Đây là điều phân biệt hiệu chuẩn trực tuyến có nguyên tắc với việc mở rộng trạng thái mù quáng.

CÂU HỎI MỞ ĐẦU

- Q1. Hiệu chuẩn dùng bảng cờ vua, SLAM dùng cảnh tự nhiên. Nêu chính xác điều mà bảng cờ vua cung cấp, và nói vì sao điều đó làm bài toán dễ hơn — không phải “vì nó chính xác” mà là gì?
- Q2. Hiệu chuẩn trực tuyến thường được mô tả là “chỉ cần thêm biến vào vector trạng thái”. Nếu đơn giản như vậy thì vì sao phép biến đổi ngoại của bạn trôi lung tung trên một chuỗi và đứng im trên chuỗi khác?
- Q3. Bạn hiệu chuẩn camera-IMU bằng cách xoay thiết bị quanh đúng một trục trong suốt quá trình. Thành phần nào của phép biến đổi ngoại không xác định được, và vì sao tích có hướng là thủ phạm?
- Q4. Một kỹ sư báo rằng hiệu chuẩn ngoại trực tuyến của anh ta hội tụ rất nhanh và rất ổn định. Nêu hai cách giải thích — một tốt một xấu — và nêu phép thử phân biệt.
- Q5. Bạn cần biết hiệu chuẩn đã trôi để yêu cầu hiệu chuẩn lại. Nhưng tại hiện trường không có target và không có chân trị. Nêu ba đại lượng đo được có thể phát hiện trôi.

2.1 Đặt vấn đề

Suốt hai phần đầu của cây, có một giả định được dùng liên tục mà không chương nào biện minh: rằng tham số hiệu chuẩn là *hằng số đã biết*. A/01 làm việc trên mặt phẳng chuẩn hoá, tức giả định đã gỡ được K và méo. A/04 đặt tư thế và cấu trúc làm biến, và lạng lẽ để hiệu chuẩn ra ngoài. B/08 viết phần dư chiếu với một hàm π cố định.

Giả định ấy sai theo hai cách, và cả hai đều tốn kém.

Sai cách thứ nhất — nó không bao giờ đúng chính xác. Mọi phép hiệu chuẩn đều có sai số, và như A/02 mục 4C đã chỉ ra, sai số ấy không biến mất mà *chảy* vào tư thế và cấu trúc theo một hoa văn có hệ thống. Bình phương tối thiểu không khử được sai lệch có hệ thống.

Sai cách thứ hai — nó thay đổi theo thời gian. Ống kính giãn nở khi nóng lên. Khung stereo biến dạng. Keo dán module lão hoá. Thiết bị bị va đập. Tham số đúng vào ngày xuất xưởng không còn đúng sau sáu tháng ở nhà khách hàng.

Bài toán gốc của chương: *làm sao ước lượng các tham số ấy, và làm sao giữ chúng đúng theo thời gian?*

Ai gặp bài toán này? Ai làm thị giác đo đạc. Nhưng mức độ đau khác nhau rất nhiều giữa nghiên cứu và sản xuất, và đó là điều đáng nói. Trong nghiên cứu, ta hiệu chuẩn một lần bằng Kalibr, lưu tệp YAML, và quên đi — vì thí nghiệm kéo dài vài giờ và thiết bị không rời khỏi bàn. Trong sản xuất, ta phải hiệu chuẩn hàng nghìn thiết bị trong dưới một phút mỗi cái, theo dõi trôi trong nhiều năm, và phát hiện khi một thiết bị cần hiệu chuẩn lại mà không có target trong tay (D/21).

Trước khi có khung hợp nhất, hiệu chuẩn và SLAM được học như hai môn riêng. Hiệu chuẩn có bàn xoay, bảng cờ vua, một phần mềm chạy một lần rồi ra một tệp. SLAM có camera chuyển động, điểm đặc trưng, quỹ đạo. Hai thứ trông không liên quan, và chúng được dạy ở hai chỗ khác nhau.

Cách nhìn đó sai, và nó sai theo một kiểu tốn kém: nó làm người ta học hai lần cùng một thứ, và tệ hơn, nó che mất việc *cùng một lý thuyết observability quyết định cả hai*. Khi một kỹ sư hỏi “vì sao hiệu chuẩn ngoại trực tuyến của tôi không hội tụ”, câu trả lời nằm ở đúng chỗ đã trả lời câu “vì sao tỉ lệ mono của tôi sai”: chuyển động không đủ kích thích thì ma trận thông tin suy biến theo một hướng, và bộ ước lượng không có cách nào biết mình đang đoán mò theo hướng đó.

Chương này lập luận rằng cả hai là *một* bài toán, và nó chứng minh điều đó bằng cách đi từ ô này sang ô kia trong bảng ở 00-map, mỗi lần thả thêm một biến.

TRỰC GIÁC

Hình dung lại hệ lò xo ở A/04: mỗi phép đo là một lò xo nối các ẩn số, và nghiệm là vị trí cân bằng.

Hiệu chuẩn bằng target và SLAM khác nhau ở *đúng một chuyện*: bạn hàn cứng cái gì vào tường.

Với target, bạn hàn cứng *cấu trúc* — bạn biết chính xác các góc bằng cờ cách nhau bao nhiêu milimet, vì bạn đã in nó ra. Hệ lò xo còn lại giải ra tham số camera và quỹ đạo của bảng. Với SLAM, bạn hàn cứng *tham số camera* — bạn đã đo chúng hôm trước. Hệ giải ra quỹ đạo và cấu trúc.

Với hiệu chuẩn trực tuyến, bạn *không hàn gì cả*. Hệ giải ra tất cả — nếu có đủ lò xo.

Cái “nếu” cuối câu là toàn bộ nội dung khó của chương, và nó chính là observability ở A/05. Khi bạn tháo một cái hàn, bạn thêm một bậc tự do; nếu không có lò xo nào căng khi bậc đó thay đổi, hệ có vô số nghiệm và bộ giải dừng ở một chỗ tùy tiện.

Từ đây rút ra câu quan trọng nhất của chương, và nó đáng thuộc lòng: *target không phải một dụng cụ chính xác kỳ diệu — nó chỉ là một cách giảm phương sai bằng cách fix cấu trúc*. Khi bạn không có target, bạn phải thay cái hàn ấy bằng thứ khác, và thứ duy nhất có sẵn là *chuyển động*. Chuyển động đa dạng làm nhiều lò xo căng theo nhiều hướng, và nó thay thế được cái hàn.

Đó là lý do mọi hướng dẫn hiệu chuẩn đều bảo bạn nghiêng bảng nhiều hướng, đưa ra sát rìa, lại gần rồi ra xa — và cũng là lý do một robot đi thẳng mãi không bao giờ tự hiệu chuẩn được.

2.2 Đi từ ô này sang ô kia

Đây là mục trung tâm của chương. Ta bắt đầu từ bài toán dễ nhất rồi thả dần các biến, và theo dõi xem mỗi lần thả đòi hỏi thêm điều gì.

A. Bước không: hiệu chuẩn nội với target

Biến: tham số nội θ_K (tiêu cự, điểm chính, hệ số méo) và tư thế của bảng ở mỗi ảnh $T_1, \dots, T_N$.

Hằng số: tọa độ 3D của các góc bảng trong hệ của bảng — *đã biết chính xác vì ta in ra nó*.

Nhân tử: với mỗi góc bảng nhìn thấy ở mỗi ảnh, một phần dư chiếu

$$\mathbf{r}_{ij} = \mathbf{z}_{ij} - \pi(T_i \mathbf{P}_j; \theta_K).$$

So với bundle adjustment ở A/04, khác biệt duy nhất là $\mathbf{P}_j$ là hằng số thay vì biến, và θ_K là biến thay vì hằng số. Cùng một bộ giải, cùng một cấu trúc thừa, cùng mọi thứ. (Cửa kiểm chứng a: dựng đồ thị nhân tử cho cả hai và so sánh.)

Điều target thực sự cung cấp — và đây là câu trả lời cho câu hỏi mở đầu thứ nhất — *không phải độ chính xác mà là việc loại bỏ một nhóm biến*. Với N ảnh và M góc bảng, SfM thuần túy sẽ có $6N + 3M$ ẩn; hiệu chuẩn bằng target chỉ có $6N + |\theta_K|$. Với M cỡ vài trăm, đó là khác biệt lớn — và quan trọng hơn số lượng, nó loại bỏ gauge tỉ lệ, vì kích thước bảng cho đơn vị mét.

B. Bước một: bỏ target, thả cấu trúc

Bây giờ bỏ bảng đi và dùng cảnh tự nhiên. Cấu trúc trở thành biến, và ta có *tự hiệu chuẩn*: bài toán SfM với tham số nội nằm trong tập biến. COLMAP làm chính xác điều này [7].

Cái được: không cần target, nên làm được ở bất cứ đâu và bất cứ lúc nào — kể cả trên robot đang chạy.

Cái mất: *phương sai tăng*, và gauge tỉ lệ quay lại. Bài toán bây giờ có nhiều biến hơn và ít ràng buộc hơn trên mỗi biến, nên bất định của θ_K lớn hơn. Và vì không còn kích thước đã biết, tỉ lệ lại là gauge tự do (A/05 mục 3).

Điều kiện mới xuất hiện: *chuyển động phải đủ đa dạng*. Bảng ?? ở A/05 liệt kê chính xác cái gì cần gì: tiêu cự cần đa dạng độ sâu, điểm chính cần đặc trưng ở nhiều vị trí trong khung hình, hệ số méo cần đặc trưng ở rìa. Với target, người thao tác cung cấp sự đa dạng ấy một cách có chủ ý; với cảnh tự nhiên, ta phải chờ nó xảy ra hoặc kiểm xem nó đã xảy ra chưa.

Đây là lần đầu trong chương mà việc thả một biến đòi thêm một điều kiện, và khuôn mẫu ấy lặp lại ở mọi bước sau.

C. Bước hai: thả ngoại

Với nhiều cảm biến, thêm phép biến đổi ngoại vào tập biến. Hai trường hợp đáng nói riêng vì điều kiện của chúng khác nhau.

Stereo: thả $T_{\text{trái} \rightarrow \text{phải}}$. Điều kiện: cảnh phải có *độ sâu đa dạng*. Lý do trực giác: nếu mọi điểm ở cùng một khoảng cách, thì một sai lệch nhỏ trong góc giữa hai camera có thể được bù bằng một sai lệch trong khoảng cách chung, và hai giả thuyết không phân biệt được. Có cả gần lẫn xa thì không bù được.

Camera-IMU: thả $T_{\text{cam} \rightarrow \text{imu}}$ và t_d . Đây là trường hợp được nghiên cứu kỹ nhất, và nó chứa một kết quả đẹp đáng dẫn lại.

Vận tốc của camera và vận tốc của IMU liên hệ qua

$$\mathbf{v}_{\text{cam}} = \mathbf{v}_{\text{imu}} + \boldsymbol{\omega} \times \mathbf{p}_{\text{cam}}^{\text{imu}},$$

với $\mathbf{p}$ là phần tịnh tiến của phép biến đổi ngoại. Số hạng $\boldsymbol{\omega} \times \mathbf{p}$ là *thứ duy nhất* trong toàn bộ hệ phương trình chứa $\mathbf{p}$.

Từ đó suy ra ngay câu trả lời cho câu hỏi mở đầu thứ ba. Nếu $\boldsymbol{\omega} = 0$ — tịnh tiến thuần — thì số hạng đó bằng không và $\mathbf{p}$ hoàn toàn không quan sát được. Nếu $\boldsymbol{\omega}$ luôn cùng một hướng — xoay quanh đúng một trục — thì tích có hướng $\boldsymbol{\omega} \times \mathbf{p}$ chỉ nhạy với thành phần của $\mathbf{p}$ *vuông góc* với $\boldsymbol{\omega}$; thành phần *song song* với trục quay không ảnh hưởng gì tới tích có hướng, nên nó không quan sát được. Cần xoay quanh ít nhất hai trục độc lập để xác định cả ba thành phần. (Cửa a: suy trực tiếp từ tính chất của tích có hướng.)

Đây là một trong những ví dụ sạch nhất trong cả cây về việc một điều kiện thực hành — “xoay thiết bị quanh cả ba trục khi hiệu chuẩn” — là hệ quả trực tiếp của một tính chất đại số. Và nó cho thấy vì sao lời khuyên truyền miệng không tổng quát hoá được: ai chỉ nghe “lắc thiết bị” mà không biết vì sao sẽ không biết phải làm gì với một cấu hình cảm biến mới.

D. Bước ba: thả tất cả, trong lúc đang chạy

Bước cuối: đặt mọi thứ vào tập biến — tư thế, vận tốc, độ lệch IMU, cấu trúc, nội, ngoại, t_d — và chạy liên tục trong khi hệ thống hoạt động. Đây chính là ô cuối cùng của bảng ở 00-map, và OpenVINS [8] là cài đặt tham chiếu.

Đây là câu trả lời cho câu hỏi mở đầu thứ hai. Mô tả “chỉ cần thêm biến vào vector trạng thái” đúng về *cơ chế* và sai về *điều kiện*. Công thức đầy đủ là:

hiệu chuẩn trực tuyến = mở rộng trạng thái + đủ kích thích + công để biết khi nào không đủ.

Về thứ hai giải thích vì sao cùng một hệ thống hội tụ trên chuỗi này và trôi trên chuỗi kia: chuỗi có xoay quanh nhiều trục kích thích $\mathbf{p}$ qua số hạng $\omega \times \mathbf{p}$, chuỗi tịnh tiến thuần thì không. Về thứ ba là mục 3.

CẦN NHỚ

Target không phải một dụng cụ chính xác kỳ diệu — nó là một cách giảm phương sai bằng cách fix cấu trúc.

Khi bạn không có target, bạn phải thay cái đó bằng thứ khác, và thứ duy nhất có sẵn là *chuyển động*. Đó là toàn bộ khác biệt giữa hiệu chuẩn trong phòng lab và hiệu chuẩn trên một robot đang chạy.

Hệ quả là một cách nhìn thống nhất cho mọi bài toán trong Bảng ở 00-map: mỗi bài toán là cùng một đồ thị nhân tử, và việc đi từ ô này sang ô kia là việc *tháo một cái hàn và thay nó bằng một điều kiện chuyển động*.

Hệ quả thực hành trực tiếp: khi thiết kế một quy trình hiệu chuẩn — dù trong lab hay trên dây chuyền — câu hỏi đúng không phải “dùng target nào” mà là **“tham số nào tôi cần, và chuyển động nào làm nó quan sát được”**. Bảng ?? ở A/05 là bảng trả lời câu đó, và mọi hướng dẫn thu dữ liệu hiệu chuẩn đều là bảng ấy viết lại thành thao tác tay.

2.3 Cổng observability: điều phân biệt có nguyên tắc với mù quáng

A. Vấn đề

Thả một tham số ra mà không kiểm điều kiện kích thích là một sai lầm có hậu quả cụ thể và nghiêm trọng hơn vẻ ngoài.

Khi một tham số không quan sát được, nó không đơn giản là “không hội tụ”. Nó *trôi tự do*, và tệ hơn, nó *hấp thụ sai số từ các nguồn khác*. Bộ giải thấy một hướng phẳng trong hàm mục tiêu, và nó dùng hướng đó để giảm phần dư gây ra bởi những nguyên nhân hoàn toàn khác — một vật động chưa loại, một mô hình méo sai, một sai lệch thời gian.

Kết quả là tình huống tệ nhất có thể: tham số hiệu chuẩn nhận một giá trị sai, phần dư trông đẹp, và sai số thật đã được giấu vào một biến mà không ai kiểm tra. Khi điều kiện chuyển động thay đổi và tham số ấy trở lại quan sát được, nó phải dịch chuyển một quãng lớn, và hệ thống trải qua một giai đoạn bất ổn không giải thích được.

B. Cơ chế cổng

Ý tưởng đơn giản và có nguyên tắc [9], [10]: theo dõi mức quan sát được của từng tham số theo thời gian thực, và chỉ cập nhật nó khi mức ấy đủ.

Cài đặt cụ thể có vài mức tinh vi.

Mức đơn giản nhất — cổng theo chuyển động. Với mỗi tham số, xác định trước một đại lượng chuyển động mà nó cần (Bảng ??), đo đại lượng ấy trên cửa sổ hiện tại, và đóng băng tham số nếu dưới ngưỡng. Với $\mathbf{p}_{cam}^{imu}$, đại lượng là mức đa dạng của hướng ω ; với t_d , là độ lớn của tốc độ góc.

Mức đúng hơn — cổng theo phổ ma trận thông tin. Dạng khối của Λ tương ứng với các tham số hiệu chuẩn, tính giá trị riêng nhỏ nhất hoặc số điều kiện, và đóng băng khi nó dưới ngưỡng. Đắt hơn nhưng tổng quát — nó không cần biết trước tham số nào cần chuyển động gì.

Mức thực dụng — prior mềm thích ứng. Thay vì đóng băng cứng, đặt một prior quanh giá trị hiện tại với độ mạnh *tỉ lệ nghịch* với mức quan sát được. Khi kích thích tốt, prior yếu và dữ liệu quyết

định; khi kích thích kém, prior mạnh và tham số giữ nguyên. Mượt hơn, tránh được các bước nhảy khi cổng đóng mở.

C. Vì sao hội tụ nhanh có thể là tin xấu

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và là một phép chẩn đoán đáng nhớ.

Hai cách giải thích cho việc hiệu chuẩn ngoại trực tuyến “hội tụ rất nhanh và rất ổn định”.

Cách tốt: chuyển động thật sự kích thích mạnh, ma trận thông tin xác định dương tốt, và tham số hội tụ về giá trị đúng. Điều này xảy ra và là điều ta muốn.

Cách xấu: tham số thực ra không quan sát được, nhưng có một prior mạnh (có thể là một prior bạn đặt, hoặc damping LM, hoặc một khởi tạo được tin quá mức) đang ghim nó. Nó “ổn định” vì nó không đi đâu cả — không phải vì dữ liệu xác nhận nó.

Phép thử phân biệt, và nó rẻ: *chạy lại với vài giá trị khởi đầu khác nhau*. Nếu kết quả hội tụ về cùng một chỗ bất kể khởi đầu, đó là cách tốt. Nếu kết quả đi theo giá trị khởi đầu, đó là cách xấu — và đây chính là phép thử (1) ở A/05 khối *Cạm bẫy*, áp dụng cho một tình huống cụ thể.

Phép thử thứ hai, đắt hơn nhưng nhiều thông tin hơn: *nhìn hiệp phương sai của tham số*, không phải giá trị của nó. Một tham số quan sát được tốt có hiệp phương sai *giảm dần* theo thời gian; một tham số bị ghim bởi prior có hiệp phương sai *đứng yên* ở đúng giá trị prior.

I 2.4 Tự đánh giá chất lượng hiệu chuẩn

Đây là mục mà nghiên cứu ít nói và sản phẩm bắt buộc phải có.

A. Bốn đại lượng thay cho RMS

A/02 khối *Cạm bẫy* đã lập luận vì sao sai số chiếu lại trung bình gần như vô dụng. Bốn đại lượng thay thế, theo thứ tự giá trị:

Một — *bản đồ phần dư dạng trường vectơ*. Chẩn đoán mô hình sai. Nếu phần dư có hoa văn, không lượng dữ liệu nào cứu được.

Hai — *hiệp phương sai của tham số, từ Λ* . Đây là đại lượng đúng để trả lời “tôi biết tham số này chính xác tới đâu”. Quan trọng: nhìn cả *tương quan* chứ không chỉ đường chéo. Tương quan cao giữa f và k_1 nghĩa là hai tham số không tách được từ dữ liệu này, và bộ số thu được nằm tùy tiện dọc một thung lũng (A/02 mục 4B).

Ba — *độ phủ của dữ liệu*. Các góc bằng có tới sát bốn góc ảnh không? Có đủ đa dạng độ sâu không? Đây là kiểm *đầu vào* chứ không phải kiểm *đầu ra*, và nó rẻ hơn nhiều vì nó chạy được *trước* khi tối ưu — một quy trình dây chuyền tốt sẽ từ chối một bộ dữ liệu kém ngay lúc thu, không phải sau khi chạy xong (D/21).

Bốn — *kiểm chéo*. Hiệu chuẩn trên một nửa dữ liệu, đo phần dư trên nửa kia. Bắt được overfitting mà ba cái trên bỏ sót.

B. Phát hiện trôi tại hiện trường

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là bài toán khó nhất trong mục này: tại hiện trường không có target, không có chân trị, và thiết bị đang phục vụ khách hàng.

Ba đại lượng đo được, xếp theo độ nhảy.

Một — phần dư có cấu trúc theo thời gian. Nếu hiệu chuẩn trôi, phần dư chiếu sẽ phát triển một hoa văn không gian giống như ở A/02 mục 4A — chỉ là bây giờ trên các đặc trưng tự nhiên thay vì trên góc bằng. Cách theo dõi rẻ: tích lũy phần dư trung bình theo từng ô lưới trên mặt phẳng ảnh, trên một cửa sổ dài, và cảnh báo khi độ lệch giữa các ô vượt ngưỡng. Rẻ vì chỉ cần vài chục con số.

Hai — tham số hiệu chuẩn trực tuyến tự nó. Nếu bạn đang chạy hiệu chuẩn trực tuyến có cổng, thì giá trị của tham số là một tín hiệu. Theo dõi nó theo thời gian; một xu hướng đơn điệu (không phải dao động) là dấu hiệu trôi vật lý. Điều kiện: cổng phải hoạt động, nếu không bạn đang theo dõi nhiễu.

Ba — tương quan với nhiệt độ. Nếu có cảm biến nhiệt — và thiết bị nhúng thường có — thì tương quan giữa tham số ước lượng và nhiệt độ vỏ máy là bằng chứng mạnh cho trôi nhiệt. Nó cũng cho một khả năng đáng giá: bù trước bằng một mô hình tham số theo nhiệt độ, thay vì chỉ phát hiện.

CẠM BÃY

Một hệ thống hiệu chuẩn trực tuyến không có cổng observability tệ hơn không có hiệu chuẩn trực tuyến.

Lý do: khi tham số không quan sát được, nó không đứng yên mà *hấp thụ sai số từ các nguồn khác*. Bộ giải thấy một hướng phẳng và dùng nó để giảm phần dư gây ra bởi những nguyên nhân hoàn toàn khác — một vật động chưa loại, một mô hình méo sai, một sai lệch thời gian chưa bù.

Kết quả là tình huống tệ nhất: tham số nhận giá trị sai, *phần dư trông đẹp*, và sai số thật bị giấu vào một biến không ai kiểm. Hệ thống trở nên khó chẩn đoán hơn so với khi tham số bị fix, vì giờ bạn có thêm một nguồn nghi ngờ.

Ba yêu cầu tối thiểu cho một hệ hiệu chuẩn trực tuyến dùng được. *Một*: có cổng, ở bất kỳ mức tinh vi nào trong ba mức ở mục 3B. *Hai*: **ghi lại** khi nào tham số được cập nhật và khi nào bị đóng băng — không có nhật ký này thì không debug được. *Ba*: có giới hạn cứng — nếu tham số đi quá xa giá trị hiệu chuẩn ban đầu một lượng không hợp lý về mặt vật lý, đó là lỗi, không phải một ước lượng; hãy báo động thay vì chấp nhận.

2.5 Suy biến đặc thù của hiệu chuẩn

Ngoài các trường hợp ở A/05, hiệu chuẩn có vài suy biến riêng đáng biết vì chúng hay gặp trong thực tế.

Chuyển động phẳng. Một robot mặt đất di chuyển trên sàn phẳng: mọi tư thế nằm trong một mặt phẳng và mọi phép xoay quanh trục thẳng đứng. Hậu quả nghiêm trọng hơn vẻ ngoài: thành phần thẳng đứng của phép biến đổi ngoại không quan sát được, và roll với pitch của nó cũng vậy. Đây là tình huống mặc định của AGV và robot dịch vụ, tức phần lớn robot thương mại — nên nó không phải một trường hợp biên mà là trường hợp thường gặp nhất. Cách xử lý: hoặc chấp nhận và fix các thành phần ấy từ thiết kế cơ khí, hoặc tạo một quy trình hiệu chuẩn riêng có nâng/nghiêng thiết bị.

Xoay thuần. Không có tịnh tiến nên không có parallax, nên cấu trúc không quan sát được (A/01); và phần tịnh tiến của ngoại thì có quan sát được qua $\omega \times \mathbf{p}$. Đây là một trường hợp thú vị vì nó ngược với trực giác thông thường: xoay thuần tệ cho SLAM nhưng *tốt* cho một phần của hiệu chuẩn ngoại.

Target phẳng và lưỡng nghĩa tư thế. Khi bảng hiệu chuẩn ở gần song song với mặt phẳng ảnh và chiếm phần nhỏ khung hình, bài toán PnP có hai nghiệm gần nhau. Đây là chủ đề của `.. /marker/`

chương 4, và trong hiệu chuẩn nó biểu hiện thành các tư thế bảng bị lật, gây phần dư lớn ở vài ảnh. Cách xử lý: nghiêng bảng đủ nhiều, và dùng bộ giải PnP trả về cả hai nghiệm rồi chọn theo phần dư.

Mọi đặc trưng ở cùng độ sâu. Làm ngoại stereo không quan sát được theo một hướng, như mục 2C đã nêu. Trong thực tế: hiệu chuẩn stereo bằng cách nhìn vào một bức tường phẳng ở một khoảng cách cố định là một sai lầm phổ biến.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [11]	Hiệu chuẩn camera-IMU và đa camera bằng batch MAP với spline liên tục thời gian	Hiện thân rõ nhất của luận đề chương này; đọc tài liệu quy trình thu dữ liệu và đối chiếu với Bảng kích thích ở A/05
OpenVINS [10]	Hiệu chuẩn trực tuyến nội, ngoại và t_d trong MSCKF	Mục 2D và 3B trong một hệ chạy thật; có cờ bật tắt từng nhóm tham số
COLMAP [7]	Tham số nội là biến trong BA; nhiều mô hình camera	Mục 2B — tự hiệu chuẩn ở dạng thuần túy nhất
Basalt [12]	Hiệu chuẩn bằng spline $SE(3)$; double-sphere	Chất lượng kỹ thuật cao; cách xử lý thời gian liên tục cho rolling shutter
[9]	Chỉ cập nhật các hướng quan sát được, dựa trên phân tích phổ	Nguồn của ý tưởng cổng ở mục 3B

MINI PROJECT SLAM · Đi từ hiệu chuẩn tới SLAM bằng cách thả dần biến, trên cùng một bộ mã · 5 ngày

Mục tiêu	đây là bài tập làm sáng tỏ luận đề trung tâm của cả cây rõ nhất. Bạn sẽ viết <i>một</i> bộ giải và dùng nó cho bốn bài toán khác nhau, chỉ bằng cách đổi danh sách biến.
Dữ liệu	một chuỗi hiệu chuẩn với target (tự quay, hoặc từ Kalibr), và một chuỗi EuRoC có chân trị.
Các bước	dựng bài toán MAP bằng Ceres hoặc GTSAM với một hàm phần dư chiếu duy nhất dùng chung; rồi chạy bốn cấu hình: (1) <i>hiệu chuẩn</i> : fix cấu trúc (toạ độ góc bảng), thả nội và tư thế; (2) <i>SfM</i> : thả cấu trúc, fix nội; (3) <i>tự hiệu chuẩn</i> : thả cả hai; (4) <i>tất cả</i> : thêm ngoại và t_d nếu có IMU. Với mỗi cấu hình, in ra hiệp phương sai của mọi tham số hiệu chuẩn và phổ giá trị riêng của Λ ; rồi chạy mỗi cấu hình trên <i>ba</i> đoạn quỹ đạo khác nhau — một đa dạng, một chỉ tịnh tiến, một chỉ xoay quanh một trục.
Xong khi	bạn có một bảng bốn cấu hình nhân ba quỹ đạo, mỗi ô ghi: số chiều không gian rỗng, hiệp phương sai của các tham số, và giá trị hội tụ. Bạn phải quan sát được ba điều: (a) cùng một bộ mã chạy cả bốn bài toán — luận đề trung tâm được chứng minh bằng tay; (b) thả thêm biến làm hiệp phương sai tăng và số chiều không gian rỗng có thể tăng; (c) trên quỹ đạo chỉ xoay quanh một trục, thành phần của $\mathbf{p}$ song song trục ấy có hiệp phương sai lớn hơn hẳn hai thành phần kia — xác nhận mục 2C bằng số của chính bạn.
Nối với	D/21-tu-prototype-len-mass-production biến quy trình này thành một quy trình dây chuyền có chỉ tiêu thời gian và chỉ tiêu chất lượng; D/22-huong-nghien-cuu-con-trong lấy hiệu chuẩn có ý thức observability làm hướng số một.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Target cung cấp *cấu trúc đã biết* — toạ độ 3D của các góc, chính xác vì ta in ra nó. Điều đó làm bài toán dễ hơn không phải vì độ chính xác của bảng mà vì nó *loại bỏ một nhóm biến*: với M góc, bài toán bớt $3M$ ẩn, và quan trọng hơn, kích thước bảng cho đơn vị mét nên gauge tỉ lệ biến

mất. Nói đúng nhất: *target là một cách giảm phương sai bằng cách fix cấu trúc*. Khi không có target, phải thay cái fix ấy bằng *chuyển động* — chuyển động đa dạng làm nhiều ràng buộc căng theo nhiều hướng và thay thế được. Đó là toàn bộ khác biệt giữa lab và robot đang chạy.

- A2. Vì mô tả ấy đúng về *cơ chế* và thiếu về *điều kiện*. Công thức đầy đủ: hiệu chuẩn trực tuyến = mở rộng trạng thái + đủ kích thích + công để biết khi nào không đủ. Trên chuỗi có xoay quanh nhiều trục, phần tịnh tiến của ngoại được kích thích qua số hạng $\omega \times \mathbf{p}$ và nó hội tụ. Trên chuỗi tịnh tiến thuần, số hạng đó bằng không nên $\mathbf{p}$ trôi tự do — và tệ hơn, nó hấp thụ sai số từ các nguồn khác. Chuỗi mà nó “đứng im” thường không phải chuỗi tốt mà là chuỗi mà bạn đã vô tình ghim nó bằng một prior mạnh hoặc bằng damping.
- A3. Thành phần của $\mathbf{p}$ song song với trục quay không xác định được. Vì số hạng duy nhất trong hệ phương trình chứa $\mathbf{p}$ là $\omega \times \mathbf{p}$, và tích có hướng *triệt tiêu* thành phần song song với ω : thay $\mathbf{p}$ bằng $\mathbf{p} + \lambda\omega$ cho đúng cùng một tích có hướng, nên cùng một phép đo. Đó là thủ phạm. Cần xoay quanh ít nhất hai trục độc lập để cả ba thành phần quan sát được. Đây là ví dụ sạch nhất trong cây về việc một lời khuyên thực hành — “xoay quanh cả ba trục” — là hệ quả trực tiếp của một tính chất đại số.
- A4. *Cách tốt*: chuyển động thật sự kích thích mạnh, ma trận thông tin xác định dương tốt, tham số hội tụ về giá trị đúng. *Cách xấu*: tham số không quan sát được nhưng bị ghim bởi một prior mạnh, damping LM, hoặc một khởi tạo được tin quá mức — nó “ổn định” vì nó không đi đâu cả, không phải vì dữ liệu xác nhận. Phép thử rẻ: chạy lại với vài giá trị khởi đầu khác nhau; hội tụ về cùng chỗ bất kể khởi đầu là cách tốt, đi theo khởi đầu là cách xấu. Phép thử thứ hai nhiều thông tin hơn: nhìn *hiệp phương sai* chứ không phải giá trị — tham số quan sát được có hiệp phương sai giảm dần, tham số bị ghim có hiệp phương sai đứng yên ở đúng giá trị prior.
- A5. Ba đại lượng. (1) *Phần dư có cấu trúc theo thời gian*: tích lũy phần dư trung bình theo từng ô lưới trên mặt phẳng ảnh trên một cửa sổ dài, cảnh báo khi độ lệch giữa các ô vượt ngưỡng — rẻ vì chỉ vài chục con số. (2) *Chính giá trị tham số hiệu chuẩn trực tuyến*: một xu hướng đơn điệu (không phải dao động) là dấu hiệu trôi vật lý; điều kiện là công phải hoạt động, nếu không bạn đang theo dõi nhiễu. (3) *Tương quan với nhiệt độ vỏ máy*: bằng chứng mạnh cho trôi nhiệt, và nó còn cho khả năng bù trước bằng một mô hình tham số theo nhiệt độ thay vì chỉ phát hiện.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn có một cái thước và muốn đo một căn phòng. Nhưng cái thước của bạn có thể bị co giãn, và bạn không biết chắc nó dài bao nhiêu.

Có hai cách xử lý.

Cách một: mượn một vật mà bạn *biết chắc* kích thước — một tờ giấy A4 chẳng hạn. Đo nó bằng cái thước của mình, so với kích thước thật, và suy ra thước của mình sai bao nhiêu. Đó là hiệu chuẩn bằng target: bạn mượn một thứ đã biết để đo một thứ chưa biết.

Cách hai: không có tờ giấy nào. Bạn chỉ có căn phòng và cái thước không đáng tin. Nghe như bó tay — nhưng không hẳn. Nếu bạn đo cùng một bức tường *theo nhiều cách khác nhau*, từ nhiều chỗ, theo nhiều hướng, thì các kết quả phải khớp với nhau. Và chính việc chúng *không* khớp cho bạn biết thước sai ở đâu.

Đó là toàn bộ chương này. Có vật chuẩn thì dễ. Không có vật chuẩn thì vẫn làm được, nhưng phải *đo nhiều kiểu khác nhau* — và “nhiều kiểu khác nhau” là chỗ mọi thứ có thể hỏng.

Vì nếu bạn đo mười lần mà lần nào cũng theo đúng một cách, mười kết quả sẽ khớp nhau hoàn hảo — và chúng cùng sai một kiểu. Bạn sẽ rất tự tin và rất nhầm.

Đây là chỗ quan trọng nhất cần hiểu, và nó áp dụng cho mọi thứ chứ không riêng camera: *sự nhất quán giữa nhiều phép đo giống nhau không chứng minh gì cả*. Chỉ sự nhất quán giữa các phép đo *thật sự khác nhau* mới chứng minh được.

Và có một hệ quả thực tế đáng nhớ. Nếu một tham số nào đó không được kiểm bởi cách bạn đang đo, thì đừng để máy tự do chỉnh nó. Nó sẽ chỉnh — và nó sẽ chỉnh sai, đồng thời dùng cái sai ấy để che giấu những lỗi khác. Một cái núm bị vặn bừa còn tệ hơn một cái núm bị khoá.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *biết chuyển động nào kiểm được tham số nào*. Không có trực giác chung cho việc này — với mỗi cấu hình cảm biến mới, phải ngồi xuống viết ra phương trình và xem tham số ấy xuất hiện ở đâu. Với camera-IMU thì nó xuất hiện trong một tích có hướng, và từ đó suy ra phải xoay quanh hai trục. Với một cấu hình khác, câu trả lời sẽ khác.

Một câu: Hiệu chuẩn và SLAM là một bài toán — khác nhau ở chỗ bạn hàn cứng cái gì, và chuyển động là thứ thay thế cho cái hàn.

2.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *hiệu chuẩn và SLAM là cùng một bài toán MAP, khác ở danh sách biến* — là một phát biểu cấu trúc, và mini project chính là phép chứng minh: nếu một bộ mã chạy được cả bốn cấu hình thì mệnh đề đúng cho phạm vi đó. Nó bị bác bỏ nếu có một bài toán hiệu chuẩn không viết được thành một đồ thị nhân tử với cùng loại nhân tử — và có một trường hợp như vậy đáng nêu: hiệu chuẩn quang trắc (A/02 mục 5B) có phần dư dạng khác (cường độ thay vì tọa độ) và một số thành phần (đường cong đáp ứng) được biểu diễn phi tham số. Nó vẫn là MAP, nhưng nó không dùng chung nhân tử với SLAM indirect, nên phát biểu cần được giới hạn: *đúng cho các bài toán hình học; hiệu chuẩn quang trắc là một họ riêng*.

Mệnh đề thứ hai — *thành phần của $\mathbf{p}$ song song trục quay không quan sát được khi chỉ xoay quanh một trục* — suy trực tiếp từ tính chất tích có hướng (cửa a) và kiểm được bằng hiệp phương sai trong mini project (cửa b). Nó bị bác bỏ nếu có một số hạng khác trong hệ phương trình cũng chứa $\mathbf{p}$ — và thực tế có: với gia tốc, số hạng $\boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{p})$ và $\dot{\boldsymbol{\omega}} \times \mathbf{p}$ cũng xuất hiện. Số hạng thứ hai đưa vào một cơ chế quan sát khác nếu tốc độ góc *thay đổi*. Nên phát biểu chặt hơn là: *với tốc độ góc hằng quanh một trục, thành phần song song không quan sát được; với tốc độ góc biến thiên, nó có thể quan sát được yếu*. Tôi chưa kiểm điều này bằng số và nó là một mục trong 99-chua-biet.md.

Mệnh đề thứ ba — *hiệu chuẩn trực tuyến không công tẻ hơn không hiệu chuẩn* — là một phán đoán về đánh đổi. Nó bị bác bỏ nếu một thí nghiệm cho thấy hiệu chuẩn trực tuyến không công vẫn cải thiện độ chính xác trung bình so với tham số fix, trên một tập chuỗi đa dạng. Điều đó *có thể* xảy ra nếu phần lớn chuỗi có kích thích tốt và tham số ban đầu khá sai. Nếu vậy, kết luận đúng không phải “công không cần thiết” mà là “lợi ích trung bình che mất rủi ro đuôi” — và với sản phẩm thì đuôi mới là thứ quan trọng (C/18).

2.7 Kết nối

Chương này là chỗ cây gập lại. Nối lên trên gần như mọi chương của Phần A: A/02-mô-hình-camera cho mô hình và cho kỹ năng chẩn đoán ở mục 4A; A/04-uoc-luong-map-va-bundle-adjustment cho khung MAP mà chương này chỉ đổi danh sách biến; A/05-observability-gauge-va-suy-bien cho toàn bộ phần điều kiện — đây là chương mà Bảng kích thích ở đó trả công hoàn toàn; A/06-bat-dinh-va-lan-truyen-sai-so cho hiệp phương sai tham số, đại lượng nên in ra thay cho RMS.

Nối ngang trong Phần C. C/13-quan-ly-ban-do chia sẻ một câu hỏi với chương này: cái gì được coi là cố định và cái gì được phép đổi theo thời gian — ở đó là bản đồ, ở đây là tham số. C/15-hop-nhat-da-cam-bien là chương mà hiệu chuẩn ngoại trở thành điều kiện tiên quyết: không thể ghép hai cảm biến mà không biết chúng đặt ở đâu so với nhau. C/18-danh-gia-va-phuong-phap-luan cho công cụ để so sánh hai quy trình hiệu chuẩn một cách có kỷ luật.

Nối xuống dưới. D/21-tu-prototype-len-mass-production là chỗ chương này gập thực tế khắc nghiệt nhất: thiết kế đồ gá hiệu chuẩn tự động dưới sáu mươi giây mỗi máy, quyết định hiệu chuẩn từng máy hay dùng tham số chung cho cả lô, phân tích độ nhạy trước khi chốt cơ khí, và thống kê SPC trên tham số qua từng lô hàng. D/22-huong-nghien-cuu-con-trong-lay-hieu

chuẩn trực tuyến có ý thức observability làm hướng số một, và lý do nằm trọn trong mục 3 của chương này: hầu hết hệ thống hoặc hiệu chuẩn mù quang hoặc không hiệu chuẩn, và việc giám sát observability theo thời gian thực rồi quyết định cập nhật biến nào vẫn còn rất thô sơ.

Nổi ngang sang cây khác: . . /IMU_Fusion/ chương 6 bàn hiệu chuẩn quán tính — đặc trưng hoá nhiễu, phương sai Allan, độ lệch — và chương 5 ở đó cho phần observability cho trường hợp quán tính. . . /marker/ chương 12 bàn hiệu chuẩn cho bài toán docking, nơi yêu cầu là milimet nên mọi thứ trong chương này phải làm chặt hơn một bậc.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu chính xác điều mà calibration target cung cấp, và nói vì sao “vì nó chính xác” là câu trả lời sai.
2. Viết công thức đầy đủ cho hiệu chuẩn trực tuyến, gồm cả về điều kiện, và giải thích vì sao thiếu về đó là nguy hiểm.
3. Tự dẫn lại vì sao thành phần của $\mathbf{p}_{\text{cam}}^{\text{imu}}$ song song với trục quay không quan sát được khi chỉ xoay quanh một trục.
4. Hiệu chuẩn trực tuyến của bạn hội tụ nhanh và ổn định. Nêu hai cách giải thích và hai phép thử phân biệt.
5. Nêu bốn đại lượng nên nhìn thay cho RMS khi đánh giá hiệu chuẩn, và nói cái nào chạy được trước khi tối ưu.
6. Nêu ba đại lượng phát hiện trôi hiệu chuẩn tại hiện trường, nơi không có target và không có chân trị.
7. Robot mặt đất đi trên sàn phẳng. Thành phần nào của hiệu chuẩn ngoại không quan sát được, và vì sao đây không phải một trường hợp biên mà là trường hợp thường gặp nhất?

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.

Hợp nhất đa cảm biến (Multi-Sensor Fusion)

PHỤ THUỘC A/04-uoc-luong-map-va-bundle-adjustment (đồ thị nhân tử — thêm cảm biến là thêm loại nhân tử); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai đúng là điều kiện bắt buộc chứ không phải điều tốt nên có); C/14-hieu-chuan-nhu-mot-nhanh-cua-slam (không ghép được hai cảm biến mà không biết chúng đặt ở đâu so với nhau). Chi tiết về IMU nằm ở ../IMU_Fusion/, về marker ở ../marker/.

TÓM TẮT

Chương này ngắn có chủ ý. Phần lớn nội dung chi tiết về từng loại cảm biến đã nằm ở các cây khác, và lặp lại nó ở đây sẽ vi phạm quy tắc một nguồn duy nhất của kho. Cái chương này đóng góp là *nguyên tắc chung* — một nguyên tắc gọn tới mức đáng ngạc nhiên — cộng một bảng tra về việc mỗi cảm biến cho gì và đòi gì. Nguyên tắc: *thêm một nguồn thông tin là thêm một loại nhân tử vào đồ thị, với hiệp phương sai đúng và cổng loại outlier riêng cho từng nguồn*. Ba vế của câu đó đều quan trọng và vế thứ ba hay bị bỏ. Nội dung gồm: nguyên tắc và ba điều kiện của nó; bảng tra sáu nhóm cảm biến với cái chúng cho, điều kiện để cái đó quan sát được, và chế độ hỏng riêng; hai trường hợp đáng nói kỹ hơn vì chúng hay bị làm sai — odometry bánh xe và GNSS; và cuối cùng là một cách nhìn về việc chọn cấu hình cảm biến như một bài toán phá gauge. Nếu chỉ nhớ một điều: *giá trị lớn nhất của một cảm biến phụ không phải độ chính xác cao hơn khi mọi thứ chạy tốt, mà là một chỗ đứng để phục hồi khi thị giác thất bại*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nhân tử (factor)** — một số hạng của hàm mục tiêu, nối các biến mà phần dư của nó phụ thuộc vào (A/04). Thêm một cảm biến nghĩa là định nghĩa một loại nhân tử mới.
- **ràng buộc nonholonomic** — ràng buộc lên *vận tốc* chứ không lên vị trí. Với robot bánh xe không trượt ngang, vận tốc theo phương ngang thân xe bằng không — một ràng buộc miễn phí và rất hữu ích.
- **odometry bánh xe** — ước lượng chuyển động từ số vòng quay bánh xe. Cho tỉ lệ metric, rẻ, tần số cao; hỏng khi trượt.
- **de-skew** — hiệu chỉnh méo của một lượt quét LiDAR do cảm biến di chuyển trong lúc quét. Cùng bản chất với vấn đề rolling shutter ở A/02.
- **hệ toàn cục so với hệ cục bộ** — SLAM làm việc trong một hệ tùy ý do nó tự chọn (gauge); GNSS cho toạ độ trong một hệ toàn cục cố định. Ghép hai hệ đòi một phép biến đổi có bốn bậc tự do phải ước lượng.
- **cổng loại outlier (outlier gating)** — cơ chế từ chối một phép đo trước khi đưa vào bộ giải. Mỗi nguồn cần cổng riêng vì phân bố outlier của chúng khác nhau.

CÂU HỎI MỞ ĐẦU

- Q1. Thêm một cảm biến vào đồ thị là thêm một loại nhân tử — nghe như một việc cơ học. Nếu ba điều kiện phải thoả, và nói điều kiện nào hay bị bỏ nhất.
- Q2. Một GNSS rẻ tiền có sai số vài mét. Thị giác của bạn chính xác tới centimet trong ngắn hạn. Nếu lý do vẫn đáng thêm GNSS, và nói lý do đó không phải “để chính xác hơn”.
- Q3. Odometry bánh xe cho tỉ lệ metric miễn phí, chính xác, tần số cao. Vì sao nó vẫn không thay thế được IMU trong hầu hết hệ thống, và chế độ hỏng của nó khác IMU thế nào?

- Q4.** Bạn dùng chung một ngưỡng Mahalanobis cho mọi loại nhân tử — cách làm gọn gàng và thống nhất. Nêu vì sao đó là một sai lầm, với một ví dụ cụ thể.
- Q5.** Chọn cấu hình cảm biến cho một robot mặt đất. Nêu cách đặt câu hỏi này sao cho nó trở thành một bài toán có tiêu chí thay vì một cuộc tranh luận về sở thích.

3.1 Đặt vấn đề

Thị giác một mình có ba điểm yếu cố hữu mà không thuật toán nào chữa được, vì chúng là tính chất của phép đo chứ không của cách xử lý.

Một — không có tỉ lệ với mono (A/05 mục 3). Hai — hỏng khi không có vân: hành lang trắng, sương mù, bóng tối, camera bị che. Ba — tần số thấp và độ trễ cao: ba mươi khung mỗi giây với vài chục mili giây xử lý là quá chậm cho một vòng điều khiển bay.

Ba điểm yếu ấy được chữa bởi ba loại cảm biến khác nhau, và đó là lý do gần như mọi hệ thống thật đều có nhiều hơn một cảm biến.

Bài toán gốc: *ghép chúng lại thế nào cho đúng?* Câu hỏi này nghe mở, và lịch sử của lĩnh vực có nhiều câu trả lời phức tạp — các sơ đồ lọc phân tầng, các cơ chế trọng số thủ công, các máy trạng thái chuyển đổi giữa các nguồn. Phần lớn chúng đã lỗi thời.

Câu trả lời hiện đại gọn tới mức đáng ngạc nhiên, và nó là hệ quả trực tiếp của A/04: *đồ thị nhân tử không quan tâm nhân tử đến từ đâu*. Một nhân tử là một hàm phần dư cộng một hiệp phương sai. Bộ giải nhìn thấy một tập nhân tử và một tập biến; nó không biết và không cần biết nhân tử nào đến từ camera, nhân tử nào từ IMU, nhân tử nào từ GNSS.

Hệ quả là việc thêm một cảm biến trở thành một việc *cục bộ*: định nghĩa biến mới nếu cần, định nghĩa phần dư, cung cấp Jacobian và hiệp phương sai. Không có gì trong bộ giải thay đổi. Đây là lý do kiến trúc đồ thị nhân tử thắng, và nó là một lợi thế lớn so với bộ lọc, nơi thêm một cảm biến nghĩa là sửa vector trạng thái và mọi ma trận liên quan.

Nhưng — và đây là toàn bộ nội dung còn lại của chương — sự gọn gàng ấy che mất ba điều kiện. Bỏ qua chúng thì việc thêm cảm biến làm hệ thống *tệ đi*, và tệ theo cách khó chẩn đoán.

TRỰC GIÁC

Hình dung một cuộc họp mà mọi người cùng ước lượng một con số — giá của một ngôi nhà chẳng hạn.

Mỗi người đưa ra một con số kèm một mức tự tin: “khoảng ba tỉ, tôi khá chắc” hoặc “khoảng bốn tỉ, nhưng tôi chỉ đoán”. Cách hợp lý để tổng hợp là trung bình có trọng số, với trọng số theo mức tự tin.

Đó chính xác là điều đồ thị nhân tử làm, và nó giải thích cả ba điều kiện.

Điều kiện một — mức tự tin phải trung thực. Nếu một người nói “tôi rất chắc” trong khi thực ra chỉ đoán, ý kiến của người đó sẽ lấn át cả phòng. Và điều tệ nhất: cuộc họp sẽ kết luận rằng *những người khác đang sai*, chứ không phải người kia đang nói quá.

Điều kiện hai — phải biết mọi người đang nói về cùng một thứ. Nếu một người nói giá tính bằng đồng và người khác tính bằng đô la, việc trung bình hoá là vô nghĩa. Trong cảm biến, “cùng một thứ” nghĩa là cùng hệ quy chiếu và cùng mốc thời gian — tức hiệu chuẩn ngoại và đồng bộ thời gian.

Điều kiện ba — phải loại người nói nhảm, và mỗi người nói nhảm theo kiểu khác nhau. Một người thỉnh thoảng nhảm một chữ số; một người khác đôi khi nói về một ngôi nhà hoàn toàn

khác. Hai kiểu sai ấy cần hai cách phát hiện, và một quy tắc chung “loại ai lệch quá xa” sẽ bắt được kiểu thứ hai mà bỏ sót kiểu thứ nhất.

Và một điều nữa, quan trọng nhất cho thực tế. Giá trị lớn nhất của việc có nhiều người trong phòng không phải là trung bình chính xác hơn khi mọi người đều tỉnh táo. Đó là khi *một người ngất xỉu* — cuộc họp vẫn tiếp tục được.

3.2 Nguyên tắc và ba điều kiện

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất.

Nguyên tắc: *thêm một nguồn thông tin là thêm một loại nhân tử vào đồ thị*. Ba điều kiện:

A. Hiệp phương sai phải trung thực

Trọng số $\Omega = \Sigma^{-1}$ quyết định nguồn này được tin bao nhiêu so với mọi nguồn khác. Khai Σ quá nhỏ nghĩa là ra lệnh cho bộ giải tin nguồn này hơn tất cả.

Hậu quả khi sai, và nó đã được nêu ở B/08 khối *Cạm bẫy*: nếu nguồn ấy có sai lệch có hệ thống, sai lệch đó chảy vào toàn bộ nghiệm, kéo theo cả những phần vốn được ước lượng tốt. Và *không có gì cảnh báo* — phần dư của các nguồn khác tăng lên, nhưng bộ giải diễn giải điều đó là “các nguồn kia kém chính xác”.

Quy tắc thực hành: trước khi thêm một loại nhân tử, trả lời được *hiệp phương sai của nó từ đâu ra*. Ba câu trả lời hợp lệ: từ datasheet của cảm biến (đã kiểm bằng đo đặc); từ một phép đo bạn tự làm (như đo σ subpixel ở B/07 mục 4B); hoặc từ một mô hình có cơ sở vật lý. Câu trả lời *không* hợp lệ: “tôi chỉnh cho tới khi nó chạy”.

B. Hệ quy chiếu và thời gian phải đúng

Hai cảm biến đặt ở hai chỗ khác nhau trên cùng thiết bị đo hai thứ khác nhau, và quan hệ giữa chúng là phép biến đổi ngoại (C/14). Sai ngoại thì mọi phần dư mang một sai lệch có hệ thống.

Sai lệch thời gian còn nguy hiểm hơn vì nó *phụ thuộc chuyển động*: một t_d sai không gây hại gì khi thiết bị đứng yên và gây hại lớn khi nó chuyển động nhanh. Hệ quả chẩn đoán đáng nhớ: nếu hệ thống của bạn chính xác khi đi chậm và tệ khi đi nhanh, nghi ngờ t_d trước khi nghi ngờ thuật toán.

Cả hai đều nên được *thả ra làm biến* theo C/14, với cổng observability.

C. Cổng loại outlier riêng cho từng nguồn

Đây là điều kiện hay bị bỏ nhất, và là câu trả lời cho câu hỏi mở đầu thứ tư.

Mỗi nguồn có *phân bố outlier riêng*, và một cổng chung không bắt được cả hai.

Ví dụ cụ thể. Outlier thị giác là các khớp sai: chúng rải rác, độc lập, và thường có phần dư rất lớn — một cổng Mahalanobis đơn giản bắt tốt. Outlier odometry bánh xe là trượt: nó *kéo dài* (cả một đoạn đường), *tương quan* (mọi phép đo trong đoạn đó đều sai cùng chiều), và phần dư của mỗi phép đo riêng lẻ có thể *nhỏ*. Một cổng Mahalanobis trên từng phép đo sẽ không bắt được nó, vì không phép đo nào lệch đủ xa — nhưng tổng của chúng thì lệch rất xa.

Phát hiện trượt cần một cơ chế khác: so tích lũy odometry với tích lũy từ một nguồn độc lập (IMU hoặc thị giác) trên một cửa sổ, và cảnh báo khi chúng phân kỳ. Đây là một công ở *mức chuỗi* chứ không ở mức phép đo, và nó không thể suy ra từ công thị giác.

Bài học tổng quát: *cổng phải được thiết kế theo cơ chế hỏng của từng nguồn*, không theo một công thức thống nhất. Một hệ thống có n loại cảm biến cần n cơ chế phát hiện, và đó là chi phí kỹ thuật thật của việc hợp nhất.

3.3 Bảng tra: mỗi cảm biến cho gì, đòi gì

Bảng 3.1. Sáu nhóm cảm biến. Cột “cho gì” nên đọc theo ngôn ngữ gauge của A/05; cột “chế độ hỏng riêng” là cột quyết định thiết kế cổng ở mục 2C.

Cảm biến	Cho gì	Điều kiện	Chế độ hỏng riêng
IMU	Tỉ lệ metric, hướng trọng lực (roll, pitch), chuyển động tần số cao	Gia tốc phải <i>thay đổi</i> để tỉ lệ quan sát được	Trôi độ lệch; bão hoà khi va đập; nhiễu rung
Odometry bánh xe	Tỉ lệ metric, chuyển động phẳng, ràng buộc nonholonomic	Bánh không trượt; đường kính bánh đã hiệu chuẩn	Trượt — kéo dài, tương quan, phần dư từng phép đo nhỏ
Stereo	Tỉ lệ metric ngay từ khung đầu	Trong tầm hữu ích; đường cơ sở ổn định	Ngoài tầm thì thoái hoá về mono; trôi đường cơ sở theo nhiệt
LiDAR	Cấu trúc metric dày, không cần ánh sáng	Bề mặt phản xạ đủ; de-skew đúng	Hành lang dài (suy biến dọc trục); mưa, bụi, kính
GNSS	Vị trí <i>tuyệt đối</i> trong hệ toàn cục	Thấy trời; ít đa đường	Đa đường trong phố; mất tín hiệu trong nhà; sai lệch tương quan
Marker / UWB	Vị trí <i>tuyệt đối</i> trong hệ đã thiết lập	Trong tầm nhìn / tầm sóng; đã lắp đặt	Ngoài tầm; marker bị che, bẩn, tháo mất

Bảng này nên đọc cùng với Bảng ở 00-map: mỗi cảm biến trong cột “cho gì” thực chất là *phá một gauge*. IMU và odometry và stereo phá gauge tỉ lệ; IMU còn ghim roll với pitch; GNSS và marker phá cả gauge tịnh tiến lẫn yaw. Nhìn theo cách đó, việc chọn cấu hình cảm biến trở thành một câu hỏi có tiêu chí — đó là câu trả lời cho câu hỏi mở đầu thứ năm.

3.4 Hai trường hợp hay bị làm sai

A. Odometry bánh xe

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và odometry bánh xe đáng một mục riêng vì nó là cảm biến *bị đánh giá thấp nhất* cho robot mặt đất.

Cái nó cho, và ba thứ này đều đáng giá. *Tỉ lệ metric* — đường kính bánh đã biết cho một thước đo, giống đường cơ sở stereo. *Tần số cao và độ trễ thấp* — thường vài trăm Hz, tốt cho vòng điều khiển. *Ràng buộc nonholonomic* — và đây là phần miễn phí ít người khai thác: với robot bánh xe không trượt ngang, vận tốc theo phương ngang thân xe *bằng không*, và với robot chạy trên sàn

phẳng, vận tốc thẳng đứng cũng bằng không. Đó là hai ràng buộc không cần cảm biến nào cả — chỉ cần biết loại robot.

Vì sao nó vẫn không thay thế IMU? Vì chế độ hỏng của hai cảm biến *bù nhau* chứ không trùng nhau. IMU trôi nhưng không bao giờ “nói dối” — nó luôn đo đúng cái nó đo, chỉ là có độ lệch. Odometry không trôi khi mọi thứ bình thường, nhưng khi bánh trượt thì nó nói dối một cách tự tin: nó báo robot đã đi ba mét trong khi robot đứng yên quay bánh.

Hai kiểu sai ấy đòi hai cách xử lý ngược nhau. Với IMU, ta *mô hình hoá* độ lệch và ước lượng nó. Với odometry, ta phải *phát hiện và loại* — không có mô hình nào cho việc trượt. Và như mục 2C đã nói, việc phát hiện ấy cần một cổng ở mức chuỗi chứ không ở mức phép đo.

Kết luận thực dụng cho robot mặt đất: *dùng cả hai*. IMU cho hướng và cho chuyển động ngắn hạn khi bánh trượt; odometry cho tỉ lệ và cho chuyển động khi IMU trôi. Chúng kiểm chéo lẫn nhau, và chính việc kiểm chéo ấy là cơ chế phát hiện trượt.

B. GNSS và hai hệ quy chiếu

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

Vấn đề đầu tiên khi thêm GNSS không phải độ chính xác mà là *hệ quy chiếu*. SLAM làm việc trong một hệ tuỳ ý mà nó tự chọn khi khởi tạo — đó chính là gauge ở A/05. GNSS cho toạ độ trong một hệ toàn cục cố định. Hai hệ liên hệ qua một phép biến đổi *chưa biết* với bốn bậc tự do: ba tịnh tiến và một yaw (roll và pitch đã bị trọng lực ghim nếu có IMU).

Phép biến đổi ấy phải được *ước lượng*, và nó là một biến trong đồ thị như mọi biến khác. Điều kiện để nó quan sát được: quỹ đạo phải đủ dài và đủ cong — một đoạn thẳng không xác định được yaw của phép căn chỉnh.

Vì sao vẫn đáng thêm GNSS dù nó kém chính xác hơn thị giác? Lý do *không* phải độ chính xác mà là *bản chất sai số*. Sai số thị giác *tích lũy* — nó nhỏ ở ngắn hạn và lớn vô hạn ở dài hạn. Sai số GNSS *bị chặn* — nó vài mét ở mọi thời điểm, hôm nay cũng như một năm sau. Hai đặc tính ấy bù nhau hoàn hảo: thị giác cho độ chính xác cục bộ, GNSS cho việc không trôi.

Đây là một ví dụ của một nguyên tắc chung đáng nhớ: *khi ghép cảm biến, hãy tìm nguồn có đặc tính sai số bù nhau, không phải nguồn chính xác nhất*. Hai cảm biến cùng trôi theo cùng kiểu không giúp được nhau; một cảm biến kém chính xác nhưng không trôi lại vô giá.

Cảnh báo về cổng, đúng theo mục 2C: sai số GNSS trong phố có *đa đường* — tín hiệu phản xạ từ toà nhà — và nó gây sai lệch *tương quan theo thời gian*, có thể hàng chục mét và kéo dài nhiều giây. Một cổng dựa trên từng phép đo sẽ không bắt được, vì hiệp phương sai mà máy thu báo cũng lạc quan trong tình huống ấy. Cần một cổng so với dự đoán từ thị giác trên một cửa sổ.

CẦN NHỚ

Giá trị lớn nhất của một cảm biến phụ không phải độ chính xác cao hơn khi mọi thứ chạy tốt, mà là một chỗ đứng để phục hồi khi nguồn chính thất bại.

Đây là điều mà các bảng so sánh độ chính xác không nắm bắt được, và nó là lý do quan trọng nhất để hợp nhất cảm biến trong sản phẩm.

Ba biểu hiện cụ thể, đã xuất hiện rải rác trong cây. *Một*: khi thị giác mất bám, odometry và IMU giữ cho hệ thống có một ước lượng thô — đủ để tiếp tục vòng điều khiển, và đủ để thu hẹp không gian tìm kiếm khi relocalize (B/12 mục 2C). *Hai*: tiên nghiệm chuyển động từ cảm biến phụ cải thiện chính front-end thị giác, theo ba đường cùng lúc (B/07 mục 5C). *Ba*: khi hai nguồn bất đồng, đó là một tín hiệu chẩn đoán — và một hệ chỉ có một nguồn thì không bao giờ có tín hiệu ấy.

Hệ quả cho việc đánh giá: đo một hệ hợp nhất bằng ATE trung bình trên một chuỗi sạch sẽ bỏ sót phần lớn giá trị của nó. Phải đo trên các chuỗi có sự cố — che camera, tắt đèn, bánh trượt — và đo tỉ lệ sống sót chứ không chỉ độ chính xác. C/18 bàn cách thiết kế phép đo đó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
VINS-Fusion [13]	Camera + IMU + GNSS tùy chọn; căn chỉnh hệ cục bộ với hệ toàn cục	Mục 4B trong một hệ thật; mã ngắn dễ đọc
GTSAM [14]	Thư viện nhân tử cho nhiều loại cảm biến; dễ thêm loại mới	Hiện thân của nguyên tắc ở mục 2 — thêm cảm biến là viết một lớp nhân tử
Kimera [15]	Camera + IMU + ngữ nghĩa trong một đồ thị	Cho thấy cả thông tin ngữ nghĩa cũng chỉ là một loại nhân tử
maplab [6]	Nhiều cảm biến, nhiều phiên, hiệu chuẩn ngoại trong cùng khung	Kết hợp chương này với C/13 và C/14
OpenVINS [8]	Camera + IMU với hiệu chuẩn ngoại và t_d trực tuyến	Mục 2B — thả ngoại và thời gian ra làm biến

MINI PROJECT SLAM · Thêm odometry bánh xe vào một hệ VIO, rồi đo giá trị thật của nó · 3 ngày

Mục tiêu	đo giá trị của một cảm biến phụ theo cách đúng — không phải bằng ATE trung bình mà bằng tỉ lệ sống sót khi có sự cố.
Dữ liệu	một robot mặt đất có bánh xe encoder, camera và IMU; tự thu ba loại chuỗi: chuỗi sạch, chuỗi có che camera vài giây, và chuỗi có bánh trượt (chạy lên một tấm thảm trơn hoặc nâng một bánh).
Các bước	thêm nhân tử odometry và ràng buộc nonholonomic vào một hệ VIO có sẵn; hiệu chuẩn đường kính bánh và ngoại bánh–camera theo C/14; cài một cổng phát hiện trượt ở mức chuỗi theo mục 2C — so tích lũy odometry với tích lũy IMU trên một cửa sổ; chạy bốn cấu hình (VIO thuần, VIO + odom không cổng, VIO + odom có cổng, VIO + odom + nonholonomic) trên cả ba loại chuỗi.
Xong khi	bạn có một bảng bốn cấu hình nhân ba chuỗi với hai cột: ATE và <i>có sống sót không</i> . Bạn phải quan sát được: (a) trên chuỗi sạch, thêm odometry cải thiện ATE ít — đúng như dự đoán, vì đó không phải giá trị chính của nó; (b) trên chuỗi che camera, chỉ các cấu hình có odometry sống sót; (c) trên chuỗi trượt, cấu hình <i>không cổng</i> tệ hơn cả VIO thuần — xác nhận mục 2C bằng dữ liệu của chính bạn, và đó là kết quả đáng giá nhất của bài này.
Nối với	C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo “tỉ lệ sống sót”; D/19-ben-vung-trong-the-gioi-thuc mở rộng danh mục sự cố cần thử.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba điều kiện: (1) *hiệp phương sai trung thực* — trọng số quyết định nguồn này được tin bao nhiêu so với mọi nguồn khác, và khai quá nhỏ làm sai lệch có hệ thống của nó chảy vào toàn bộ nghiệm mà không có gì cảnh báo; (2) *hệ quy chiếu và thời gian đúng* — ngoại và t_d , cả hai nên thả ra làm biến theo C/14; (3) *cổng loại outlier riêng cho từng nguồn*. Điều kiện (3) hay bị bỏ nhất, vì nó đòi hiểu cơ chế hỏng của từng cảm biến chứ không suy ra được từ một công thức chung.
- A2. Vì *bản chất sai số bù nhau*, không phải vì độ chính xác. Sai số thị giác tích lũy — nhỏ ở ngắn hạn, lớn vô hạn ở dài hạn. Sai số GNSS bị chặn — vài mét ở mọi thời điểm, hôm nay cũng như một năm sau. Ghép lại: thị giác cho độ chính xác cục bộ, GNSS cho việc không trôi. Nguyên tắc chung đáng nhớ: khi ghép cảm biến, tìm nguồn có *đặc tính sai số bù nhau*, không phải nguồn

chính xác nhất — hai cảm biến cùng trôi theo cùng kiểu không giúp được nhau. Lưu ý: thêm GNSS đòi ước lượng một phép biến đổi bốn bậc tự do giữa hệ cục bộ và hệ toàn cục, và nó cần quỹ đạo đủ cong để quan sát được yaw.

- A3. Vì chế độ hỏng của hai cảm biến *bù nhau* chứ không trùng nhau. IMU trôi nhưng không bao giờ nói dối — nó luôn đo đúng cái nó đo, chỉ có độ lệch, nên ta *mô hình hoá* độ lệch và ước lượng nó. Odometry không trôi khi bình thường, nhưng khi bánh trượt thì nó nói dối một cách tự tin — báo đã đi ba mét trong khi robot đứng yên quay bánh — và không có mô hình nào cho việc trượt, nên phải *phát hiện và loại*. Hai kiểu sai đòi hai cách xử lý ngược nhau. Kết luận: dùng cả hai, và chính việc chúng kiểm chéo lẫn nhau là cơ chế phát hiện trượt.
- A4. Vì mỗi nguồn có *phân bố outlier riêng*. Ví dụ cụ thể: outlier thị giác là khớp sai — rải rác, độc lập, phần dư rất lớn — nên công Mahalanobis trên từng phép đo bất tốt. Outlier odometry là trượt — *kéo dài cả một đoạn, tương quan* (mọi phép đo sai cùng chiều), và phần dư của từng phép đo riêng lẻ có thể *nhỏ*. Công Mahalanobis không bắt được vì không phép đo nào lệch đủ xa, nhưng tổng của chúng thì lệch rất xa. Cần một công ở *mức chuỗi*: so tích lũy odometry với tích lũy từ nguồn độc lập trên một cửa sổ. Bài học: công phải thiết kế theo cơ chế hỏng của từng nguồn; n loại cảm biến cần n cơ chế phát hiện.
- A5. Đặt nó thành một bài toán *phá gauge*: liệt kê các bậc tự do không quan sát được của cấu hình thuần thị giác (bảy với mono, theo A/05), rồi hỏi mỗi cảm biến phá được bậc nào và với điều kiện gì — Bảng 3.1 là bảng trả lời. Thêm hai tiêu chí nữa để bài toán đầy đủ: (a) *đặc tính sai số có bù nhau không* (tích lũy so với bị chặn); (b) *chế độ hỏng có độc lập không* — hai cảm biến cùng chết khi mất điện hoặc cùng chết trong sương mù thì không cho khả năng phục hồi. Với robot mặt đất, kết quả thường là stereo + IMU + odometry bánh xe, và D/21 lập luận vì sao đó là điểm ngọt.

NÓI LẠI THẬT ĐƠN GIẢN

Hình dung một cuộc họp để đoán giá một ngôi nhà. Mỗi người đưa ra một con số kèm mức tự tin. Cách hợp lý để tổng hợp là trung bình có trọng số theo mức tự tin. Đó chính xác là điều một hệ hợp nhất cảm biến làm.

Từ hình ảnh đó suy ra được cả ba chỗ có thể hỏng.

Nếu một người nói “tôi rất chắc” trong khi thực ra chỉ đoán bừa, ý kiến người đó lấn át cả phòng — và tệ hơn, cuộc họp sẽ kết luận rằng những người kia đang sai chứ không phải người này đang nói quá. Đây là chuyện xảy ra khi bạn khai mức tin cậy của một cảm biến quá cao.

Nếu một người tính bằng đồng và người khác tính bằng đô la, trung bình hoá là vô nghĩa. Với cảm biến, điều này nghĩa là phải biết chúng đặt ở đâu so với nhau và đo vào lúc nào.

Và nếu có người nói nhầm, phải loại — nhưng mỗi người nói nhầm theo một kiểu. Một người thỉnh thoảng nhầm một chữ số, dễ thấy ngay. Một người khác thì nói rất hợp lý nhưng về một ngôi nhà hoàn toàn khác, và cả buổi không ai phát hiện. Hai kiểu ấy cần hai cách bắt. Đây là chỗ mà một quy tắc chung “loại ai lệch quá xa” sẽ bắt kiểu thứ nhất và bỏ sót kiểu thứ hai — đúng cái xảy ra với bánh xe bị trượt.

Nhưng điều quan trọng nhất thì không nằm ở việc trung bình. Giá trị lớn nhất của việc có nhiều người trong phòng không phải là con số chính xác hơn khi mọi người đều tỉnh táo. Đó là lúc *một người ngất xỉu* — cuộc họp vẫn tiếp tục được.

Với một cái máy, “ngất xỉu” nghĩa là camera bị che, đèn tắt, hay đi vào một hành lang trắng trơn. Một hệ chỉ có camera sẽ dừng lại ở đó. Một hệ có thêm vài nguồn khác thì vẫn đi tiếp được — kém chính xác hơn, nhưng vẫn đi. Và với một sản phẩm, khác biệt giữa “kém chính xác” và “dừng lại” là khác biệt giữa được và không được.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *biết mỗi cảm biến nói dối theo kiểu nào*. Không có quy tắc chung; phải hiểu vật lý của từng cảm biến. Bánh xe nói dối khi trượt; GPS nói dối khi tín hiệu dội vào toà nhà; camera nói dối khi có một chiếc xe tải đi ngang chiếm hết khung hình. Ba kiểu nói dối ấy chẳng liên quan gì tới nhau, và mỗi kiểu cần một cách bắt riêng.

Một câu: Thêm một cảm biến là thêm một ý kiến vào cuộc họp — và điều quan trọng nhất không phải nó nói gì, mà là nó có thành thật về mức nó chắc chắn hay không.

3.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *thêm cảm biến là thêm nhân tử, không đổi bộ giải* — là hệ quả cấu trúc của đồ thị nhân tử (cửa a). Nó bị bác bỏ nếu có một loại phép đo không viết được thành một nhân tử với Jacobian và hiệp phương sai — và có: các phép đo có *cấu trúc rời rạc*, chẳng hạn “robot đang ở trong phòng A” từ một bộ phân loại, không phải một phần dư liên tục. Chúng cần các kỹ thuật khác (ước lượng đa giả thuyết), và đó là một giới hạn thật của khung này chứ không phải một ngoại lệ nhỏ.

Mệnh đề thứ hai — *mỗi nguồn cần cổng riêng* — kiểm được bằng mini project phần chuỗi trượt. Nó bị bác bỏ nếu một cổng chung đủ tốt cho mọi nguồn trên một tập sự cố đa dạng. Tôi cho là không, dựa trên lập luận về phân bố outlier, nhưng mức độ nghiêm trọng thì tôi chưa đo cho các cặp cảm biến ngoài odometry–thị giác.

Mệnh đề thứ ba — *giá trị chính của cảm biến phụ là khả năng phục hồi, không phải độ chính xác* — là một phán đoán về ưu tiên, và nó phụ thuộc ứng dụng. Với một hệ quét đo đặc chạy trong điều kiện kiểm soát, độ chính xác đúng là tiêu chí chính và mệnh đề này sai. Nó đúng cho robot hoạt động không giám sát, tức cho mục tiêu đã nêu ở 00-map (*tôi suy ra từ mục tiêu ấy, chưa đổi chiều nguồn*).

3.6 Kết nối

Nói lên trên. A/04-uoc-luong-map-va-bundle-adjustment cho khung mà nguyên tắc ở mục 2 là hệ quả trực tiếp. A/05-observability-gauge-va-suy-bien cho ngôn ngữ để đọc Bảng 3.1: mỗi cảm biến phá một gauge. A/06-bat-dinh-va-lan-truyen-sai-so là chương mà điều kiện một ở mục 2A phụ thuộc hoàn toàn vào — và nó cũng cảnh báo rằng hiệp phương sai đáng tin là thứ khó có. C/14-hieu-chuan-nhu-mot-nhanh-cua-slam là điều kiện hai.

Nói xuống dưới. C/18-danh-gia-va-phuong-phap-luan là chỗ khối *Cần nhớ* trở thành phương pháp: đo một hệ hợp nhất bằng ATE trung bình là bỏ sót phần lớn giá trị của nó. D/19-ben-vung-trong-the-gioi-thuc mở rộng danh mục sự cố mà cảm biến phụ giúp vượt qua. D/21-tu-prototype-len-mass-production là chỗ việc chọn cấu hình cảm biến trở thành một quyết định có chi phí BOM, có ngân sách điện, và có hệ quả về dung sai cơ khí — và là chỗ lập luận “stereo + IMU + odometry là điểm ngọt cho robot mặt đất” được trình bày đầy đủ.

Nói ngang. . . /IMU_Fusion/ là cây đầy đủ về nhánh quán tính: tiền tích phân (chương 7), bộ lọc và tối ưu (chương 8, 9), khởi tạo (chương 10), ràng buộc chuyển động và đa cảm biến (chương 12). Chương này cố ý không lặp lại. . . /marker/ là cây về nhánh fiducial, nơi marker đóng vai trò neo tuyệt đối — và nó cho thấy cái giá thật của giải pháp ấy: lắp đặt, hiệu chuẩn, bảo trì.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu nguyên tắc chung để thêm một cảm biến, và ba điều kiện của nó. Điều kiện nào hay bị bỏ nhất và vì sao?
2. Vì sao một nhân tử có hiệp phương sai sai lại tệ hơn không có nhân tử đó, và vì sao không có gì cảnh báo?
3. Outlier thị giác và outlier odometry khác nhau thế nào về phân bố, và vì sao một cổng chung không bắt được cả hai?
4. Odometry bánh xe và IMU đều cho tỉ lệ metric. Chế độ hỏng của chúng khác nhau thế nào, và vì sao điều đó nghĩa là nên dùng cả hai?

5. Thêm GNSS đòi ước lượng một phép biến đổi mấy bậc tự do, và điều kiện nào để nó quan sát được?
6. Nêu nguyên tắc chọn cảm biến theo “đặc tính sai số bù nhau”, và cho một ví dụ về hai cảm biến *không* bù nhau.
7. Vì sao đo một hệ hợp nhất bằng ATE trung bình trên chuỗi sạch là bỏ sót phần lớn giá trị của nó?

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.

Dense mapping và biểu diễn (Dense Mapping and Representation)

PHỤ THUỘC A/01-hình-học-da-thi (triangulation, stereo); A/06-bat-dinh-va-lan-truyen-sai-so ($\sigma_z \approx z^2 \sigma_d / (bf)$) — quan hệ này quyết định chất lượng mọi bản đồ dày); C/13-quan-ly-ban-do (bài toán kích thước, ở đây gay gắt hơn nhiều bậc).

TÓM TẮT

Bản đồ thưa gồm vài nghìn điểm đủ để định vị nhưng không đủ để *làm gì với thế giới*. Một robot cần biết chỗ nào đi được; một hệ AR cần biết bề mặt ở đâu để đặt vật ảo lên; một hệ quét cần một mô hình có thể in ra. Chương này về các biểu diễn dày và về một câu hỏi mà ít ai hỏi thẳng: *ai cần chúng, để làm gì, và cái giá là bao nhiêu?* Nội dung có bốn phần. Một: lấy độ sâu dày từ ảnh — stereo, multi-view, và giới hạn vật lý mà không thuật toán nào vượt qua được. Hai: bốn họ biểu diễn cổ điển — TSDF, surfel, octree, mesh — với tiêu chí chọn dựa trên *việc bạn định dùng bản đồ để làm gì*, không dựa trên chất lượng hình ảnh. Ba: các biểu diễn học được — NeRF và Gaussian splatting — là hướng nóng nhất hiện nay, với một đánh giá thẳng thắn về chỗ chúng đã tốt và chỗ chưa. Bốn: một đánh đổi mà tài liệu hay bỏ qua. Nếu chỉ nhớ một điều: *bản đồ dày đẹp hơn nhưng độ chính xác tư thế thường không hơn bản đồ thưa* — và hiểu vì sao là hiểu được vị trí đúng của cả chương này trong một hệ thống.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bản đồ thưa (sparse map)** — tập vài nghìn tới vài trăm nghìn điểm 3D rời rạc, mỗi điểm ứng với một đặc trưng ảnh. Đủ để định vị, không đủ để mô tả bề mặt.
- **bản đồ dày (dense map)** — biểu diễn mô tả *bề mặt* hoặc *thể tích* của cảnh, không chỉ vài điểm rời rạc. Tồn bộ nhớ hơn hàng trăm tới hàng nghìn lần.
- **TSDF** — *truncated signed distance function* — mỗi voxel lưu khoảng cách có dấu tới bề mặt gần nhất, cắt ở một ngưỡng. Bề mặt là tập mức không. Hợp nhất nhiều lượt quét bằng trung bình có trọng số.
- **surfel** — một mảnh bề mặt nhỏ có vị trí, pháp tuyến, bán kính và màu. Biểu diễn không cần lưới, nên dễ biến dạng khi bản đồ được hiệu chỉnh.
- **octree** — cấu trúc cây chia không gian thành tám phần đệ quy, chỉ chia sâu ở chỗ có nội dung. Tiết kiệm bộ nhớ cho không gian phần lớn là trống.
- **ESDF** — *euclidean signed distance field* — mỗi voxel lưu khoảng cách tới chương ngại gần nhất. Đây là thứ mà bộ lập kế hoạch đường đi thật sự cần, không phải một mô hình đẹp.
- **NeRF** — *neural radiance field* — biểu diễn cảnh bằng một mạng ánh xạ từ (vị trí, hướng nhìn) sang (màu, mật độ), render bằng ray marching.
- **3D Gaussian Splatting** — biểu diễn cảnh bằng hàng triệu Gaussian 3D dị hướng có màu và độ trong, render bằng phép chiếu và trộn — nhanh hơn NeRF nhiều bậc vì không cần lấy mẫu dọc tia.

CÂU HỎI MỞ ĐẦU

- Q1.** Bản đồ dày dùng nhiều dữ liệu hơn bản đồ thưa hàng trăm lần. Vì sao nó thường *không* cho tư thế chính xác hơn?
- Q2.** Bộ lập kế hoạch đường đi của bạn cần biết chỗ nào đi được. Nêu biểu diễn nó thật sự cần, và nói vì sao một mesh đẹp không phải câu trả lời.
- Q3.** Stereo cho độ sâu ở mọi pixel có vấn. Nêu ba vùng mà nó chắc chắn thất bại, và nói vì sao đó là giới hạn vật lý chứ không phải giới hạn thuật toán.
- Q4.** 3D Gaussian Splatting cho hình ảnh đẹp hơn hẳn TSDF với cùng dữ liệu. Nêu ba lý do vì sao một robot sản phẩm năm 2026 có thể vẫn nên dùng TSDF.
- Q5.** Bạn có một bản đồ dày và loop closure vừa xảy ra, dịch chuyển các keyframe đi vài mét. Chuyện gì phải xảy ra với bản đồ dày, và vì sao đó là vấn đề khó hơn với TSDF so với surfel?

4.1 Đặt vấn đề

Mọi chương trước làm việc với bản đồ thưa: vài nghìn điểm 3D, mỗi điểm ứng với một góc hoặc một đốm trong ảnh. Bản đồ ấy đủ cho việc định vị — đó là toàn bộ nội dung Phần B — và nó rẻ.

Nó không đủ cho gần như mọi thứ khác. Một robot không thể lập kế hoạch đường đi từ một đám mây điểm thưa, vì giữa hai điểm có thể là không gian trống hoặc có thể là một bức tường và bản đồ không nói. Một hệ AR không thể đặt một vật ảo lên bàn nếu nó không biết mặt bàn ở đâu. Một hệ quét không thể xuất ra một mô hình.

Bài toán gốc: từ cùng những ảnh ấy, dựng một mô tả bề mặt hoặc thể tích của cảnh.

Ai gặp bài toán này? Ba nhóm với ba yêu cầu rất khác nhau, và gộp chúng là một lỗi phổ biến vì nó dẫn tới việc chọn sai biểu diễn.

Nhóm một — điều hướng. Robot cần biết chỗ nào đi được. Yêu cầu: đúng về mặt an toàn (thà báo có chướng ngại khi không có còn hơn ngược lại), cập nhật nhanh, và ở một biểu diễn mà bộ lập kế hoạch dùng trực tiếp được. *Không cần đẹp.*

Nhóm hai — tương tác. AR, robot gấp vật. Cần bề mặt chính xác ở vùng quan tâm, với pháp tuyến. Cần đẹp ở mức đủ để không gây khó chịu.

Nhóm ba — tái tạo. Quét để lưu trữ, để in 3D, để dựng mô hình. Cần chính xác và đẹp; *không cần* thời gian thực.

Ba nhóm ấy dẫn tới ba lựa chọn biểu diễn khác nhau, và phần lớn nhầm lẫn trong lĩnh vực này đến từ việc đọc một bài báo viết cho nhóm ba rồi áp dụng cho nhóm một.

Có một câu hỏi thứ hai, ít được hỏi thẳng và quan trọng hơn: *bản đồ dày có giúp việc định vị không?* Trực giác nói có — nhiều dữ liệu hơn thì tốt hơn. Mục 4 lập luận rằng câu trả lời chủ yếu là *không*, và hiểu vì sao là hiểu được vị trí đúng của cả chương này.

TRỰC GIÁC

Hình dung sự khác nhau giữa một tấm bản đồ du lịch và một mô hình sa bàn.

Bản đồ du lịch có vài chục ký hiệu: nhà thờ ở đây, ga tàu ở kia, ngã tư ở đó. Nó đủ để bạn biết mình đang ở đâu và đi hướng nào. Nó nhẹ, bạn gấp bỏ túi được, và nó không nói gì về việc vỉa hè cao bao nhiêu centimet.

Sa bàn thì mô phỏng cả thành phố — từng bức tường, từng bậc thềm. Nó nặng, tốn công dựng, nhưng nó trả lời được những câu mà bản đồ không trả lời được: cái xe đẩy của tôi có qua được chỗ này không? Cái kệ này có đủ chỗ đặt vật kia không?

Hai thứ phục vụ hai mục đích, và đây là chỗ trực giác hay sai: *sa bàn không giúp bạn định vị tốt hơn bản đồ*. Để biết mình đang ở đâu, bạn vẫn nhìn nhà thờ và ga tàu — những thứ dễ nhận. Cả một bức tường phẳng của sa bàn không nói gì thêm về vị trí của bạn, vì mọi chỗ trên bức tường ấy trông như nhau.

Đó là toàn bộ mục 4 của chương, phát biểu bằng hình ảnh: thông tin về *vị trí* nằm ở những chỗ *khác biệt*, và bản đồ thừa đã lấy hết những chỗ ấy rồi. Phần dày thêm vào là phần *không khác biệt* — hữu ích để biết hình dạng thế giới, vô ích để biết mình ở đâu trong đó.

Còn một chuyện nữa. Khi bạn phát hiện mình vẽ sai và phải kéo bản đồ cho đúng (loop closure), thì với tấm bản đồ du lịch, bạn chỉ cần dịch vài chục ký hiệu. Với sa bàn, bạn phải *làm lại* cả một vùng — và đó là lý do bản đồ dày khó sống chung với SLAM hơn nhiều so với vẽ ngoài.

4.2 Lấy độ sâu dày

A. Stereo và giới hạn vật lý

Với cặp stereo đã hiệu chỉnh, bài toán là tìm disparity cho *mọi* pixel. Khác với B/07, nơi ta chọn vài trăm điểm tốt, ở đây ta phải xử lý cả những pixel không có gì đặc biệt.

SGM [16] là phương pháp cổ điển vẫn được dùng rộng rãi, và ý tưởng của nó đáng biết. Bài toán đúng là tối ưu một hàm năng lượng hai chiều với số hạng dữ liệu (độ khớp) và số hạng trơn (disparity của pixel cạnh nhau nên gần nhau) — nhưng tối ưu năng lượng 2D là NP-khó. SGM xấp xỉ bằng cách tối ưu chính xác dọc *nhiều đường một chiều* qua mỗi pixel rồi cộng chi phí lại. Rẻ, song song hoá tốt, và chất lượng đủ cho hầu hết ứng dụng.

Ba vùng mà stereo *chắc chắn* thất bại, và đây là câu trả lời cho câu hỏi mở đầu thứ ba — điều quan trọng là cả ba là giới hạn *vật lý*, không phải giới hạn thuật toán.

Một — vùng không có vân. Một bức tường trắng: mọi pixel trong một hàng trông giống hệt nhau, nên không có thông tin nào xác định disparity. Không thuật toán nào tạo ra thông tin từ chỗ không có; cái tốt nhất có thể làm là *nội suy* từ biên và *thừa nhận* rằng đó là nội suy.

Hai — vùng bị che khuất. Một phần cảnh nhìn thấy từ camera trái nhưng bị che với camera phải. Không có tương ứng nào tồn tại. Phát hiện được bằng kiểm tra trái-phải, và câu trả lời đúng là đánh dấu “không biết”.

Ba — bề mặt phản chiếu hoặc trong suốt. Gương, kính, nước, kim loại bóng. Ở đây tệ hơn hai trường hợp trên: khớp điểm *thành công* nhưng nó khớp với ảnh phản chiếu, nên độ sâu thu được là độ sâu của một vật không tồn tại ở đó. Đây là kiểu hỏng im lặng, và nó là lý do robot đâm vào cửa kính.

Ngoài ba vùng ấy, có một giới hạn định lượng chi phối mọi thứ: $\sigma_z \approx z^2 \sigma_d / (bf)$ từ A/06. Nó nói rằng dù thuật toán hoàn hảo, chất lượng độ sâu suy giảm theo bình phương khoảng cách. Một bản đồ dày ở xa là một bản đồ dày *sai*, và vẽ nó ra trông vẫn đẹp.

B. Multi-view và độ sâu học được

Với nhiều khung thay vì hai, ta có multi-view stereo. Lợi ích: nhiều đường cơ sở khác nhau nên nhiều tầm độ sâu được phục vụ tốt; và nhiều góc nhìn nên ít bị che khuất hơn. PatchMatch stereo và các biến thể là chuẩn cho tái tạo ngoại tuyến.

Độ sâu học được (B/08 mục 4C) giải quyết được vấn đề vùng không có vân — vì mô hình mang tiên nghiệm về việc các bề mặt thường trông thế nào. Đây là một đóng góp thật và là chỗ học máy giúp rõ ràng.

Cảnh báo vẫn như ở B/08: kết quả *trông* chắc chắn ở mọi pixel, kể cả những pixel mà hình học không nói gì. Một bản đồ dày hoàn toàn từ độ sâu học được là một bản đồ mà một phần là phép đo và một phần là phỏng đoán, *không có gì phân biệt hai phần*. Với ứng dụng an toàn — robot tránh chướng ngại — đó là điều nguy hiểm. Cách xử lý đúng: giữ hai kênh, một cho giá trị và một cho nguồn gốc (đo được hay suy ra), và để bộ lập kế hoạch đối xử với chúng khác nhau.

4.3 Bốn họ biểu diễn cổ điển

Bảng 4.1. Bốn họ biểu diễn. Cột “dùng cho” là tiêu chí chọn đúng — chọn theo việc bạn định làm gì với bản đồ, không theo chất lượng hình ảnh.

Biểu diễn	Cơ chế	Dùng cho	Hỏng ở
TSDF	Voxel lưu khoảng cách có dấu; bề mặt là tập mức không; hợp nhất bằng trung bình có trọng số	Tái tạo bề mặt chất lượng cao; quét trong nhà	Bộ nhớ theo thể tích; khó biến dạng khi loop closure
Surfel	Các mảnh bề mặt rời có vị trí, pháp tuyến, bán kính	Khi bản đồ phải <i>biến dạng</i> — ElasticFusion	Không có bề mặt liền; khó truy vấn “điểm này trông hay đặc”
Octree / occupancy	Chia không gian đệ quy; mỗi ô lưu xác suất bị chiếm	Điều hướng — trả lời trực tiếp “chỗ này đi được không”	Không mô tả bề mặt chính xác; độ phân giải hữu hạn
Mesh	Lưới tam giác	Hiển thị, in 3D, mô phỏng vật lý	Đắt để cập nhật gia tăng; khó xử lý topology thay đổi

Hai điểm đáng nói kỹ hơn bảng.

A. Điều hướng cần ESDF, không cần mesh

Đây là câu trả lời cho câu hỏi mở đầu thứ hai, và là một trong những nhầm lẫn tốn kém nhất trong lĩnh vực.

Bộ lập kế hoạch đường đi không cần biết bề mặt trông thế nào. Nó cần trả lời hai câu: *điểm này có trống không?* và *điểm này cách chướng ngại gần nhất bao xa?* Câu thứ hai đặc biệt quan trọng vì phần lớn bộ lập kế hoạch hiện đại dùng một hàm chi phí dựa trên khoảng cách tới chướng ngại, và chúng cần cả *gradient* của hàm đó.

ESDF trả lời trực tiếp cả hai. Một mesh đẹp thì không — để trả lời “cách chướng ngại bao xa” từ một mesh, bạn phải tính khoảng cách tới tam giác gần nhất, và làm điều đó hàng nghìn lần mỗi chu kỳ lập kế hoạch là quá đắt.

Bài học tổng quát: *chọn biểu diễn theo truy vấn bạn cần trả lời, không theo cái trông giống thế giới nhất*. Một bản đồ occupancy độ phân giải thô mà trả lời truy vấn trong micro giây hữu ích hơn một mesh hoàn hảo mà mỗi truy vấn tốn mili giây.

B. Vấn đề biến dạng khi loop closure

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và là vấn đề kỹ thuật khó nhất của bản đồ dày.

Khi loop closure xảy ra (B/11), các keyframe dịch chuyển — có thể vài mét. Bản đồ thưa xử lý dễ: mỗi landmark gắn với một keyframe neo, nên nó dịch theo. Bản đồ dày thì khó, và mức khó phụ thuộc vào biểu diễn.

Với TSDF, dữ liệu được *hợp nhất* vào một lưới voxel cố định trong không gian. Sau khi các keyframe dịch, lưới ấy không còn đúng — và không có cách nào “dịch” một lưới voxel đã trộn dữ liệu từ nhiều góc nhìn. Cách duy nhất đúng là *tích hợp lại từ đầu* từ các khung gốc, tức phải lưu toàn bộ ảnh độ sâu, tức tốn bộ nhớ khổng lồ. Các cách thoả hiệp: chia bản đồ thành các khối gắn với keyframe (mỗi khối dịch theo keyframe của nó, chấp nhận các đường nối không khớp), hoặc chỉ tích hợp lại vùng bị ảnh hưởng nhiều nhất.

Với surfel, mỗi surfel gắn với một keyframe nên nó dịch theo một cách tự nhiên — đây chính là lý do ElasticFusion [17] chọn surfel. Cái giá là mất bề mặt liền và khó trả lời truy vấn thể tích.

Đây là một ví dụ rõ về việc *lựa chọn biểu diễn bị chi phối bởi một yêu cầu hệ thống mà tài liệu về biểu diễn không nhắc tới*. Nếu bạn đọc một bài về TSDF mà không nghĩ tới loop closure, bạn sẽ chọn nó rồi phát hiện vấn đề sau nhiều tháng.

4.4 Biểu diễn học được: ảnh chụp tại 9/2026

CẠM BÃY

Mục này chốt tại tháng 9/2026 và là phần hết hạn nhanh nhất của cả cây. Lĩnh vực này chuyển động rất nhanh; mọi đánh giá dưới đây nên được rà lại sau sáu tháng.

Ngoài ra, tôi **không dẫn con số so sánh chất lượng** giữa các phương pháp. Lý do như ở B/07: các chỉ số (PSNR, độ chính xác bề mặt) phụ thuộc mạnh vào bộ dữ liệu và vào cách đo, và các bài báo trong lĩnh vực này thường so với baseline được cài lại chứ không phải số công bố gốc — đúng cỡ đồ ở research-rules.md mục 4. Điều tôi đưa ra là đánh giá về *cơ chế* và về *mức trưởng thành cho sản phẩm*, và cả hai đều là phán đoán của tôi chứ không phải kết quả đo.

A. NeRF-SLAM

iMAP [18] và NICE-SLAM [19] thay biểu diễn tường minh bằng một mạng: cảnh là các trọng số của mạng, và cả tư thế lẫn trọng số được tối ưu cùng lúc bằng phần dư quang trắc — về bản chất là một phương pháp direct (B/08) với một biểu diễn cấu trúc rất khác.

Cái được: biểu diễn liên tục, nội suy tự nhiên vào vùng chưa quan sát, bộ nhớ không tỉ lệ với thể tích.

Cái mất, và đây là ba điểm quyết định cho sản phẩm: *chậm* — render cần lấy mẫu nhiều điểm dọc mỗi tia; *quên* — một mạng huấn luyện tăng dần có xu hướng quên vùng cũ khi học vùng mới, và cách chữa (giữ lại các khung cũ để huấn luyện lại) làm mất lợi thế bộ nhớ; và *không có bảo đảm* — không cách nào biết một vùng trong biểu diễn là phép đo hay là ảo giác nội suy.

B. 3D Gaussian Splatting

3DGS [20] thay mạng bằng hàng triệu Gaussian 3D dị hướng có màu và độ trong, render bằng phép chiếu và trộn theo thứ tự độ sâu. Điểm quyết định: *không cần lấy mẫu dọc tia*, nên nhanh hơn NeRF nhiều bậc trong khi chất lượng hình ảnh tương đương hoặc hơn.

SplaTAM [21] và MonoGS [22] đưa nó vào SLAM: Gaussian là bản đồ, và tư thế được ước lượng bằng cách tối ưu phần dư quang trắc qua render khả vi.

Đây là hướng nóng nhất tại thời điểm chốt, và lý do dễ hiểu: nó cho hình ảnh đẹp hơn mọi biểu diễn cổ điển với cùng dữ liệu, và nó đủ nhanh để nghĩ tới thời gian thực.

Ba lý do vì sao một robot sản phẩm năm 2026 vẫn có thể nên dùng TSDF — đây là câu trả lời cho câu hỏi mở đầu thứ tư.

Một — truy vấn. 3DGS tối ưu cho việc *render*, không cho việc trả lời “điểm này có trống không”. Một đám Gaussian không phải một hàm occupancy, và trích xuất bề mặt hoặc ESDF từ nó là một bước thêm, không rẻ và không được bảo đảm. Với điều hướng, đây là điểm chặn.

Hai — bộ nhớ và phần cứng. Hàng triệu Gaussian, mỗi cái vài chục tham số, cộng với việc render cần GPU. Với một robot dịch vụ chạy trên SoC nhúng, đây là ngân sách không có.

Ba — tính dự đoán được và độ trưởng thành. Chế độ hồng của TSDF đã được hiểu rõ sau mười lăm năm; chế độ hồng của 3DGS-SLAM thì chưa. Với sản phẩm, đây là lập luận nặng ký, và nó là cùng lập luận với việc chọn ORB thay vì SuperPoint ở B/07 mục 2B.

Đánh giá thẳng thắn của tôi tại 9/2026: 3DGS là một bước tiến thật và rất có thể là tương lai của biểu diễn cảnh; nó *chưa* là lựa chọn cho một robot điều hướng chạy trên phần cứng hạn chế. Với nhóm ba — tái tạo ngoại tuyến — nó đã rất mạnh. D/22 xếp “3DGS cho localization thực dụng” làm một trong tám khoảng trống, và lý do nằm ở ba điểm trên.

4.5 Đánh đổi bị bỏ qua

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là mục mà tôi cho là có giá trị cao nhất của chương.

Bản đồ dày thường không cho tư thế chính xác hơn bản đồ thưa. Điều này phản trực giác và đáng dẫn ra lý do.

Thông tin về tư thế nằm ở đâu? Quay lại Jacobian ở A/03: một quan sát đóng góp vào ma trận thông tin tỉ lệ với *gradient* của hàm đo. Với phần dư quang trắc, pixel không có gradient đóng góp *đúng bằng không* (B/08 mục 2B). Với phần dư chiếu, một điểm không phân biệt được với lân cận của nó không cho tương ứng đáng tin.

Nói cách khác: *thông tin về tư thế tập trung ở những chỗ có cấu trúc*, và một front-end thưa tốt đã lấy gần hết những chỗ ấy. Phần dày thêm vào chủ yếu là các vùng trơn — chúng cho biết *hình dạng* bề mặt giữa các điểm đã biết, nhưng chúng không thêm ràng buộc mạnh cho tư thế.

Có ngoại lệ và đáng nêu rõ để công bằng. Trong môi trường rất nghèo đặc trưng — một hành lang trắng có gradient rất yếu nhưng khác không — direct method dày khai thác được phần thông tin ít ỏi mà front-end thưa bỏ qua, vì detector cần một cực đại rõ ràng để tồn tại. Đây là lợi thế thật của direct, đã nêu ở B/08 mục 2A. Nhưng đó là lợi thế của *phần dư dày*, không phải của *bản đồ dày* — hai thứ khác nhau: ta có thể dùng phần dư dày để ước lượng tư thế mà vẫn chỉ lưu một bản đồ thưa.

Kết luận thiết kế, và nó gọn: *xây bản đồ dày vì bạn cần nó cho một truy vấn cụ thể — điều hướng, tương tác, hiển thị — không phải vì bạn kỳ vọng nó cải thiện định vị.* Nếu mục tiêu là định vị chính xác hơn, hãy đầu tư vào front-end (B/07), vào hiệu chuẩn (C/14), hoặc vào quản lý bản đồ (C/13) — ba chỗ đó cho lợi ích lớn hơn nhiều trên mỗi đơn vị công sức.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
OctoMap [23]	Octree occupancy có xác suất; truy vấn nhanh	Lựa chọn mặc định cho điều hướng; mục 3A
KinectFusion [24]	TSDF thời gian thực trên GPU	Công trình khai sinh dòng TSDF; đọc để hiểu cơ chế hợp nhất
ElasticFusion [17]	Surfel để bản đồ biến dạng được khi loop closure	Mục 3B — lựa chọn biểu diễn bị chi phối bởi yêu cầu hệ thống
NICE-SLAM [19]	NeRF phân cấp cho SLAM	Mục 4A; chú ý vấn đề quên và tốc độ
SplaTAM [21], MonoGS [22]	3DGS-SLAM với render khả vi	Mục 4B; ảnh chụp tại 9/2026, rà lại sau sáu tháng
SGM trong OpenCV	Stereo dày cổ điển, chạy được trên CPU	Mục 2A; vẫn là baseline đáng tin cho sản phẩm

MINI PROJECT SLAM · Đo xem bản đồ dày có cải thiện định vị không — trên dữ liệu của chính bạn · 3 ngày

Mục tiêu	kiểm mệnh đề ở mục 5 thay vì tin nó, và đồng thời đo cái giá thật của bản đồ dày.
Dữ liệu	một chuỗi RGB-D có chân trị (TUM-RGBD), cộng một chuỗi tự quay trong môi trường <i>nghèo đặc trưng</i> — một hành lang trắng, một phòng trống. Chuỗi thứ hai là chỗ ngoại lệ ở mục 5 có thể xuất hiện.
Các bước	chạy ba cấu hình trên cùng dữ liệu: (a) SLAM thừa thuần (ORB-SLAM3 RGB-D), (b) SLAM thừa + dựng TSDF <i>sau</i> khi đã có tư thế, (c) một hệ dày dùng phần dư quang trắc dày để ước lượng tư thế; đo ATE, bộ nhớ, thời gian mỗi khung cho cả ba; rồi làm một loop closure và đo thời gian cùng chất lượng của việc cập nhật bản đồ dày.
Xong khi	bạn có một bảng ba cấu hình với bốn cột. Bạn phải quan sát được: (1) trên chuỗi có đặc trưng tốt, ATE của (a) và (c) gần nhau trong khi bộ nhớ chênh nhau hai bậc — xác nhận mục 5; (2) trên chuỗi nghèo đặc trưng, (c) có thể thắng rõ — xác nhận ngoại lệ; (3) chi phí cập nhật bản đồ dày sau loop closure lớn hơn bạn nghĩ, và đó là con số ít ai công bố.
Nối với	C/13-quan-ly-ban-do cho bài toán kích thước ở quy mô này; D/20-toi-uu-hieu-nang-va-he-thong cho ngân sách bộ nhớ thật trên phần cứng nhúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì thông tin về tư thế tập trung ở những chỗ *có cấu trúc*, và một front-end thừa tốt đã lấy gần hết những chỗ ấy. Theo Jacobian ở *A/03*, đóng góp vào ma trận thông tin tỉ lệ với gradient của hàm đo; pixel không có gradient đóng góp đúng bằng không. Phần dày thêm vào chủ yếu là các vùng trơn — chúng cho biết *hình dạng* bề mặt giữa các điểm đã biết nhưng không thêm ràng buộc mạnh cho tư thế. Ngoại lệ: môi trường rất nghèo đặc trưng, nơi gradient yếu nhưng khác không và detector thừa không tồn tại được. Nhưng đó là lợi thế của *phần dư dày*, không phải của *bản đồ dày* — ta có thể dùng phần dư dày mà vẫn chỉ lưu bản đồ thừa.
- A2. Nó cần một trường khoảng cách — ESDF — trả lời trực tiếp hai câu: điểm này có trống không, và cách chướng ngại gần nhất bao xa (kèm gradient, vì phần lớn bộ lập kế hoạch hiện đại dùng hàm chi phí theo khoảng cách). Một mesh đẹp không phải câu trả lời vì để tính khoảng cách tới chướng ngại từ mesh, bạn phải tìm tam giác gần nhất, và làm điều đó hàng nghìn lần mỗi chu kỳ là quá đắt. Bài học tổng quát: *chọn biểu diễn theo truy vấn bạn cần trả lời, không theo cái trông giống thế giới nhất*.
- A3. Ba vùng: (a) *không có vân* — mọi pixel trong một hàng giống hệt nhau nên không có thông tin xác định disparity; (b) *bị che khuất* — không tương ứng nào tồn tại; (c) *phản chiếu hoặc trong suốt* — tệ nhất, vì khớp *thành công* nhưng khớp với ảnh phản chiếu, cho độ sâu của một vật không có ở đó, tức hổng im lặng. Cả ba là giới hạn vật lý vì thông tin *không tồn tại trong dữ liệu* — không thuật toán nào tạo ra thông tin từ chỗ không có. Cái tốt nhất có thể làm với (a) là nội suy và *thừa nhận* rằng đó là nội suy; với (b) là đánh dấu “không biết”; với (c) thì chưa có lời giải tốt.

- A4. (1) *Truy vấn*: 3DGS tối ưu cho render, không cho việc trả lời “điểm này có trống không”; một đám Gaussian không phải hàm occupancy, và trích ESDF từ nó là một bước thêm không rẻ và không bảo đảm — với điều hướng đây là điểm chặn. (2) *Bộ nhớ và phần cứng*: hàng triệu Gaussian cộng yêu cầu GPU, ngoài ngân sách của một SoC nhúng. (3) *Tính dự đoán được*: chế độ hồng của TSDF đã hiểu rõ sau mười lăm năm, của 3DGS-SLAM thì chưa — cùng lập luận với việc chọn ORB thay vì SuperPoint ở B/07. Đánh giá của tôi tại 9/2026: 3DGS rất mạnh cho tái tạo ngoại tuyến, chưa phải lựa chọn cho robot điều hướng trên phần cứng hạn chế.
- A5. Bản đồ dày phải *biến dạng theo*. Với **TSDF** điều này khó vì dữ liệu đã được *hợp nhất* vào một lưới voxel cố định trong không gian, và không có cách nào “dịch” một lưới đã trộn dữ liệu từ nhiều góc nhìn — cách duy nhất đúng là tích hợp lại từ đầu, tức phải lưu toàn bộ ảnh độ sâu gốc. Các thỏa hiệp: chia thành khối gắn với keyframe (chấp nhận đường nối không khớp), hoặc chỉ tích hợp lại vùng ảnh hưởng nhiều nhất. Với **surfel** thì dễ hơn nhiều vì mỗi surfel gắn với một keyframe nên dịch theo tự nhiên — đó chính là lý do ElasticFusion chọn surfel. Đây là ví dụ rõ về việc lựa chọn biểu diễn bị chi phối bởi một yêu cầu hệ thống mà tài liệu về biểu diễn không nhắc.

NÓI LẠI THẬT ĐƠN GIẢN

Có hai loại bản đồ. Một tấm bản đồ du lịch gấp bỏ túi, ghi vài chục địa điểm để nhận. Và một cái sa bàn mô phỏng cả thành phố tới từng bậc thềm.

Bản đồ du lịch đủ để bạn biết mình ở đâu. Sa bàn thì trả lời được những câu khác: cái xe đẩy của tôi có lọt qua chỗ này không, cái kệ kia có đủ chỗ đặt vật này không.

Và đây là chỗ trực giác hay sai: *sa bàn không giúp bạn định vị tốt hơn*. Để biết mình ở đâu, bạn vẫn nhìn nhà thờ và ga tàu — những thứ dễ nhận ra. Cả một bức tường phẳng trong sa bàn không nói gì thêm về vị trí của bạn, vì mọi chỗ trên bức tường ấy trông như nhau.

Nên câu hỏi đúng không phải “bản đồ nào tốt hơn” mà là “*tôi định dùng bản đồ để làm gì*”. Nếu để biết mình ở đâu, tấm gấp bỏ túi là đủ và rẻ hơn hàng trăm lần. Nếu để quyết định đi lối nào, bạn cần thứ khác — nhưng không phải sa bàn đẹp, mà là một thứ trả lời nhanh câu hỏi “chỗ này có đi được không”. Hai thứ đó không giống nhau: một cái nhìn giống thế giới, một cái trả lời nhanh câu hỏi bạn cần hỏi.

Còn một chuyện nữa, ít ai nghĩ tới. Khi bạn phát hiện mình vẽ sai và phải kéo bản đồ lại cho đúng, thì với tấm gấp bỏ túi bạn chỉ cần dịch vài chục ký hiệu. Với sa bàn, bạn phải đập đi làm lại cả một khu — vì các mảnh đã dính chặt vào nhau và không dịch riêng được. Đó là lý do bản đồ chi tiết khó sống chung với một hệ thống hay phải sửa lại chính mình.

Chỗ không rút gọn được: chỗ không rút gọn được là các bề mặt phản chiếu. Một tấm gương hay một cửa kính cho hệ thống một phép đo *trông hoàn toàn hợp lệ* về một vật không tồn tại ở đó. Khác với vùng trống trơn — nơi hệ thống biết mình không biết — ở đây nó tự tin và sai. Chưa có lời giải tốt, và đó là lý do robot vẫn đâm vào cửa kính.

Một câu: *Xây bản đồ dày vì bạn cần trả lời một câu hỏi cụ thể, không vì bạn kỳ vọng nó giúp bạn biết mình đang ở đâu.*

4.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *bản đồ dày không cải thiện độ chính xác tư thế* — suy từ cấu trúc Jacobian (cửa a) và kiểm được bằng mini project. Nó bị bác bỏ nếu một hệ dày vượt hệ thưa một cách nhất quán trên các chuỗi *có đặc trưng tốt* — tức ngoài ngoại lệ đã nêu. Tôi cho là không, nhưng cần nói rõ một giới hạn của lập luận: nó giả định front-end thưa đã *tốt*. Với một front-end thưa kém, bổ sung thông tin dày đúng là có ích, và lúc đó kết luận đúng là “*hãy sửa front-end*” chứ không phải “*hãy dùng dày*”.

Mệnh đề thứ hai — *ba vùng thất bại của stereo là giới hạn vật lý* — đúng theo định nghĩa cho hai vùng đầu (thông tin không tồn tại). Với vùng thứ ba (phản chiếu) thì *không hoàn toàn*: thông tin

có tồn tại — một hệ có thể nhận ra rằng đây là gương nếu nó suy luận về tính nhất quán của hình học phản xạ qua nhiều góc nhìn. Nên phát biểu chặt hơn: *với hình học hai khung thuần túy thì đó là giới hạn; với suy luận cấp cao hơn thì có thể phát hiện*. Đây là một hướng nghiên cứu chưa được khai thác nhiều.

Mệnh đề thứ ba — đánh giá về 3DGS tại 9/2026 — là một phán đoán về mức trưởng thành, và nó là phần dễ hết hạn nhất của cả cây. Nó bị bác bỏ bởi thời gian: nếu trong hai năm nữa có một hệ 3DGS chạy điều hướng trên SoC nhúng với ESDF trích xuất được tin cậy, đánh giá này sai. Tôi cho rằng khả năng đó không nhỏ, và đó là lý do khối *Cạm bẫy* ở mục 4 yêu cầu rà lại sau sáu tháng.

4.7 Kết nối

Nối lên trên. *A/01-hình-hoc-da-thi* cho triangulation và hình học stereo. *A/06-bat-dinh-va-lan-truyen-sai-so* cho quan hệ $\sigma_z \approx z^2 \sigma_d / (bf)$, thứ chỉ phối chất lượng của *mọi* bản đồ dày và giải thích vì sao bản đồ dày ở xa là bản đồ dày sai. *B/08-mo-hình-phan-du* cho phần dư quang trắc và cho lời cảnh báo về prior độ sâu học được, áp dụng nguyên vẹn ở mục 2B.

Nối ngang. *C/13-quan-ly-ban-do* là chương mà bài toán kích thước được đặt ra; ở đây nó gay gắt hơn hai tới ba bậc độ lớn, và vấn đề biến dạng khi loop closure ở mục 3B là một bài toán mà chương 13 không gặp. *C/17-hoc-may-trong-slam* bàn kỹ hơn về các mô hình sinh ra độ sâu và về việc hiệu chuẩn bất định của chúng.

Nối xuống dưới. *D/20-toi-uu-hieu-nang-va-he-thong* là chỗ ngân sách bộ nhớ và tính toán trở thành ràng buộc cứng — và bản đồ dày là thứ tiêu tốn ngân sách ấy nhiều nhất. *D/22-huong-nghien-cuu-con-trong-xếp* “biểu diễn neural cho localization thực dụng” làm một trong tám khoảng trống, với lý do nằm trọn trong mục 4B.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao bản đồ dày thường không cho tư thế chính xác hơn? Trả lời bằng ngôn ngữ gradient và ma trận thông tin, rồi nêu ngoại lệ.
2. Bộ lập kế hoạch cần biểu diễn nào, và vì sao mesh không phải câu trả lời dù nó mô tả thể giới chính xác hơn?
3. Nêu ba vùng mà stereo thất bại, và nói vùng nào hỏng im lặng — tức nguy hiểm nhất.
4. Vì sao TSDF khó biến dạng sau loop closure còn surfel thì dễ? Nêu hai cách thoả hiệp cho TSDF.
5. Nêu ba lý do một robot sản phẩm có thể vẫn chọn TSDF thay vì 3DGS năm 2026, và nói lý do nào là điểm chặn cứng.
6. Một bản đồ dày trộn giữa phép đo và độ sâu học được. Vì sao đó là nguy hiểm cho ứng dụng an toàn, và cách xử lý đúng là gì?
7. Nếu mục tiêu là định vị chính xác hơn, nêu ba chỗ nên đầu tư thay vì xây bản đồ dày.

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.

Học máy trong SLAM (Learning in SLAM)

PHỤ THUỘC A/06-bat-dinh-va-lan-truyen-sai-so (bất định — chương này lập luận rằng đây là chỗ học máy đóng góp được nhiều nhất và hiện đang đóng góp ít nhất); B/07-dac-trung-va-data-association (front-end học được); B/08-mo-hinh-phan-du (prior học được và ba điều kiện của nó); B/12-relocalization (trường hợp APR — một bài học phương pháp).

TÓM TẮT

Học máy đã thay đổi phần lớn thị giác máy tính, và câu hỏi tự nhiên là nó thay đổi SLAM tới đâu. Chương này lập luận rằng câu trả lời không đồng đều: có những chỗ học máy *đã* thắng rõ ràng và nên dùng ngay, có những chỗ nó chưa thắng và lý do là cấu trúc chứ không phải tạm thời, và có một chỗ nó *ên* đóng góp nhiều mà hiện đang đóng góp ít. Nội dung đi theo ba nhóm ấy. Nhóm một — front-end, prior độ sâu, place recognition — là nơi học máy thay một module có giao diện hẹp trong khi giữ nguyên back-end hình học, và đó là lý do nó thắng. Nhóm hai — SLAM đầu-cuối — là nơi nó chưa thắng, và mục 3 phân tích vì sao lý do là cấu trúc. Nhóm ba là bất định: đây là chỗ mà một mô hình học được *có thể* đóng góp nhiều nhất — học ra hiệp phương sai từ dữ liệu thay vì đoán — và là chỗ hiện có ít công trình nhất. Nếu chỉ nhớ một điều: *front-end học được cộng back-end hình học là hướng đúng*, và lý do không phải là thoả hiệp mà là phân công theo thế mạnh.

KIẾN THỨC CẦN CÓ TRƯỚC

- **front-end học được** — thay thế detector, descriptor hoặc matcher bằng một mạng học từ dữ liệu, giữ nguyên phần ước lượng hình học phía sau.
- **SLAM đầu-cuối (end-to-end)** — học một mạng ánh xạ trực tiếp từ chuỗi ảnh sang quỹ đạo, không có bước hình học tường minh nào.
- **BA khả vi (differentiable bundle adjustment)** — cài đặt bundle adjustment sao cho gradient chảy được qua nó, cho phép huấn luyện các thành phần phía trước bằng lan truyền ngược qua bộ giải.
- **foundation model 3D** — mô hình lớn huấn luyện trên dữ liệu đa dạng, cho ra cấu trúc 3D hoặc tương ứng trực tiếp từ ảnh mà không cần hiệu chuẩn hay tư thế cho trước.
- **hiệu chuẩn bất định (uncertainty calibration)** — tính chất mà một mô hình dự đoán kèm mức tin cậy *đúng* — khi nó nói “90% chắc” thì nó đúng 90% số lần. Phần lớn mạng nơ-ron không có tính chất này mà không được sửa.
- **ngoài phân bố (out-of-distribution)** — dữ liệu khác với dữ liệu huấn luyện. Mọi mô hình học được đều suy giảm ở đây, và vấn đề thật là chúng thường suy giảm *một cách im lặng*.
- **semantic SLAM** — đưa thông tin loại đối tượng vào bài toán ước lượng — landmark không chỉ là một điểm mà là “một góc của cái bàn”.

CÂU HỎI MỞ ĐẦU

- Q1. Học máy thắng rõ ở front-end nhưng chưa thắng ở back-end. Nêu một lý do *cấu trúc* cho sự bất đối xứng đó — không phải “vì chưa ai làm tốt”.
- Q2. DROID-SLAM dùng mạng cho luồng quang và một bộ giải BA ở giữa vòng huấn luyện. Nêu điều mà kiến trúc này giữ lại từ thế giới hình học, và vì sao việc giữ lại đó quan trọng.

- Q3. Một mô hình độ sâu cho ước lượng ở mọi pixel. Nếu điều nguy hiểm nhất về đầu ra của nó trong ngữ cảnh SLAM, và nói vì sao nó khác với một cảm biến độ sâu kém chính xác.
- Q4. Nếu bạn được chọn *một* chỗ để áp dụng học máy vào một hệ SLAM sản phẩm đang chạy, chọn chỗ nào và vì sao?
- Q5. Mô hình của bạn đạt kết quả tốt trên mọi benchmark công khai. Nếu ba câu hỏi phải trả lời trước khi đưa nó vào sản phẩm, và nói câu nào khó nhất.

5.1 Đặt vấn đề

Trong mười năm, học máy đã thay thế các phương pháp thủ công ở gần như mọi bài toán thị giác: phân loại, phát hiện, phân đoạn, sinh ảnh. Câu hỏi tự nhiên: vì sao SLAM vẫn chủ yếu là hình học?

Câu trả lời hời hợt là “vì chưa ai làm tốt”. Câu trả lời thật thú vị hơn, và nó nằm ở *bản chất của bài toán*.

Các bài toán mà học máy thắng có chung một đặc điểm: chúng là bài toán *nhận thức* — ánh xạ từ dữ liệu cảm giác sang một nhãn hoặc một mô tả, nơi quan hệ đúng là phức tạp, khó viết ra, và chỉ học được từ ví dụ. “Con mèo trông thế nào” không có công thức.

SLAM thì khác. Phần lõi của nó — cho các tương ứng, tìm tư thế và cấu trúc — là một bài toán *ước lượng* có một mô hình toán *đúng* và *đã biết*. Phép chiếu phối cảnh không phải thứ cần học từ dữ liệu; nó là hình học, đã đúng từ thế kỷ mười lăm. Một mạng học lại nó từ ví dụ sẽ học được một xấp xỉ tệ hơn, và nó chỉ đúng trong phạm vi dữ liệu huấn luyện — đúng như trường hợp APR ở B/12 mục 3B.

Nhưng SLAM *không chỉ* là bài toán ước lượng. Nó có một phần nhận thức rất lớn, và đó chính là nơi mọi khó khăn thực tế nằm: pixel nào tương ứng pixel nào, ảnh này có phải chỗ tôi đã đi qua không, vật kia có đang chuyển động không, độ sâu ở vùng trơn này khoảng bao nhiêu. Không câu nào trong số đó có công thức.

Từ đó ra một cách phân chia rõ ràng, và nó là luận điểm của chương: *học máy nên làm phần nhận thức, hình học nên làm phần ước lượng*. Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và nó là một lý do cấu trúc chứ không phải một quan sát tạm thời.

CẠM BẤY

Chương này chốt tại tháng 9/2026 và cùng với C/16 là phần hết hạn nhanh nhất của cây. Mọi đánh giá về mức trưởng thành dưới đây là *phán đoán của tôi*, không phải kết quả đo, và nên được rà lại sau sáu tháng.

Tôi cũng **không dẫn con số benchmark** cho bất kỳ so sánh nào trong chương. Lý do đã nêu ở B/07: các con số này phụ thuộc mạnh vào bộ dữ liệu và cách đo, và trong lĩnh vực học máy còn thêm một vấn đề — các baseline hình học thường được cài lại bởi nhóm đề xuất phương pháp học được, và theo *research-rules.md* mục 4 thì đó là một cờ đỏ cần cảnh giác.

Điều tôi đưa ra là phân tích về *cơ chế*: mỗi cách tiếp cận giữ lại gì từ hình học, bỏ đi gì, và điều đó có hệ quả gì. Phân tích ấy bền hơn con số.

TRỰC GIÁC

Hình dung việc đo một căn phòng bằng thước dây, và bạn có một người trợ lý. Có những việc người trợ lý làm tốt hơn bạn: *nhận ra* cái góc tường ở ảnh này chính là cái góc tường ở ảnh kia, dù ánh sáng khác và góc nhìn khác. Đó là việc nhận thức — nó đòi kinh nghiệm, và không có quy tắc nào viết ra được. Và có những việc bạn *không* nên nhờ trợ lý: cộng các số đo lại. Phép cộng có quy tắc đúng, đã biết, và không có lý do gì để đoán nó từ kinh nghiệm. Một người trợ lý giỏi “ước lượng tổng” vẫn thua một cái máy tính bỏ túi. Đó là toàn bộ sự phân công trong chương này. Học máy là người trợ lý có kinh nghiệm: giao cho nó việc nhận ra, việc phán đoán, việc điền vào chỗ trống. Hình học là cái máy tính: giao cho nó việc tính. Và có một chuyện thứ ba, quan trọng hơn cả hai. Người trợ lý tốt không chỉ nói “cái góc này khớp với cái góc kia” — anh ta nói thêm “tôi khá chắc” hoặc “cái này tôi đoán thôi”. Câu thứ hai đáng giá hơn nhiều so với vẻ ngoài, vì nó cho bạn biết nên tin điều gì. Phần lớn các mô hình học được hiện nay là những người trợ lý *không bao giờ nói câu thứ hai*. Họ trả lời mọi câu hỏi với cùng một giọng tự tin, kể cả những câu họ chưa từng gặp. Và đó là vấn đề lớn nhất khi đưa họ vào một hệ thống phải tự biết mình sai.

5.2 Nơi học máy đã thắng**A. Front-end**

Đây là chỗ rõ ràng nhất, và là câu trả lời cho câu hỏi mở đầu thứ tư.

SuperPoint [25], SuperGlue [26], LightGlue [27], LoFTR [28] — tất cả thay một module với *giao diện hẹp*: vào là hai ảnh, ra là một tập tương ứng. Mọi thứ phía sau không đổi.

Ba lý do vì sao đây là chỗ thắng, và chúng giải thích cả vì sao nó dễ áp dụng.

Một — đây thật sự là bài toán nhận thức. “Hai mảng ảnh này có phải cùng một điểm vật lý không” không có công thức; nó phụ thuộc vào việc bề mặt trông thế nào từ các góc khác nhau dưới các ánh sáng khác nhau, và đó là kiến thức thống kê về thế giới.

Hai — *giao diện hẹp và kiểm được*. Đầu ra là một tập tương ứng, và mọi tương ứng đều đi qua RANSAC và kiểm định hình học phía sau (B/07 mục 5). Nghĩa là một mô hình sai không phá hệ thống ngay — nó chỉ làm tỉ lệ inlier giảm, và tăng hình học bắt được.

Ba — *thay thế được từng phần*. Đổi detector mà giữ nguyên matcher, hoặc ngược lại. Rủi ro triển khai thấp.

Cái giá đã nêu ở B/07 mục 2B và vẫn đúng: cần GPU, khó dự đoán khi ra ngoài phân bố, và mất tính tất định. Ba điều đó là lý do sản phẩm còn do dự chứ không phải vì chất lượng.

B. Prior độ sâu và place recognition

Hai chỗ thắng nữa với cùng cấu trúc lập luận.

Độ sâu học được (B/08 mục 4C, C/16 mục 2B) giải quyết được vùng không có vân, nơi hình học không có thông tin. Điều kiện dùng an toàn đã nêu: khởi tạo chứ không ràng buộc, và hiệp phương sai phải phản ánh sự thật.

Place recognition học được (B/11 mục 2C) bền hơn nhiều với thay đổi vẻ ngoài — mùa, ngày đêm. Đây rõ ràng là bài toán nhận thức, và các descriptor thủ công không có cách nào bắt được “cùng một chỗ, mùa khác”.

Điểm chung của cả ba chỗ thắng, và đáng nêu thành nguyên tắc: *học máy thắng ở nơi bài toán không có mô hình đúng, và nơi đầu ra được kiểm bởi một tầng hình học phía sau*. Hai điều kiện ấy đi cùng nhau.

5.3 Nơi nó chưa thắng, và vì sao

A. SLAM đầu-cuối

Ý tưởng: học một mạng ánh xạ từ chuỗi ảnh sang quỹ đạo, bỏ hết hình học.

Lập luận vì sao khó, và nó là lý do cấu trúc chứ không phải kỹ thuật.

Một — *nó phải học lại thứ đã biết*. Phép chiếu phối cảnh, hợp thành tư thế, ràng buộc epipolar — tất cả đều là toán đúng. Một mạng học chúng từ dữ liệu sẽ học một xấp xỉ, và xấp xỉ ấy chỉ đúng trong phạm vi dữ liệu huấn luyện. Trường hợp APR ở B/12 mục 3B là minh họa rõ nhất: mô hình nội suy giữa các tư thế huấn luyện thay vì học hình học.

Hai — *không có gauge*. SLAM có bảy bậc tự do gauge (A/05); một mạng đầu-cuối phải *chọn* một quy ước, và nó chọn quy ước của tập huấn luyện. Điều đó nghĩa là nó không tổng quát hoá sang môi trường có thang độ khác.

Ba — *không có hiệp phương sai có nghĩa*. Đầu ra là một tư thế, không phải một phân bố. Với A/06 là một chương của cây, đây là một thiếu sót nghiêm trọng cho ứng dụng.

Bốn — *không mở rộng được về thời gian*. Một mạng không có cơ chế nào tương đương với loop closure: nó không giữ một bản đồ có thể được sửa lại khi có thông tin mới.

B. Kiến trúc lai: DROID-SLAM

DROID-SLAM [29] là ví dụ tốt nhất về cách làm đúng, và nó là câu trả lời cho câu hỏi mở đầu thứ hai.

Kiến trúc: một mạng lập dự đoán luồng quang dày giữa các khung, và một lớp *bundle adjustment* lấy các luồng ấy làm phép đo rồi giải cho tư thế và độ sâu. Lớp BA nằm *trong* vòng lặp, và nó khả vi nên gradient chảy ngược qua nó để huấn luyện mạng.

Điều kiện trúc này giữ lại từ thế giới hình học — và đây là phần quan trọng: *ràng buộc hình học vẫn được áp đặt cứng*. Mạng không được phép đề xuất một quỹ đạo vi phạm phép chiếu; nó chỉ đề xuất *phép đo*, và hình học quyết định quỹ đạo. Nghĩa là mọi tính chất đúng đắn của BA — tính nhất quán hình học, khả năng dùng hiệp phương sai, khả năng thêm ràng buộc từ cảm biến khác — đều được giữ.

Vì sao việc giữ lại đó quan trọng: nó là thứ cho phép hệ thống *tổng quát hoá* ra ngoài dữ liệu huấn luyện theo hướng hình học. Mạng có thể chưa từng thấy một chuyển động như thế này, nhưng nếu nó cho luồng quang đúng thì BA cho tư thế đúng — vì BA không cần học.

Đây là lập luận chính của chương, phát biểu ở dạng cụ thể: *đặt học máy ở chỗ sinh ra phép đo, đặt hình học ở chỗ biến phép đo thành trạng thái*.

C. Foundation model 3D

DUST3R [30], MAST3R [31], VGGT [32] đi theo một hướng khác và đáng chú ý: cho hai hoặc nhiều ảnh bất kỳ, trực tiếp cho ra cấu trúc 3D và tương ứng, *không cần* hiệu chuẩn hay tư thế cho trước.

Đây là một thay đổi thật về cách nghĩ, vì nó xoá bỏ giả định mà cả A/01 và A/02 dựa vào — rằng ta biết tham số nội. Với ảnh thu thập từ nguồn không rõ, đó là một lợi ích lớn.

Đánh giá của tôi tại 9/2026: rất hứa hẹn cho SfM và cho tái tạo từ ảnh không hiệu chuẩn; *chưa* là lựa chọn cho SLAM thời gian thực trên thiết bị, vì chi phí tính toán và vì thiếu cơ chế cho hoạt động dài hạn (bản đồ, loop closure, quản lý). Cách dùng thực dụng nhất hiện nay có lẽ là làm *bộ khởi tạo* và làm công cụ dựng bản đồ ngoại tuyến, rồi một hệ hình học nhẹ chạy localization trên bản đồ đó. (*Đây là phán đoán của tôi, chưa đối chiếu với một khảo sát có hệ thống.*)

5.4 Nơi nó nên đóng góp mà chưa

Đây là mục mà tôi cho là có giá trị cao nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ ba.

A. Bất định là chỗ trống

A/06 lập luận rằng hiệp phương sai đáng tin là thứ SLAM thiếu một cách hệ thống, và rằng bốn nguồn của bệnh quá tự tin đều khó chữa bằng phân tích.

Đây chính là loại bài toán mà học máy *nên* giải: ước lượng một phân bố từ dữ liệu, khi ta không viết được mô hình giải tích cho nó. “Khớp điểm này đáng tin tới đâu, cho loại cảnh này, dưới ánh sáng này” không có công thức — nhưng có rất nhiều dữ liệu.

Vậy mà số công trình về bất định học được trong SLAM ít hơn nhiều so với số công trình về front-end học được. (*Đây là ấn tượng của tôi từ việc đọc rải rác, không phải kết quả của một khảo sát có hệ thống — và làm khảo sát ấy là một mục trong 99-chưa-biết.md.*)

Tôi cho rằng có hai lý do. *Một*: khó đo. Để biết một ước lượng bất định có đúng không, cần chân trị và cần thống kê — tức cần NEES (A/06 mục 4A), và đó là một phép đo mà cộng đồng ít dùng. Không có chỉ số dễ đo thì không có tiến bộ dễ công bố. *Hai*: ít hấp dẫn. Một bài báo cải thiện ATE dễ đăng hơn một bài báo cải thiện tính trung thực của hiệp phương sai, dù cái thứ hai có giá trị ứng dụng cao hơn.

B. Vấn đề hiệu chuẩn bất định

Có một khó khăn thật, không chỉ là thiếu quan tâm, và nó đáng nêu rõ.

Mạng nơ-ron nổi tiếng là *không được hiệu chuẩn* về mặt xác suất: khi một mô hình phân loại nói “99% chắc”, nó thường đúng ít hơn 99% số lần. Với SLAM, hệ quả nghiêm trọng hơn so với với phân loại, vì hiệp phương sai không chỉ được *đọc* mà được *dùng làm trọng số* — một hiệp phương sai quá nhỏ kéo cả nghiệm đi (B/08 khối *Cạm bẫy*).

Và vấn đề gay gắt nhất: bất định phải đúng ngoài *phân bố huấn luyện*, vì đó chính là lúc ta cần nó nhất. Một mô hình biết mình không chắc khi gặp cảnh lạ là đúng thứ ta muốn — và đó cũng là thứ khó nhất để huấn luyện, vì theo định nghĩa ta không có dữ liệu huấn luyện cho “cảnh lạ”.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba: điều nguy hiểm nhất về đầu ra của một mô hình độ sâu không phải nó sai, mà là nó *sai với vẻ chắc chắn như nhau ở mọi pixel*. Một cảm biến độ sâu kém chính xác vẫn tốt hơn, vì sai số của nó có cấu trúc đã biết — nó tệ ở xa, tệ trên bề mặt bóng, và ta mô hình hoá được điều đó. Mô hình học được không cho ta cấu trúc ấy.

C. Semantic SLAM: một hướng khác

Đưa thông tin loại đối tượng vào bài toán: landmark không chỉ là một điểm mà là “một góc của cái bàn”. CubeSLAM [33] và QuadricSLAM [34] biểu diễn đối tượng bằng các hình đơn giản và đưa chúng vào đồ thị như landmark cấp cao.

Ba lợi ích tiềm năng. *Data association tốt hơn*: hai điểm thuộc hai đối tượng khác loại chắc chắn không phải cùng một điểm — một ràng buộc mạnh và miễn phí. *Xử lý vật động*: biết cái gì là xe thì biết cái gì có thể di chuyển (D/19). *Bản đồ ổn định hơn*: các đối tượng cố định (tường, cột) đáng tin hơn các đối tượng di động (ghế), và bản đồ nên phản ánh điều đó — một hướng tấn công thật vào bài toán lifelong ở C/13 mục 4C.

Đánh giá tại 9/2026: lợi ích thứ ba là lợi ích lớn nhất và ít được khai thác nhất. Phần lớn công trình semantic SLAM tập trung vào việc dựng bản đồ có nhãn, trong khi giá trị thực dụng cao hơn nằm ở việc dùng nhãn để *quyết định tin cái gì*. (Phán đoán của tôi.)

CẦN NHỚ

Học máy làm phần nhận thức, hình học làm phần ước lượng — và ranh giới giữa hai phần là chỗ *phép đo* được sinh ra.

Cụ thể: đặt mô hình học được ở chỗ trả lời các câu không có công thức — pixel nào khớp pixel nào, ảnh này có phải chỗ cũ không, độ sâu ở vùng trơn này khoảng bao nhiêu, điểm này có nằm trên vật động không. Đặt hình học ở chỗ biến các câu trả lời ấy thành trạng thái.

Ba lý do vì sao ranh giới này đúng, không phải một thoả hiệp. *Một*: phần ước lượng có mô hình đúng và đã biết, nên học nó từ dữ liệu chỉ cho một xấp xỉ tệ hơn. *Hai*: giữ hình học nghĩa là giữ được hiệp phương sai, gauge, khả năng thêm cảm biến, và loop closure — bốn thứ mà một mạng đầu-cuối mất hết. *Ba*: đầu ra của mô hình học được đi qua một tầng kiểm định hình học, nên một mô hình sai làm giảm chất lượng chứ không phá hệ thống. Và một điều kiện bắt buộc mà hiện ít được đáp ứng: **mô hình phải nói được nó chắc tới đâu**, và con số ấy phải được hiệu chuẩn trên dữ liệu thật. Không có nó, phép đo do mô hình sinh ra không đưa vào đồ thị một cách hợp lệ được — xem B/08 mục 4C.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
SuperPoint + LightGlue [27]	Front-end học được, giao diện hẹp	Mục 2A — chỗ dễ áp dụng nhất và lợi ích rõ nhất
DROID-SLAM [29]	Luồng học được + lớp BA khả vi trong vòng lặp	Mục 3B — kiến trúc lai làm đúng; đọc để thấy ranh giới đặt ở đâu
DPVO [35]	Phiên bản nhẹ hơn nhiều của cùng ý tưởng	Cho thấy chi phí có thể giảm mạnh mà giữ kiến trúc
MASt3R [31], VGGT [32]	Foundation model 3D không cần hiệu chuẩn	Mục 3C — thay đổi cách nghĩ về SfM; chốt 9/2026
Theseus [36]	Thư viện tối ưu phi tuyến khả vi	Công cụ để xây kiến trúc lai; hạ tầng cho mục 3B
DynaSLAM [37]	Phân đoạn ngữ nghĩa để loại vật động	Ứng dụng thực dụng nhất của semantic; xem D/19

MINI PROJECT SLAM · Thay front-end bằng bản học được, rồi đo cái mà benchmark không đo · 3 ngày

Mục tiêu	đo lợi ích thật của front-end học được trên dữ liệu của bạn, và đo cái giá mà bảng so sánh không nhắc — hành vi ngoài phân bố và tính tất định.
Dữ liệu	một chuỗi chuẩn (EuRoC), cộng <i>ba</i> chuỗi tự thu cổ ý ngoài phân bố: một chuỗi ánh sáng rất kém, một chuỗi trong môi trường khác hẳn dữ liệu huấn luyện phổ biến (nhà kho công nghiệp, ngoài trời ban đêm), và một chuỗi có chuyển động rất nhanh.
Các bước	chạy cùng một hệ SLAM với hai front-end — ORB và SuperPoint+LightGlue — trên cả bốn chuỗi; đo ATE, tỉ lệ tracking, thời gian mỗi khung; rồi chạy <i>mỗi cấu hình hai lần trên cùng dữ liệu</i> và kiểm xem kết quả có bit-exact không.
Xong khi	bạn có một bảng hai front-end nhân bốn chuỗi với bốn cột. Ba điều cần quan sát: (1) trên chuỗi chuẩn, học được thắng hoặc hoà — đúng như mong đợi; (2) trên các chuỗi ngoài phân bố, xem nó thắng ở đâu và <i>thua ở đâu</i> — đây là phần benchmark không cho bạn; (3) kiểm tính tất định — nếu hai lần chạy cho kết quả khác nhau, bạn vừa phát hiện một rào cản triển khai thật mà không bài báo nào nhắc tới (D/20).
Nối với	D/19-ben-vung-trong-the-gioi-thuc cho danh mục điều kiện khó đầy đủ hơn; D/21-tu-prototype-len-mass-production bàn về chi phí cập nhật mô hình và tái kiểm định.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Lý do cấu trúc: phần lõi của SLAM là bài toán *ước lượng* có một mô hình toán *đúng và đã biết* — phép chiếu phối cảnh không phải thứ cần học từ dữ liệu. Một mạng học lại nó sẽ học một xấp xỉ chỉ đúng trong phạm vi dữ liệu huấn luyện, như trường hợp APR ở B/12. Front-end thì ngược lại: “hai mảng ảnh này có phải cùng một điểm không” là bài toán *nhận thức*, không có công thức, phụ thuộc vào kiến thức thống kê về việc bề mặt trông thế nào từ các góc và ánh sáng khác nhau. Học máy thắng ở nơi không có mô hình đúng, và thua ở nơi có.
- A2. Nó giữ lại việc *áp đặt cứng các ràng buộc hình học*: mạng chỉ được đề xuất *phép đo* (luồng quang), còn lớp BA quyết định quỹ đạo. Mạng không được phép đề xuất một quỹ đạo vi phạm phép chiếu. Việc giữ lại đó quan trọng vì nó cho phép hệ tổng quát hoá theo hướng hình học ra ngoài dữ liệu huấn luyện — mạng có thể chưa từng thấy chuyển động này, nhưng nếu nó cho luồng đúng thì BA cho tư thế đúng, vì BA không cần học. Nó cũng giữ được hiệp phương sai, gauge, khả năng thêm cảm biến và loop closure — bốn thứ mà mạng đầu-cuối mất hết.
- A3. Điều nguy hiểm nhất là nó *sai với vẻ chắc chắn như nhau ở mọi pixel* — kể cả ở những pixel mà hình học không nói gì và mô hình chỉ đang nội suy từ tiên nghiệm. Nó khác với một cảm biến độ sâu kém chính xác ở chỗ sai số của cảm biến có *cấu trúc đã biết*: nó tệ ở xa theo z^2 , tệ trên bề mặt bóng, và ta mô hình hoá được điều đó rồi đặt trọng số tương ứng. Mô hình học được không cho ta cấu trúc ấy, nên ta không biết đặt trọng số thế nào — và với ứng dụng an toàn, một bản đồ trộn giữa phép đo và phỏng đoán mà không phân biệt được hai phần là nguy hiểm.
- A4. *Front-end* — cụ thể là matcher. Ba lý do: (1) đây thật sự là bài toán nhận thức nên lợi ích lớn; (2) giao diện hẹp — vào hai ảnh, ra một tập tương ứng — nên thay được mà không đụng gì khác; (3) đầu ra đi qua RANSAC và kiểm định hình học, nên một mô hình kém làm giảm tỉ lệ inlier chứ không phá hệ thống. Rủi ro triển khai thấp nhất trên mỗi đơn vị lợi ích. Điều kiện: phải có ngân sách GPU, và phải chấp nhận mất tính tất định — hai cái giá thuộc về D/20 chứ không phải về chất lượng.
- A5. Ba câu: (1) nó hành xử thế nào ngoài phân bố huấn luyện, và nó có *biết* mình đang ở ngoài không? (2) nó có *tất định* không — cùng đầu vào cho cùng đầu ra bit-exact, điều kiện để tái hiện một lỗi từ hiện trường? (3) *chi phí cập nhật là bao nhiêu* — đổi mô hình đòi tái kiểm định những gì? Câu (1) khó nhất, vì theo định nghĩa ta không có dữ liệu để kiểm “cảnh lạ”, và vì việc huấn luyện một mô hình *biết* mình không chắc khi gặp cảnh lạ là bài toán chưa giải. Điểm số tốt trên benchmark công khai không trả lời câu nào trong ba câu ấy.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn đo một căn phòng bằng thước dây và có một người trợ lý giàu kinh nghiệm.

Có việc anh ta làm giỏi hơn bạn: nhận ra cái góc tường trong tấm ảnh này chính là cái góc tường trong tấm ảnh kia, dù chụp lúc khác, sáng khác, đứng ở chỗ khác. Việc đó không có quy tắc — nó là kinh nghiệm.

Và có việc bạn không nên nhờ anh ta: cộng các con số lại. Phép cộng có quy tắc đúng, đã biết từ lâu.

Một người “ước lượng tổng” giỏi đến mấy vẫn thua cái máy tính bỏ túi.

Đó là toàn bộ cách phân công. Giao việc nhận ra cho người có kinh nghiệm; giao việc tính cho cái máy tính. Và ranh giới giữa hai việc nằm đúng ở chỗ con số được sinh ra: người trợ lý đọc ra các con số, cái máy tính xử lý chúng.

Nhưng có một chuyện thứ ba, và nó quan trọng hơn cả hai chuyện trên.

Một người trợ lý tốt không chỉ nói “góc này khớp góc kia”. Anh ta nói thêm: “cái này tôi chắc” hoặc “cái này tôi đoán thôi, ánh sáng tối quá”. Câu thứ hai đáng giá hơn nhiều so với vẻ ngoài, vì nó cho bạn biết nên tin đến đâu — và nên đi kiểm lại chỗ nào.

Phần lớn các mô hình học máy hiện nay là những người trợ lý *không bao giờ nói câu thứ hai*. Họ trả lời mọi câu với cùng một giọng chắc chắn, kể cả những câu họ chưa từng gặp bao giờ. Bạn đưa cho họ một tấm ảnh chụp trong một nhà kho mà họ chưa từng thấy loại nào tương tự, và họ vẫn trả lời rành rọt.

Đó là vấn đề lớn nhất khi đưa họ vào một cái máy phải tự biết mình sai. Và điều trở trêu là: dạy máy *nói không chắc* đúng chỗ lại là thứ mà máy học lẽ ra nên giỏi — ước lượng mức tin cậy từ dữ liệu là đúng loại việc của nó. Chỉ là chưa mấy ai làm.

Chỗ không rút gọn được: chỗ không rút gọn được là việc dạy một mô hình biết mình đang gặp thứ chưa từng thấy. Theo định nghĩa, bạn không có dữ liệu huấn luyện cho “những thứ chưa từng thấy”. Mọi cách tiếp cận hiện có đều là cách vòng quanh vấn đề ấy, và không cách nào thoả đáng.

Một câu: *Giao việc nhận ra cho cái học được, giao việc tính cho cái đã biết — và đòi cái học được phải nói rõ khi nó chỉ đang đoán.*

5.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *học máy nên làm nhận thức, hình học nên làm ước lượng* — bị bác bỏ nếu một hệ đầu-cuối thuần túy vượt một hệ lai trên các chuỗi *ngoài phân bố huấn luyện*, với cùng ngân sách tính toán. Điều kiện “ngoài phân bố” là chỗ phải kiểm nghiệm khác, vì trong phân bố thì một mạng đủ lớn có thể thắng mà không chứng minh gì về khả năng tổng quát hoá. Tôi cho là mệnh đề vững, nhưng ranh giới *đang dịch* — các foundation model ở mục 3C hấp thụ ngày càng nhiều phần hình học vào mô hình, và tôi không chắc phát biểu này còn nguyên vẹn sau vài năm.

Mệnh đề thứ hai — *bất định học được là chỗ trống* — dựa trên ấn tượng đọc rải rác chứ không trên khảo sát, và tôi đã ghi rõ điều đó ở mục 4A. Nó bị bác bỏ đơn giản bằng cách chỉ ra một nhánh công trình đáng kể mà tôi bỏ sót. Đây là mục có rủi ro sai cao nhất trong chương, và việc làm khảo sát cho nó nằm trong 99-chua-biet.md.

Mệnh đề thứ ba — *mạng nơ-ron không được hiệu chuẩn về xác suất* — là một kết quả đã biết rộng rãi trong cộng đồng học máy, nhưng tôi *không* dẫn được một nguồn cụ thể trong ngữ cảnh SLAM, và tôi không biết mức độ nghiêm trọng của nó cho các mô hình độ sâu và luồng quang cụ thể đang dùng. Đây là một khoản nợ kiểm chứng.

Mệnh đề thứ tư — đánh giá về mức trưởng thành của foundation model tại 9/2026 — là phán đoán và sẽ hết hạn. Rà lại sau sáu tháng.

5.6 Kết nối

Nối lên trên. B/07-dac-trung-va-data-association là chỗ front-end học được được bàn ở mức chi tiết, cùng cái giá của nó. B/08-mo-hinh-phan-du mục 4C cho ba điều kiện đưa một prior học được vào đồ thị, và chúng áp dụng nguyên vẹn cho mọi nguồn học được. B/12-relocalization mục 3B cho trường hợp APR, và bài học phương pháp ở đó — so với một đường cơ sở tầm thường — nên áp dụng cho mọi khẳng định trong chương này. A/06-bat-dinh-va-lan-truyen-sai-so là chương mà mục 4 nối vào: bất định là chỗ trống, và lý do nó là chỗ trống nằm ở chỗ nó khó đo.

Nối ngang. C/16-dense-mapping-va-bieu-dien chia sẻ đặc điểm hết hạn nhanh, và hai chương nên được rà cùng nhau. C/18-danh-gia-va-phuong-phap-luan cho công cụ để đánh giá một thành phần học được một cách có kỷ luật — đặc biệt là yêu cầu về baseline và về đánh giá ở đuôi phân bố.

Nối xuống dưới. D/19-ben-vung-trong-the-gioi-thuc là chỗ semantic segmentation có ứng dụng thực dụng nhất: loại vật động. D/20-toi-uu-hieu-nang-va-he-thong bàn về chi phí tính toán và về tính tất định — hai cái giá lớn nhất của front-end học được trong sản phẩm. D/21-tu-prototype-len-mass-production bàn về chi phí cập nhật mô hình, tái kiểm định, và OTA cho mô hình — những thứ không tồn tại với một thuật toán thủ công. D/22-huong-nghien-cuu-con-trong-lay “front-end học được + back-end hình học với bất định được truyền đúng” làm hướng số năm, và lý do nằm trọn trong mục 4 của chương này.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu lý do *cấu trúc* vì sao học máy thắng ở front-end mà chưa thắng ở back-end.
2. Ba điều kiện chung của các chỗ mà học máy đã thắng là gì, và vì sao chúng đi cùng nhau?
3. DROID-SLAM giữ lại gì từ hình học, và điều đó cho hệ thống khả năng gì mà một mạng đầu-cuối không có?
4. Nêu bốn thứ mà một hệ SLAM đầu-cuối mất so với một hệ hình học.
5. Vì sao bất định là chỗ mà học máy *nên* đóng góp nhiều nhất, và nêu hai lý do vì sao hiện nó đóng góp ít?
6. Vì sao việc huấn luyện một mô hình “biết mình không chắc khi gặp cảnh lạ” là bài toán khó về mặt nguyên tắc?
7. Nêu ba câu hỏi phải trả lời trước khi đưa một mô hình học được vào sản phẩm, và nói vì sao benchmark không trả lời câu nào.

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.

Đánh giá và phương pháp luận (Evaluation and Methodology)

PHỤ THUỘC A/05-observability-gauge-va-suy-bien (số bậc tự do gauge quyết định căn chỉnh mấy bậc — căn sai là che mất lỗi); A/06-bat-dinh-va-lan-truyen-sai-so (NEES và bệnh quá tự tin); B/11-loop-closure-va-nhat-quan-toan-cuc và B/12-relocalization (đánh giá có chi phí lỗi bất đối xứng).

TÓM TẮT

Mọi chương trước nợ chương này một câu trả lời: *làm sao biết một thay đổi thật sự là cải tiến?* Câu hỏi nghe đơn giản và nó bị trả lời sai một cách có hệ thống trong cả lĩnh vực lẫn trong thực hành công nghiệp. Nội dung có bốn phần. Một: các chỉ số quỹ đạo — ATE và RPE — với một chi tiết quyết định mà nhiều người làm sai: *căn chỉnh mấy bậc tự do*, và căn sai thì che mất đúng thứ ta muốn đo. Hai: đo *tính trung thực* chứ không chỉ độ chính xác, tức NEES — một phép đo rẻ mà gần như không ai làm. Ba: đánh giá các thành phần có chi phí lỗi bất đối xứng (loop closure, relocalization), nơi một con số trung bình là vô nghĩa và đường cong precision-recall ở vùng precision cao mới là thứ đáng nhìn. Bốn: kỷ luật phương pháp — ablation, mô phỏng, nhiều seed, và nguyên tắc quan trọng nhất của cả chương: *đánh giá ở đuôi phân bố, không ở trung bình*. Nếu chỉ nhớ một điều: *một hệ tốt hơn về trung bình nhưng tệ hơn ở đuôi là một hệ tệ hơn* — vì sản phẩm hỏng ở đuôi, không hỏng ở trung bình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ATE** — *absolute trajectory error* — sai số vị trí giữa quỹ đạo ước lượng và chân trị sau khi căn chỉnh hai quỹ đạo. Đo sai số *toàn cục*, nên một loop closure tốt cải thiện nó nhiều.
- **RPE** — *relative pose error* — sai số của phép biến đổi tương đối trên các đoạn có độ dài cố định. Đo *drift cục bộ*, không phụ thuộc vào việc có đóng vòng hay không.
- **căn chỉnh (alignment)** — phép biến đổi áp lên quỹ đạo ước lượng để đưa nó về cùng hệ với chân trị, trước khi tính sai số. Số bậc tự do của phép biến đổi này *phải* bằng số bậc gauge của hệ.
- **ablation study** — thí nghiệm tắt từng thành phần một để đo đóng góp của nó. Chỉ hợp lệ khi mọi thứ khác được giữ nguyên và khi có đủ số lần chạy.
- **đuôi phân bố** — các trường hợp xấu nhất — phân vị 95, 99, hoặc trường hợp hỏng hoàn toàn. Đây là nơi hệ thống sản phẩm thất bại, và là nơi trung bình không nói gì.
- **ý nghĩa thống kê** — mức tin cậy rằng một khác biệt quan sát được không phải do ngẫu nhiên. SLAM có nguồn ngẫu nhiên thật (RANSAC, thứ tự luồng), nên một lần chạy không chứng minh gì.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn đánh giá một hệ VIO bằng cách căn chỉnh *Sim(3)* với chân trị rồi tính ATE — cách làm phổ biến. Vì sao đó là một sai lầm, và nó che mất điều gì cụ thể?
- Q2. Hệ thống mới của bạn có ATE trung bình thấp hơn hệ cũ 15% trên mười chuỗi. Nêu ba lý do vì sao điều đó vẫn chưa chứng minh nó tốt hơn.
- Q3. Đo NEES cần chân trị, nên nó chỉ làm được trong mô phỏng. Vì sao điều đó *không* phải lý do để bỏ qua nó, và nó cho biết gì mà ATE không cho biết?

- Q4.** Bạn so hệ của mình với một baseline mà bạn tự cài đặt, và bạn thắng. Nêu phép kiểm bắt buộc trước khi tin kết quả đó.
- Q5.** Hệ A có ATE trung bình 0,08 m và thất bại hoàn toàn trên 2% số lần chạy. Hệ B có ATE trung bình 0,12 m và chưa bao giờ thất bại. Hệ nào tốt hơn, và câu trả lời phụ thuộc vào gì?

6.1 Đặt vấn đề

Mười bảy chương trước đưa ra hàng chục lựa chọn thiết kế: detector nào, phần dư nào, kiến trúc back-end nào, chính sách bản đồ nào. Mỗi lựa chọn được biện minh bằng lập luận về cơ chế. Không lựa chọn nào được biện minh bằng số liệu — và đó là có chủ ý, vì chương này giải thích vì sao các số liệu phổ biến thường không biện minh được gì.

Bài toán gốc: *cho hai phiên bản của một hệ thống, làm sao biết cái nào tốt hơn?*

Nghe đơn giản. Nó khó vì bốn lý do chồng lên nhau.

Một — “tốt hơn” không phải một chiều. Một hệ có thể chính xác hơn nhưng chậm hơn, hoặc chính xác hơn ở trung bình nhưng hỏng nhiều hơn ở đuôi, hoặc chính xác hơn trong nhà và tệ hơn ngoài trời.

Hai — chân trị khó có. Hệ đo ngoài (motion capture) chỉ có trong phòng lab và chỉ trong một thể tích nhỏ. Ngoài đó, ta không biết sự thật.

Ba — SLAM có tính ngẫu nhiên thật. RANSAC lấy mẫu ngẫu nhiên; thứ tự luồng không tắt định; một khác biệt nhỏ trong thời điểm có thể đổi keyframe nào được chọn. Hai lần chạy cùng một hệ trên cùng dữ liệu cho hai kết quả khác nhau.

Bốn — và đây là lý do nghiêm trọng nhất: điều ta đo thường không phải điều ta quan tâm. Ta đo ATE trung bình trên các chuỗi mà hệ thống chạy được. Cái ta quan tâm là hệ thống có chạy được ở nhà khách hàng suốt tám tiếng hay không — và hai thứ đó tương quan yếu hơn nhiều so với vẻ ngoài.

Trước khi có các chỉ số chuẩn, mỗi bài báo tự định nghĩa cách đo, và so sánh giữa các bài là bất khả. TUM-RGBD [38] chuẩn hoá ATE và RPE; rpg_trajectory_evaluation [39] chuẩn hoá cách căn chỉnh và cách báo cáo drift theo đoạn. Đó là tiền bộ thật.

Nhưng chuẩn hoá chỉ số không giải quyết bốn lý do ở trên. Chương này lập luận rằng phần lớn giá trị của việc đánh giá nằm ở *thiết kế thí nghiệm*, không ở việc chọn chỉ số — và rằng thiết kế thí nghiệm là chỗ được đầu tư ít nhất.

TRỰC GIÁC

Hình dung bạn đánh giá hai tài xế.

Cách đơn giản: cho mỗi người lái một quãng đường và đo thời gian. Tài xế nhanh hơn thắng.

Ba chỗ sai của cách đó, và chúng ánh xạ chính xác sang ba chỗ sai của việc đánh giá SLAM.

Một — bạn đo trên một quãng đường. Tài xế A nhanh trên đường cao tốc, tài xế B nhanh trong phố. Một con số duy nhất giấu mất điều đó, và nếu bạn định thuê người lái trong phố thì bạn vừa chọn sai.

Hai — bạn đo trung bình, không đo tệ nhất. Tài xế A nhanh hơn 10% nhưng thỉnh thoảng đâm xe. Tài xế B chậm hơn nhưng chưa bao giờ. Con số trung bình nói A thắng; bất kỳ ai thuê tài xế đều biết B mới là lựa chọn.

Ba — bạn không hỏi họ có biết mình đang chạy ẩu không. Một người lái nhanh và biết lúc nào nên chậm lại thì an toàn. Một người lái nhanh và luôn nghĩ mình đang an toàn thì không. Hai người này có cùng thời gian và cùng số lần đâm xe cho tới nay — nhưng họ không giống nhau chút nào.

Chuyện thứ ba là chuyện tình tế nhất và là chuyện mà gần như không ai đo. Nó tương ứng với việc đo *tính trung thực của hiệp phương sai*: một hệ biết mình sai bao nhiêu thì dùng được, một hệ luôn nghĩ mình đúng thì không — dù hai hệ có cùng ATE.

Và một chuyện thứ tư, về sự công bằng. Nếu bạn để tài xế A lái chiếc xe quen của anh ta và bắt tài xế B lái một chiếc xe lạ, thì kết quả không nói gì về hai tài xế. Trong đánh giá SLAM, chuyện này xảy ra liên tục dưới dạng: người đề xuất phương pháp mới tự cài đặt baseline.

6.2 Chỉ số quỹ đạo, và cái bẫy căn chỉnh

A. ATE và RPE đo hai thứ khác nhau

ATE căn chỉnh toàn bộ quỹ đạo ước lượng với chân trị rồi tính sai số vị trí ở mỗi thời điểm. Nó đo *sai số toàn cục*, nên nó rất nhạy với việc có đóng vòng hay không: một loop closure tốt cải thiện ATE mạnh.

RPE lấy các đoạn có độ dài cố định — chẳng hạn mọi đoạn 10 m — và đo sai số của phép biến đổi tương đối trên mỗi đoạn. Nó đo *drift cục bộ*, và nó gần như không bị ảnh hưởng bởi loop closure.

Hai chỉ số trả lời hai câu khác nhau, và dùng nhầm là một lỗi thật. Nếu bạn đang cải thiện odometry, RPE là chỉ số đúng — ATE sẽ bị chi phối bởi việc có đóng vòng hay không, và nó che mất đóng góp của bạn. Nếu bạn đang cải thiện loop closure, ATE là chỉ số đúng — RPE gần như không đổi.

Cách báo cáo tốt nhất, theo [39]: *drift theo đoạn*, tức RPE tính trên nhiều độ dài đoạn khác nhau và biểu diễn dưới dạng phần trăm quãng đường. Nó cho một bức tranh đầy đủ hơn một con số, và nó so sánh được giữa các chuỗi có độ dài khác nhau — điều mà ATE tuyệt đối không làm được.

B. Căn chỉnh mấy bậc tự do: cái bẫy chính

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là lỗi mà tôi cho là phổ biến nhất trong đánh giá SLAM.

Quỹ đạo ước lượng sống trong một hệ quy chiếu tùy ý — đó là gauge ở A/05. Trước khi so với chân trị, phải căn chỉnh. Câu hỏi: căn chỉnh *mấy bậc tự do*?

Câu trả lời: *đúng bằng số bậc gauge của hệ*, và A/05 mục 3B cho bảng:

Mono thuần thị giác: $Sim(3)$, bảy bậc — vì tỉ lệ là gauge tự do. Stereo: $SE(3)$, sáu bậc. VIO: *bốn* bậc — ba tịnh tiến và yaw, vì trọng lực ghim roll với pitch.

Căn sai thì sao? Hai hướng sai với hai hậu quả khác nhau.

Căn quá nhiều bậc — ví dụ căn $Sim(3)$ cho một hệ VIO — *che mất lỗi*. Phép căn chỉnh sẽ tự động co giãn quỹ đạo ước lượng cho khớp với chân trị, nên một hệ có sai số tỉ lệ 10% sẽ cho ATE giống hết một hệ có tỉ lệ đúng. Bạn vừa xoá đúng thứ bạn cần đo — và với VIO, tỉ lệ là một trong những thứ quan trọng nhất cần kiểm, vì nó là thứ IMU đóng góp.

Căn quá ít bậc — ví dụ căn bốn bậc cho một hệ mono — cho ATE lớn giả tạo, vì phần gauge chưa được hấp thụ sẽ xuất hiện thành sai số.

Hệ quả thực hành: *luôn ghi rõ căn chỉnh mấy bậc khi báo cáo ATE*, và nghi ngờ mọi bảng so sánh không ghi. Và nếu bạn đang so hai hệ có cấu hình cảm biến khác nhau — mono với VIO chẳng hạn — thì ATE của chúng *không so sánh trực tiếp được*, vì chúng phải căn chỉnh với số bậc khác nhau.

CẠM BẤY

Căn chỉnh $Sim(3)$ cho một hệ có IMU là cách phổ biến nhất để tự lừa mình.

Cơ chế: phép căn chỉnh cơ giãn quỹ đạo ước lượng cho khớp chân trị, nên nó *hấp thụ* sai số tỉ lệ. Một hệ VIO có tỉ lệ sai 10% — một lỗi nghiêm trọng, thường do khởi tạo kém hoặc do kích thích không đủ (B/10) — sẽ cho ATE gần như giống hệt một hệ có tỉ lệ đúng.

Bạn sẽ thấy một con số đẹp và kết luận hệ thống tốt, trong khi thứ mà IMU được thêm vào để cung cấp — đơn vị mét — đang sai.

Ba quy tắc để tránh. *Một*: căn chỉnh đúng số bậc theo cấu hình cảm biến, và **ghi rõ trong mọi báo cáo**. *Hai*: với hệ có tỉ lệ quan sát được, hãy **báo cáo riêng sai số tỉ lệ** như một con số độc lập — nó là thứ đáng theo dõi nhất và nó biến mất khỏi ATE nếu căn sai. *Ba*: khi đọc một bảng so sánh không ghi số bậc căn chỉnh, hãy coi các con số là không diễn giải được, chứ không phải là “chắc họ làm đúng”.

6.3 Đo tính trung thực, không chỉ độ chính xác

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và là mục mà tôi cho là có tỉ lệ giá trị trên mức phổ biến cao nhất trong chương.

A. NEES và điều nó cho biết

Từ A/06 mục 4A: $\varepsilon = \tilde{\mathbf{x}}^\top \Sigma^{-1} \tilde{\mathbf{x}}$, và nếu Σ trung thực thì $\mathbb{E}[\varepsilon]$ bằng số chiều.

ATE trả lời: *bạn sai bao nhiêu*. NEES trả lời: *bạn có thành thật về việc mình sai bao nhiêu không*. Hai câu hỏi khác nhau, và một hệ có thể tốt ở câu một và tệ ở câu hai — thực tế đó là trường hợp thường gặp (A/06 mục 4).

Vì sao câu hai quan trọng với sản phẩm: mọi module phía trên dùng hiệp phương sai để ra quyết định. Bộ lập kế hoạch quyết định có đi qua khe hẹp không; loop closure quyết định có chấp nhận một ứng viên không (B/11 mục 3B); relocalization quyết định có tin kết quả không (B/12 mục 4B). Ba quyết định ấy đều sai nếu hiệp phương sai quá nhỏ.

Cái giá: cần chân trị, nên chỉ làm được trong mô phỏng hoặc trên bộ dữ liệu có hệ đo ngoài.

B. Vì sao “cần chân trị” không phải lý do bỏ qua

Lập luận thường gặp: NEES chỉ đo được trong mô phỏng, mà mô phỏng không giống thực tế, nên nó không có giá trị.

Lập luận ấy sai ở một chỗ quan trọng. NEES trong mô phỏng không đo “hệ thống của tôi tốt tới đâu trong thực tế”. Nó đo một thứ khác và cụ thể hơn: *cài đặt hiệp phương sai của tôi có đúng không, với chính mô hình mà tôi đã giả định*. Đó là một câu hỏi về *tính đúng đắn của cài đặt*, và mô phỏng là chỗ lý tưởng để trả lời nó, vì ở đó mọi giả định đều thoả chính xác.

Nói cách khác: nếu NEES lệch trong mô phỏng — nơi nhiều đúng là Gauss, trọng số đúng là σ bạn đã dùng để sinh dữ liệu — thì bạn có *lỗi cài đặt*, không phải vấn đề mô hình. Nhầm hiệp phương sai biên với điều kiện (A/06 mục 2B), quên nhân Adjoint khi đổi quy ước (A/03), sai một

hệ số ở đâu đó. Đây là những lỗi mà không phép đo nào khác bắt được, và chúng *im lặng* — hệ thống vẫn cho ATE tốt.

Đây là lập luận quan trọng nhất của mục này: *mô phỏng không dùng để chứng minh hệ thống tốt, mà để tách lỗi cài đặt khỏi lỗi mô hình*. Chạy trong mô phỏng trước là cách rẻ nhất để biết phần nào của vấn đề là do bạn viết sai và phần nào là do thế giới khó.

C. Khi không có chân trị

Tại hiện trường, bốn đại lượng thay thế đáng theo dõi, và chúng là nền cho giám sát sức khoẻ ở D/21: số inlier và tỉ lệ inlier; số điều kiện hoặc giá trị riêng nhỏ nhất của Λ ; vết của hiệp phương sai tư thế; và tỉ lệ tracking.

Không đại lượng nào trong bốn cái đó đo sai số. Chúng đo các *điều kiện* mà sai số nhỏ phụ thuộc vào. Quan hệ giữa chúng và sai số thật phải được *hiệu chuẩn một lần* trên dữ liệu có chân trị, rồi dùng ngoài hiện trường. Đó là một quy trình đáng làm và ít ai làm.

I 6.4 Đánh giá thành phần có chi phí lỗi bất đối xứng

Loop closure (B/11) và relocalization (B/12) có chung một đặc điểm: chi phí của dương tính giả lớn hơn âm tính giả nhiều bậc. Đánh giá chúng bằng một con số duy nhất là vô nghĩa.

A. Precision-recall, và vùng đáng nhìn

Cách đúng là đường cong precision-recall. Nhưng — và đây là điểm quan trọng — *điểm F1 tối đa là tiêu chí sai*, vì F1 giả định precision và recall có giá trị ngang nhau.

Với chi phí bất đối xứng, vùng đáng nhìn là vùng *precision rất cao*. Câu hỏi đúng là: ở *precision 99,9%, recall của tôi là bao nhiêu?* Một hệ có recall 60% ở precision 99,9% hữu ích hơn một hệ có recall 95% ở precision 95%, dù con số thứ hai trông đẹp hơn — vì 5% dương tính giả nghĩa là bản đồ bị phá định kỳ.

Cách báo cáo tốt: chọn một mức precision từ *yêu cầu sản phẩm* — “tôi chấp nhận một loop closure sai trên một nghìn” — rồi báo cáo recall ở mức đó. Con số ấy có nghĩa và so sánh được.

B. Đánh giá trên truy vấn không có câu trả lời

Một thiếu sót có hệ thống của các benchmark, đã nêu ở B/12 khối *Cạm bẫy*: chúng chỉ chứa các truy vấn *có* câu trả lời trong bản đồ.

Trong thực tế, phần lớn truy vấn relocalization xảy ra ở những chỗ mà hệ thống *chưa* lập bản đồ — robot bị đưa sang một phòng mới, hoặc bản đồ đã lỗi thời. Hành vi đúng là trả về “không biết”. Không benchmark nào đo điều đó.

Cách khắc phục cho bộ dữ liệu nội bộ: cố ý đưa vào các truy vấn ngoài bản đồ và đo tỉ lệ hệ thống trả lời sai thay vì nói không biết. Đây là một trong những phép đo có giá trị cao nhất cho sản phẩm và gần như không tồn tại trong tài liệu.

I 6.5 Kỷ luật phương pháp

A. Nhiều lần chạy và ý nghĩa thống kê

Đây là câu trả lời cho câu hỏi mở đầu thứ hai.

SLAM có nguồn ngẫu nhiên thật: RANSAC lấy mẫu ngẫu nhiên, thứ tự luồng không tất định, và một khác biệt nhỏ về thời điểm đổi keyframe nào được chọn. Độ tán giữa các lần chạy có thể so sánh được với khác biệt giữa hai phương pháp.

Quy tắc tối thiểu: *chạy mỗi cấu hình nhiều lần với các seed khác nhau, và báo cáo cả trung vị lẫn độ tán. Một cải thiện 15% trong khi độ tán giữa các lần chạy là 20% thì chưa chứng minh gì.*

Ba lý do vì sao 15% trên mười chuỗi vẫn chưa đủ: *một*, có thể nằm trong độ tán chạy-tới-chạy; *hai*, có thể do một hoặc hai chuỗi chi phối trong khi các chuỗi khác tệ đi — nên phải nhìn phân bố chứ không nhìn trung bình; *ba*, mười chuỗi có thể không đại diện cho môi trường triển khai.

Một cách rẻ để loại nguồn ngẫu nhiên: làm hệ thống *tất định* — seed cố định, thứ tự xử lý cố định. Điều này đáng làm vì lý do khác quan trọng hơn: nó là điều kiện để debug một lỗi từ hiện trường (D/20).

B. Baseline phải tử tế

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó là một quy tắc mà `research-rules.md` mục 7 nêu rõ.

Nếu bạn tự cài đặt baseline và bạn thắng, phép kiểm bắt buộc là: *baseline của tôi có đạt được con số mà bài gốc công bố không?* Nếu không, bạn đang so với một phiên bản bị làm yếu, và kết quả không nói gì.

Đây không phải một cảnh báo lý thuyết. Rất nhiều cải tiến biến mất khi baseline được chỉnh tử tế, và lý do có cấu trúc: người đề xuất phương pháp mới dành hàng tháng tinh chỉnh nó và vài ngày cài baseline.

Quy tắc thực hành: dùng mã nguồn gốc của baseline khi có; nếu phải tự cài, hãy tái lập được con số công bố *trước* khi so sánh; và nếu không tái lập được, hãy ghi rõ điều đó thay vì im lặng.

C. Ablation có kỷ luật

Tắt từng thành phần một, giữ nguyên mọi thứ khác, và đo. Nghe hiển nhiên, và nó hay bị làm sai theo ba cách.

Sai một — *tắt nhiều thứ cùng lúc*. Nếu tắt cả loop closure lẫn local BA, bạn không biết cái nào đóng góp gì.

Sai hai — *không chỉnh lại tham số*. Tắt một thành phần có thể làm các tham số tối ưu thay đổi. Nếu bạn tắt M-estimator mà giữ nguyên ngưỡng RANSAC, bạn đang đo một cấu hình chưa được tinh chỉnh, không phải đóng góp của M-estimator.

Sai ba — *không chạy đủ nhiều lần*. Áp dụng đúng như mục 5A.

D. Đánh giá ở đuôi, không ở trung bình

Đây là nguyên tắc quan trọng nhất của chương, và là câu trả lời cho câu hỏi mở đầu thứ năm.

Với hệ A (ATE trung bình 0,08 m, thất bại 2%) so với hệ B (ATE 0,12 m, chưa bao giờ thất bại): câu trả lời phụ thuộc vào *chi phí của một lần thất bại*.

Với một bài báo, thất bại nghĩa là một chuỗi bị loại khỏi bảng, nên hệ A thắng. Với một sản phẩm, thất bại nghĩa là robot dừng giữa nhà khách hàng và ai đó phải tới xử lý — và 2% trên hàng nghìn máy mỗi ngày là một dòng chi phí vận hành thật. Hệ B thắng, và thắng không sát sao.

Đây là chỗ mục tiêu sản phẩm ở 00-map tạo ra một khác biệt thật về phương pháp: *SOTA thật nằm ở đuôi phân bố, không ở trung bình*. Ba hệ quả cụ thể.

Một — báo cáo phân vị, không chỉ trung bình. Phân vị 95 và 99 của ATE, và của thời gian mỗi khung. B/09 mini project đã nhấn mạnh điều này cho thời gian.

Hai — đếm và phân loại thất bại. Số lần mất bám, số lần relocalization sai, số lần bản đồ bị phá. Các con số này thường không xuất hiện trong bài báo và chúng là con số quan trọng nhất cho sản phẩm.

Ba — thiết kế thí nghiệm nhắm vào đuôi. Đừng chỉ chạy trên các chuỗi mà hệ thống chạy được. Chạy trên các chuỗi khó, các chuỗi có sự cố, các chuỗi ngoài phân bố. D/21 lập luận rằng đây là lý do phải có bộ dữ liệu nội bộ phản ánh môi trường khách hàng.

CẦN NHỚ

Bốn câu hỏi mà một đánh giá đầy đủ phải trả lời, và ba câu sau thường bị bỏ.

Một — chính xác tới đâu? ATE và RPE, căn chỉnh đúng số bậc. Đây là câu duy nhất mà phần lớn bài báo trả lời.

Hai — có thành thật không? NEES trong mô phỏng để kiểm cài đặt hiệp phương sai; các đại lượng thay thế ngoài hiện trường.

Ba — hỏng thế nào và bao nhiêu lần? Tỷ lệ thất bại, phân loại nguyên nhân, và hành vi khi hỏng — nó dừng lại hay tiếp tục báo số sai?

Bốn — ở đuôi thì sao? Phân vị 95 và 99, không phải trung bình. Với sản phẩm, đây là câu quan trọng nhất.

Và một nguyên tắc bao trùm: **đo trước, cải tiến sau**. Cải tiến trước khi đo là cách nhanh nhất để tối ưu sai thứ — bạn sẽ dành ba tháng làm giảm ATE trung bình 10% trong khi vấn đề thật là 2% số lần chạy bị hỏng vì một nguyên nhân bạn chưa từng đo.

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
evo [40]	Công cụ tính ATE, RPE với các chế độ căn chỉnh khác nhau	Dễ dùng; chú ý chọn đúng chế độ căn chỉnh — mặc định không phải lúc nào cũng đúng cho hệ của bạn
rpg_trajectory_evaluation [39]	Căn chỉnh theo cấu hình cảm biến; drift theo đoạn; nhiều lần chạy	Cài đặt tham chiếu của mục 2; đọc tài liệu về lựa chọn căn chỉnh
OpenVINS [8]	Mô phỏng có chân trị; tính NEES trực tiếp	Một trong ít hệ mã nguồn mở đo tính nhất quán chứ không chỉ độ chính xác
TUM-RGBD [38]	Định nghĩa chuẩn của ATE và RPE	Nguồn của mục 2A
OpenLORIS [4]	Bộ dữ liệu nhiều phiên, môi trường thay đổi	Một trong ít bộ dữ liệu cho phép đánh giá hoạt động dài hạn

MINI PROJECT SLAM · Đánh giá lại một kết quả bạn đã tin, theo bốn câu hỏi ở khối Cần nhớ · 3 ngày

Mục tiêu	phát hiện xem một kết luận bạn đã rút ra từ các chương trước có đứng vững dưới một đánh giá có kỷ luật hay không.
Dữ liệu	chọn một mini project bạn đã làm ở các chương trước — so ba phương pháp triangulation, hay so hai front-end, hay so ba back-end — và dùng lại dữ liệu đó, cộng thêm vài chuỗi khó.

Các bước	chạy lại so sánh cũ nhưng lần này: (a) căn chỉnh đúng số bậc theo cấu hình cảm biến và ghi rõ; (b) chạy mỗi cấu hình <i>mười</i> lần với seed khác nhau và báo cáo trung vị cùng khoảng phân vị; (c) báo cáo phân vị 95 và 99 chứ không chỉ trung bình; (d) đếm và phân loại các lần thất bại; (e) nếu có mô phỏng, tính NEES.
Xong khi	bạn trả lời được: kết luận cũ của bạn có còn đúng không, và nếu có thì với mức tin cậy nào? Kết quả có giá trị nhất của bài này là khi câu trả lời là <i>không</i> — bạn vừa tránh được việc xây tiếp trên một kết luận sai. Nếu độ tán giữa mười lần chạy lớn hơn khác biệt giữa hai phương pháp, hãy ghi điều đó vào 99-chua-biet.md như một câu hỏi mở thay vì kết luận bừa.
Nối với	D/21-tu-prototype-len-mass-production biến quy trình này thành regression test tự động chạy mỗi commit; D/22-huong-nghien-cuu-con-trong dựa vào nó để phân biệt một khoảng trống thật với một khoảng trống tưởng tượng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì *Sim(3)* có bảy bậc gồm cả *tỉ lệ*, nên phép căn chỉnh sẽ co giãn quỹ đạo ước lượng cho khớp chân trị và *hấp thụ* sai số tỉ lệ. Một hệ VIO có tỉ lệ sai 10% — lỗi nghiêm trọng, thường do khởi tạo kém hoặc kích thích không đủ — sẽ cho ATE gần giống hệ có tỉ lệ đúng. Bạn vừa xoá đúng thứ cần đo, và đó là thứ mà IMU được thêm vào để cung cấp. Với VIO phải căn *bốn* bậc (ba tịnh tiến + yaw), vì trọng lực ghim roll và pitch. Và nên báo cáo sai số tỉ lệ như một con số riêng.
- A2. (1) Có thể nằm trong *độ tán chạy-tới-chạy* — SLAM có ngẫu nhiên thật từ RANSAC và thứ tự luồng, và độ tán có thể so sánh được với 15%. (2) Có thể do *một hoặc hai chuỗi chi phối* trong khi các chuỗi khác tẻ đi — phải nhìn phân bố chứ không nhìn trung bình. (3) Mười chuỗi có thể *không đại diện* cho môi trường triển khai. Và một lý do thứ tư quan trọng hơn cả ba: 15% ở *trung bình* không nói gì về *đuôi*, và sản phẩm hỏng ở đuôi. Cách kiểm: chạy nhiều seed, báo cáo trung vị và độ tán, báo cáo phân vị 95 và 99, và đếm số lần thất bại.
- A3. Vì NEES trong mô phỏng không đo “hệ tôi tốt tới đâu trong thực tế” — nó đo một thứ khác và cụ thể hơn: *cài đặt hiệp phương sai của tôi có đúng không, với chính mô hình tôi đã giả định*. Mô phỏng là *chỗ lý tưởng* cho câu hỏi ấy vì ở đó mọi giả định thoả chính xác. Nếu NEES lệch trong mô phỏng thì bạn có *lỗi cài đặt* — nhằm hiệp phương sai biên với điều kiện, quên Adjoint khi đổi quy ước, sai một hệ số — và đó là loại lỗi im lặng mà không phép đo nào khác bắt được, vì ATE vẫn tốt. Nguyên tắc: *mô phỏng không dùng để chứng minh hệ tốt, mà để tách lỗi cài đặt khỏi lỗi mô hình*.
- A4. Phép kiểm bắt buộc: *baseline của tôi có đạt con số mà bài gốc công bố không?* Nếu không, bạn đang so với một phiên bản bị làm yếu và kết quả không nói gì. Đây không phải cảnh báo lý thuyết mà có lý do cấu trúc: người đề xuất phương pháp mới dành hàng tháng tinh chỉnh nó và vài ngày cài baseline. Quy tắc: dùng mã nguồn gốc khi có; nếu tự cài thì phải tái lập được con số công bố *trước* khi so sánh; nếu không tái lập được thì ghi rõ điều đó thay vì im lặng.
- A5. Phụ thuộc vào *chi phí của một lần thất bại*. Với một bài báo, thất bại nghĩa là một chuỗi bị loại khỏi bảng, nên A thắng. Với một sản phẩm, thất bại nghĩa là robot dừng giữa nhà khách hàng và ai đó phải tới xử lý — và 2% trên hàng nghìn máy mỗi ngày là một dòng chi phí vận hành thật, chưa kể thiệt hại về uy tín. B thắng, và không sát sao. Với mục tiêu sản phẩm ở 00-map, nguyên tắc là *SOTA thật nằm ở đuôi phân bố*: báo cáo phân vị thay vì trung bình, đếm và phân loại thất bại, và thiết kế thí nghiệm nhắm vào các chuỗi khó chứ không chỉ các chuỗi chạy được.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn muốn thuê một tài xế và có hai ứng viên. Cách đơn giản nhất: cho mỗi người lái một quãng, đo thời gian, chọn người nhanh hơn.

Cách đó sai ở ba chỗ, và ba chỗ ấy chính là ba lỗi phổ biến nhất khi đánh giá một hệ định vị.

Thứ nhất, bạn đo trên một quãng đường. Người A nhanh trên cao tốc, người B nhanh trong phố. Một con số duy nhất giấu mất điều đó, và nếu bạn cần người lái trong phố thì bạn vừa chọn nhầm.

Thứ hai, bạn đo trung bình chứ không đo lúc tệ nhất. Người A nhanh hơn mười phần trăm nhưng thỉnh thoảng đâm xe. Người B chậm hơn nhưng chưa bao giờ. Con số trung bình bảo chọn A; bất kỳ ai thật sự đi thuê tài xế đều biết phải chọn B.

Thứ ba, và tinh tế nhất: bạn không hỏi họ có *biết* lúc nào mình đang chạy ẩu không. Một người lái nhanh và biết lúc nào nên chậm lại thì an toàn. Một người lái nhanh và lúc nào cũng nghĩ mình đang ổn thì không. Hai người này có cùng thời gian và cùng số lần đâm xe *cho tới nay* — nhưng họ không giống nhau chút nào, và bạn sẽ biết điều đó vào một ngày không đẹp trời.

Cái thứ ba là cái gần như không ai đo. Nó tương ứng với việc hỏi một cái máy: “mày nói mày sai hai centimet — điều đó có thật không?” Rất ít người hỏi câu đó, và câu trả lời thường là không.

Còn một chuyện nữa, về sự công bằng khi so sánh. Nếu bạn để người A lái chiếc xe quen của anh ta và bắt người B lái một chiếc xe lạ, thì kết quả chẳng nói gì về hai người. Chuyện này xảy ra liên tục trong nghiên cứu, dưới dạng: người đề xuất phương pháp mới tự cài lại phương pháp cũ để so sánh — và dành hàng tháng cho cái của mình, vài ngày cho cái của người khác.

Chỗ không rút gọn được: chỗ không rút gọn được là việc *không có sự thật để so* ngoài phòng thí nghiệm. Trong phòng lab có hệ đo ngoài cho biết chân trị; ở nhà khách hàng thì không. Nên mọi thứ ta đo được ngoài hiện trường đều là dấu hiệu gián tiếp, và việc nối các dấu hiệu ấy với sai số thật là một việc phải làm một lần trong lab rồi mang ra dùng — và rất ít nơi làm.

Một câu: *Đo trước rồi hãy cải tiến — và đo ở chỗ tệ nhất, không ở chỗ trung bình.*

6.6 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *căn chỉnh phải đúng số bậc gauge* — suy từ A/05 (cửa a) và được [39] thiết lập (cửa c). Nó kiểm được trực tiếp: lấy một quỹ đạo, nhân tỉ lệ 1,1, rồi tính ATE với căn $Sim(3)$ và với căn $SE(3)$; nếu con số thứ nhất không đổi thì mệnh đề được xác nhận.

Mệnh đề thứ hai — *NEES trong mô phỏng có giá trị vì nó tách lỗi cài đặt khỏi lỗi mô hình* — là một lập luận về mục đích của phép đo, không phải một sự kiện thực nghiệm. Nó bị bác bỏ nếu trong thực tế, các lỗi cài đặt hiệp phương sai hiểm tới mức phép đo này không đáng công. Tôi cho là ngược lại — tôi ngờ chúng rất phổ biến, dựa trên việc phân biệt biên với điều kiện là một điểm tinh tế — nhưng tôi không có số liệu (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ ba — *SOTA thật nằm ở đuôi* — phụ thuộc vào mục tiêu. Nó đúng cho sản phẩm robot hoạt động không giám sát, là mục tiêu đã nêu ở 00-map. Nó *sai* cho một ứng dụng mà thất bại rẻ và độ chính xác trung bình là thứ được trả tiền — quét đo đặc có người vận hành chẳng hạn, nơi một lần hỏng chỉ tốn một lần quét lại. Nếu rõ điều này vì nguyên tắc ở mục 5D không phổ quát.

Mệnh đề thứ tư — *benchmark không đo truy vấn ngoài bản đồ* — là một quan sát về các bộ dữ liệu hiện có, và nó bị bác bỏ nếu một benchmark mới đưa vào phần đó. Nếu điều đó xảy ra thì đó là tin tốt, và tôi mong nó xảy ra.

6.7 Kết nối

Chương này là chương mà mọi chương trước đều nợ, nên nổi lên trên là nổi tới tất cả.

Cụ thể: A/05-observability-gauge-va-suy-bien cho quy tắc căn chỉnh ở mục 2B — không có chương đó thì quy tắc này trông tùy tiện. A/06-bat-dinh-va-lan-truyen-sai-so cho NEES và cho lý do vì sao tính trung thực quan trọng. B/11-loop-closure-va-nhat-quan-toan-cuc và B/12-relocalization cho hai thành phần có chi phí lỗi bất đối xứng ở mục 4. C/15-hop-nhat-da-cam-bien khối *Cần nhớ* nêu rằng đo một hệ hợp nhất bằng ATE trung bình là bỏ sót phần lớn giá trị của nó — mục 5D ở đây là cách khắc phục.

Nối xuống dưới. D/21-tu-prototype-len-mass-production là chỗ chương này trở thành quy trình: bộ dữ liệu nội bộ phản ánh môi trường khách hàng, regression test tự động cho mỗi commit, ma trận thử nghiệm, và nguyên tắc không bao giờ tin benchmark công khai là đủ. D/22-huong-nghien-cuu-con-trong phụ thuộc vào chương này ở một mức sâu hơn: không thể phân biệt một khoảng trống *thật* với một khoảng trống *tưởng tượng* nếu không có kỷ luật đo lường — và ba trong tám hướng ở đó (bất định đáng tin, dự đoán thất bại, xử lý suy biến có nguyên tắc) chỉ có nghĩa khi ta đo được chúng.

Nối sang research-rules.md: mục 5B và 5C của chương này là mục 7 của tài liệu đó áp dụng cho SLAM. Nếu bạn định nâng một phần của cây này lên thành một dự án khảo sát hoặc nghiên cứu, chương này là cầu nối.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Căn chỉnh mấy bậc cho mono, stereo, VIO — và nói căn quá nhiều bậc che mất điều gì cụ thể.
2. ATE và RPE đo hai thứ khác nhau. Nếu bạn đang cải thiện odometry thì dùng cái nào, và vì sao cái kia sẽ che mất đóng góp của bạn?
3. NEES cần chân trị nên chỉ đo được trong mô phỏng. Vì sao điều đó không phải lý do bỏ qua, và nó bắt được loại lỗi nào mà ATE không bắt được?
4. Vì sao điểm F1 tối đa là tiêu chí sai cho loop closure, và tiêu chí đúng là gì?
5. Nêu ba cách làm sai một ablation study.
6. Hệ A nhanh và thỉnh thoảng hỏng, hệ B chậm và chưa bao giờ hỏng. Câu trả lời phụ thuộc vào gì, và với mục tiêu của cây này thì chọn cái nào?
7. Nêu bốn câu hỏi mà một đánh giá đầy đủ phải trả lời, và nói vì sao ba câu sau thường bị bỏ.

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.

Nguồn

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A Versatile and Accurate Monocular SLAM System,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, 2015, mức tin A.
- [2] M. Dymczyk, S. Lynen, T. Cieslewski, et al., “The Gist of Maps — Summarizing Experience for Lifelong Localization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; nguồn của map summarization, 2015.
- [3] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.
- [4] X. Shi, D. Li, P. Zhao, et al., “Are We Ready for Service Robots? The OpenLORIS-Scene Datasets for Lifelong SLAM,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B; bộ dữ liệu hiếm hoi đo thay đổi môi trường theo thời gian, 2020.
- [5] F. Pittaluga, S. J. Koppal, S. B. Kang, et al., “Revealing Scenes by Inverting Structure from Motion Reconstructions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của mỗi lo map inversion, 2019.
- [6] T. Schneider, M. Dymczyk, M. Fehr, et al., “maplab: An Open Framework for Research in Visual-Inertial Mapping and Localization,” *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, 2018, mức tin B.
- [7] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2016.
- [8] P. Geneva, K. Eickenhoff, W. Lee, et al., “OpenVINS: A Research Platform for Visual-Inertial Estimation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A, 2020.
- [9] J. Maye, P. Furgale, and R. Siegwart, “Self-Supervised Calibration for Robotic Systems,” in *IEEE Intelligent Vehicles Symposium (IV)*, mức tin B; nguồn của ý tưởng chỉ cập nhật hướng quan sát được, 2013.
- [10] T. Schneider, M. Li, C. Cadena, et al., “Observability-Aware Self-Calibration of Visual and Inertial Sensors for Ego-Motion Estimation,” in *IEEE Sensors Journal*, mức tin B, 2019.
- [11] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của khung hiệu chuẩn liên tục theo thời gian bằng B-spline, 2013.
- [12] V. Usenko, N. Demmel, D. Schubert, et al., “Visual-Inertial Mapping with Non-Linear Factor Recovery,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, 2020, mức tin B; hệ thống Basalt.
- [13] T. Qin, P. Li, and S. Shen, “VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator,” *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018, mức tin A.
- [14] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.

- [15] A. Rosinol, M. Abate, Y. Chang, et al., “Kimera: an Open-Source Library for Real-Time Metric-Semantic Localization and Mapping,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin B, 2020.
- [16] H. Hirschmüller, “Stereo Processing by Semiglobal Matching and Mutual Information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008, mức tin A.
- [17] T. Whelan, R. F. Salas-Moreno, B. Glocker, et al., “ElasticFusion: Real-Time Dense SLAM and Light Source Estimation,” *The International Journal of Robotics Research*, vol. 35, no. 14, pp. 1697–1716, 2016, mức tin A.
- [18] E. Sucar, S. Liu, J. Ortiz, et al., “iMAP: Implicit Mapping and Positioning in Real-Time,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2021.
- [19] Z. Zhu, S. Peng, V. Larsson, et al., “NICE-SLAM: Neural Implicit Scalable Encoding for SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2022.
- [20] B. Kerbl, G. Kopanas, T. Leimkühler, et al., “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics*, vol. 42, no. 4, 2023, mức tin A.
- [21] N. Keetha, J. Karhade, K. M. Jatavallabhula, et al., “SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [22] H. Matsuki, R. Murai, P. H. J. Kelly, et al., “Gaussian Splatting SLAM,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [23] A. Hornung, K. M. Wurm, M. Bennewitz, et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013, mức tin A.
- [24] R. A. Newcombe, S. Izadi, O. Hilliges, et al., “KinectFusion: Real-Time Dense Surface Mapping and Tracking,” in *IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, mức tin A, 2011.
- [25] D. DeTone, T. Malisiewicz, and A. Rabinovich, “SuperPoint: Self-Supervised Interest Point Detection and Description,” in *CVPR Workshops*, mức tin B, 2018.
- [26] P.-E. Sarlin, D. DeTone, T. Malisiewicz, et al., “SuperGlue: Learning Feature Matching with Graph Neural Networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2020.
- [27] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, “LightGlue: Local Feature Matching at Light Speed,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin B, 2023.
- [28] J. Sun, Z. Shen, Y. Wang, et al., “LoFTR: Detector-Free Local Feature Matching with Transformers,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2021.
- [29] Z. Teed and J. Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin A, 2021.
- [30] S. Wang, V. Leroy, Y. Cabon, et al., “DUST3R: Geometric 3D Vision Made Easy,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2024.
- [31] V. Leroy, Y. Cabon, and J. Revaud, “Grounding Image Matching in 3D with MAST3R,” in *European Conference on Computer Vision (ECCV)*, mức tin B, 2024.
- [32] J. Wang, M. Chen, N. Karaev, et al., “VGGT: Visual Geometry Grounded Transformer,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin C; rất mới, chưa có tái lập độc lập rộng rãi, 2025.

- [33] S. Yang and S. Scherer, “CubeSLAM: Monocular 3-D Object SLAM,” in *IEEE Transactions on Robotics*, mức tin B, 2019.
- [34] L. Nicholson, M. Milford, and N. Sünderhauf, “QuadricSLAM: Dual Quadrics from Object Detections as Landmarks in Object-Oriented SLAM,” in *IEEE Robotics and Automation Letters*, mức tin B, 2019.
- [35] Z. Teed, L. Lipson, and J. Deng, “Deep Patch Visual Odometry,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2023.
- [36] L. Pineda, T. Fan, M. Monge, et al., “Theseus: A Library for Differentiable Nonlinear Optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, mức tin B, 2022.
- [37] B. Bescos, J. M. Fácil, J. Civera, et al., “DynaSLAM: Tracking, Mapping, and Inpainting in Dynamic Scenes,” *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4076–4083, 2018, mức tin B.
- [38] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của định nghĩa ATE và RPE, 2012.
- [39] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nguồn của quy ước căn chỉnh và drift theo đoạn, 2018.
- [40] M. Grupp, *evo: Python Package for the Evaluation of Odometry and SLAM*, phần mềm mã nguồn mở, mức tin C; công cụ, không phải công trình phản biện, 2017.